Technische Universität Hamburg

Data Science Foundations

Low-Dimensional Control under Embodiment Constraints: From LQR to Reinforcement Learning

Bachelor's Thesis
am Institut Data Science Foundations
Technische Universität Hamburg

vorgelegt von
Alexander Michael Kranz
am
31.10.2026

Erstprüfer:	Prof. Dr. N. Ay
Zweitprüfer	Dr. Manfred Eppe
Betreuung:	Dr. Adwait Datar

Alexander Michael Kranz
geboren am: 09.02.2004
Matrikelnummer: 590156
Studiengang: Data Science

Abstract

This thesis investigates the effects of low-dimensional structures in the control architectures for embodied agents, starting from simple LQR-based solvers and expanding to RL-approaches implemented using proximal policy optimization. As the basis serves the double integrator, which is first artificially overparameterized by having the controller observe and act in a higher dimension and use projection matrices to change between it and the dimension of the underlying system. Afterwards, we also introduce a system with camera-based observations and a more complex actuation structure. Throughout our experiments, we notice that low-dimensional controllers are difficult to implement successfully, and require careful consideration during design. However, if done right, a combination of fixed, briefly pretrained encoders together with a smaller, longer-trained actor can perform similarly to a fully connected one. In some cases we even notice improved robustness to measurement noise, but not under our perception-based experiments. In comparison, a fully trained controller seems to be more robust to control noise.

Zusammenfassung

Diese Arbeit untersucht die Effekte von niederdimensionalen Strukturen in der Regelungsarchitektur von Agenten mit einem Körper. Hierfür beginnen wir mit LQR-basierten Reglern und erweitern unsere Ansätze danach zu Reinforcement Learning, für welches wir Proximal Policy Optimization verwenden. Die Basis bildet der Doppelintegrator, welchen wir künstlich überparametrisieren, indem wir den Regler in einer höheren Dimension beobachten und agieren lassen, und mit Hilfe von Projektionsmatrizen zwischen dieser und der echten Dimension des Systems transferieren. Danach stellen wir ein kamera-basiertes System vor, welches eine komplexere Motorenstruktur besitzt. In unseren Experimenten bemerken wir, dass es schwierig ist niederdimensionale Regler effektiv zu implementieren und in jedem Falle genaue Planung bei dem Aufbau benötigt. Wenn es jedoch gelingt, können Kombinationen aus statischen, vortrainierten Codierern und kleineren, länger trainierten neuronalen Netzen Leistungen erzielen, die vergleichbar mit einem ganzheitlich trainierten Netz sind. In manchen Fällen beobachten wir auch bessere Resistenz gegenüber Messungsstörungen, allerdings nicht in unseren Kamera-basierten Experimenten. Entgegengesetzt scheint ein ganzheitlich trainierter Regler resistenter gegenüber Regelungsstörungen zu sein.

Contents

1 Introduction **1**

2 Background **3**
2.1 Preliminaries . . . 3
2.1.1 Linear State-Space Models . . . 3
2.1.2 The Double Integrator . . . 5
2.1.3 The Linear Quadratic Regulator . . . 6
2.2 Motivation . . . 8
2.3 Experimental Setup . . . 10
2.3.1 The Modified Loop . . . 10
2.3.2 Metrics and Robustness . . . 13
2.3.3 Test Protocol . . . 16
2.4 Approach . . . 18

3 Robustness of LQR Solvers in Different Dimensions **21**
3.1 A Known Model . . . 21
3.1.1 Setting . . . 21
3.1.2 Results . . . 24
3.1.3 Noise Experiments . . . 25
3.1.4 Discussion . . . 27
3.2 Unknown Model across Dimensions . . . 27
3.2.1 Setting . . . 27
3.2.2 Results . . . 30
3.2.3 Noise Experiments on Large Inflation . . . 31
3.2.4 Discussion . . . 34
3.3 Discussion . . . 36

4 Robustness of Reinforcement Learning Approaches **37**
4.1 PPO and Its Variants . . . 37
4.1.1 Proximal Policy Optimization . . . 37
4.1.2 Training Setup . . . 38
4.1.3 Policy Architectures . . . 38
4.2 Comparison across Dimensions . . . 41
4.3 Noise on Large Inflation . . . 43

4.4 Discussion 47

5 Perception-Based Experiments **49**
5.1 Artificial Inflation to Pixel Size 49
5.1.1 Setting 49
5.1.2 Results 50
5.2 The New Environment and the Position-Velocity Encoder 52
5.2.1 Pixelated Environment 52
5.2.2 Position-Velocity Encoder 53
5.3 Effect of Visual Inputs on the Solvers 54
5.3.1 Setting 54
5.3.2 Additional Metrics 55
5.3.3 Results 56
5.4 Noisy Perception 59
5.4.1 Setting 59
5.4.2 Results 60
5.5 Discussion 62

6 Conclusion **63**
6.1 Conclusions 63
6.2 Summary of Contributions 63
6.3 Limitations 64
6.4 Future Research 64

A Nomenclature **65**
A.1 Notation 65
A.2 Noise Intensities 69
A.3 Noise Intensities of the Perception Environment 70
A.4 Policy Architectures 70

B Complete Simulation Results **73**
B.1 Learning Rate of *TrP* 73
B.2 Nuclear-Norm Weight of S-PI 74
B.3 N4SID Identification Length 75
B.4 Sample Size 76
B.5 Learning Rate of *MfN* 78
B.6 Step Size and Nuclear-Norm Weight of *MfS* 78
B.7 Parameters for the RL Solvers 80
B.7.1 Deviations from SB3 Standard Parameters 80
B.7.2 Learning Rates for the RL Solvers 80
B.7.3 Stopping Rule of the RL Solvers 81
B.7.4 Additional Plots for the Perception Experiments 84

Bibliography **85**

List of Figures

3.1 Suboptimality S of the solvers on the true model for each inflation size. The boxes indicate the interquartile range, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show episodes outside this range. 24
3.2 Degradation of the solvers on the true model under *large* inflation and all noise channels at once, over the intensities. 25
3.3 Relative degradation κ of the solvers on the true model under *large* inflation at intensity *divergent*. The boxes indicate the interquartile range over the ten seeds, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show the individual seeds. . 26
3.4 Relative degradation κ of the solvers on the true model under *large* inflation and control noise at intensity *divergent*. The boxes indicate the interquartile range over the ten seeds, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show the individual seeds. 26
3.5 Policy size $|K|$ (Definition 2.1) learned without the model, for each inflation size. 31
3.6 Suboptimality S of the solvers without the model for each inflation size. The boxes indicate the interquartile range, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show episodes outside this range. 32
3.7 Degradation of the solvers without the model under *large* inflation and external disturbance. In (a), the boxes indicate the interquartile range over the ten seeds, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show the individual seeds. . 33
3.8 Degradation of the solvers without the model under *large* inflation and control noise. In (a), the boxes indicate the interquartile range over the ten seeds, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show the individual seeds. 34
3.9 Degradation of the solvers without the model under *large* inflation and measurement noise of increasing intensity. 35
3.10 Position, velocity and control input on the clean loop under *large* inflation. 36

4.1 Training suboptimality $(J - J^*)/J^*$ of the PPO variants under *large* inflation. Values below 0 can occur as the cost J is only estimated from rollouts. 42
4.2 Suboptimality S of the PPO variants for each inflation size. 44
4.3 Degradation of the PPO variants under *large* inflation and parametric uncertainty. 44
4.4 Degradation of the PPO variants under *large* inflation and external disturbance. 45
4.5 Degradation of the PPO variants under *large* inflation and control noise. 46
4.6 Degradation of the PPO variants under *large* inflation and measurement noise of increasing intensity. 46

5.1 Training suboptimality $(J - J^*)/J^*$ of the PPO variants under *pixel* inflation. 50
5.2 Trajectories of the Baseline and the PPO variants under *pixel* inflation from the same initial state. 51
5.3 Suboptimality S of the PPO variants under *large* (Section 4.2) and *pixel* inflation. 52
5.4 Observation and actuation in the camera environment. 53
5.5 Training suboptimality $(J - J^*)/J^*$ of the PPO variants. 57
5.6 Suboptimality S of the PPO variants 58
5.7 Terminated episodes and wasted thrust of the PPO variants. 59
5.8 An observation without measurement noise, and with measurement noise of intensity *standard* and *divergent*. 59
5.9 Absolute degradation $\Delta\hat{J}/\hat{J}^*$ of the PPO variants in the camera environment under parametric uncertainty and external disturbance. 60
5.10 Absolute degradation $\Delta\hat{J}/\hat{J}^*$ of the PPO variants in the camera environment under control and measurement noise. 61

A.1 Actor architectures of our PPO variants. For each layer, the number below indicates its neurons. The critic receives the same encoder structure and uses its own 64-64 tanh perceptron with a scalar output. Visualization created with Claude Code. 71

B.1 Training suboptimality of *TrP* over the solver iterations for each learning rate under *large* inflation. Median and range over ten seeds. 74
B.2 Training suboptimality of *TrS* over the solver iterations for each nuclear-norm weight λ under *large* inflation. Median and range over ten seeds. 74
B.3 Median training suboptimality of *IdP* per trajectory length T and Hankel horizon under *large* inflation. 75
B.4 Median training suboptimality of *IdP* per trajectory length T and Hankel horizon under *medium* inflation. 76

B.5 Training suboptimality of *MfN* over the environment steps for each rollout length l and number of trajectories N_{traj} under *large* inflation. Median and range over ten seeds. 77
B.6 Final training suboptimality of *MfN* for each rollout length l and number of trajectories N_{traj} under *large* inflation. 77
B.7 Training suboptimality of *MfN* over the environment steps for each learning rate under *large* inflation. Median and range over ten seeds. 78
B.8 Training suboptimality for different learning rates for *Lin* under *large* inflation. 81
B.9 Training suboptimality of the RL solvers over the environment steps under *large* inflation. 83
B.10 Example trajectories of the PPO variants. 84

List of Tables

2.1 Preset inflation sizes. 12
2.2 Parameters of the convergence rule (Definition 2.2) for the LQR solvers. 17
2.3 Environment step budget of the RL solvers per inflation size. 18

3.1 Size of the policy architectures on the true model, across every inflation size. 24
3.2 Identification settings per inflation size. 28
3.3 Iterations until convergence of the solvers without the model, as mean and standard deviation over the ten seeds, for each inflation size. 31
3.4 Median relative degradation κ over the ten seeds under *large* inflation and parametric uncertainty, and the range over the seeds at intensity *divergent*. 32

4.1 Stability of the PPO variants for each inflation size, without noise. Stable counts the seeds whose controller kept the plant stable and contained the mass in every test episode, runaway the test episodes (out of 1000) that end with the mass outside the range of initial positions. A linear gain is evaluated by its spectral radius only. . . . 42
4.2 Actor parameters of the PPO variants for each inflation size, split into those trained by PPO and those frozen after pretraining. . . . 43

5.1 Initial learning rate η_0 at the *pixel* size, extrapolated from *large* with (5.1). 50
5.2 Median, best and worst test episode of the suboptimality S of the PPO variants under *pixel* inflation, without noise, pooled over the stable seeds. 52
5.3 Actor parameters of the PPO variants in the camera environment, split into those trained by PPO and those frozen after pretraining. . 57
5.4 Held-out R^2 of a linear probe from the latent state of each PPO variant to the true position, the true velocity and the control of the oracle, together with the final training suboptimality. 58

A.2 Standard deviation per noise channel and intensity, calibrated to the cost ratio κ_ℓ of (2.38). Every value reaches its ratio within 0.9 %. 70
A.3 Standard deviation per noise channel and intensity, calibrated to the cost ratio κ_ℓ of (2.38). Every value reaches its ratio within 9.4 %. Measurement noise is set by hand and not calibrated. 70

B.1 TrP under *large* inflation, mean and standard deviation over the seeds. 73
B.2 MfN per rollout length l and number of trajectories N_{traj} for each inflation size, mean and standard deviation over the stable seeds. . 76
B.3 MfN under *large* inflation, mean and standard deviation over the seeds. Times to 1 % average over the seeds that reach it. Stable counts the seeds whose controller left the plant stable and kept the mass contained in every test episode. 78
B.4 MfS per nuclear-norm weight λ and step size η under *large* inflation, mean and standard deviation over the stable seeds. Every stable run takes 50 iterations and 10^7 environment steps. Stable counts the seeds whose controller left the plant stable and kept the mass contained in every test episode. 79
B.5 Parameters of the PPO solvers that differ from the SB3 defaults. . . 80
B.6 Initial learning rate η_0 of the PPO solvers per inflation size. 80
B.7 Stopping-rule settings with the lowest median regret. 81
B.8 RL runs best policy, as a share of the step budget, and the regret of the policy after 75 % of the budget, of the final policy, and of the policy with the best $\bar{g}_j$. 82

Chapter 1

Introduction

For a robot, our world is a messy place, consisting of a multitude of variables, all influencing what a correct behavior would be at any given moment. Understandably, newer, larger models have led to better (at least theoretical) performance, both in vision [10] and in continuous control [39, 29]. However, when transferring them over from training in a digital environment to an actual embodiment, not only do limitations in memory space and energy usage arise [48, p. 7], but the real world also behaves slightly differently, which the controller needs to be robust against [62, 42]. So ideally, we should remove unnecessary parameters, while still maintaining enough to help us adapt to variable environments. This leaves us with the question: *How does enforcing low-dimensional structures in controllers for embodied agents affect their performance and robustness to noise?*

Inspired by nature, research has shown that complex behavior can be expressed by simple control architectures if combined with the right embodiment [35, 38]. However, finding this simple architecture remains difficult as an embodiment might offer a large number of sensors and actuators, which outnumber the dimension governing the underlying dynamics [64, 30]. Synergizing parameters, like muscles do in nature [7], is thus the goal for a controller. And while low-dimensionality for embodied agents has been studied before [37, 6], the robustness of such controllers has, to our knowledge, not yet been investigated systematically, starting from simple linear controllers, moving on to more complex ones based on reinforcement learning (RL), and ending with perception-based agents acting in simulated environments. This is where we aim to contribute.

To be able to quantify performance and optimality, we rely on a very well-researched example [23, 18]: the linear quadratic regulator (LQR), which, together with some related work, we will introduce in Chapter 2. We use the well-known double integrator [45] as our environment, and artificially inflate it to higher input and output dimensions via projection for our controllers to train on, to simulate overparameterization. In Chapter 3, we compare LQR-based approaches on this system, followed by introducing noise to the environment during testing and comparing

the robustness between the trained controllers. The next step consists of moving from these simpler, gain-matrix-based controllers to RL approaches based on proximal policy optimization [52] (Chapter 4), while still maintaining our optimal controller from before as the comparison. Robustness evaluations are a part of this again. Finally, in Chapter 5, we inflate the system even further to dimensions matching our next experiment. Through a simulated environment, still based on the double integrator, which will be observed by a camera and influenced by multiple differently angled thrusters, we introduce more realism. We again introduce noise in a second step here, which allows us to better understand the behavior of low-dimensional controllers throughout our experiments. We conclude in Chapter 6 with our findings that low-dimensional structures are difficult to implement with a performance gain and often lead to worse results, which, however, could be caused by our controller design. Possible future experiments, related to limitations, are also proposed.

Chapter 2

Background

Before turning to the experiments, we need to lay the groundwork and reasoning for them. We start by defining some important concepts from control theory, as well as introducing the system that serves as our core example throughout (Section 2.1). Then, we discuss the current state of research around embodiment, low-dimensional control, and in that context robustness as well (Section 2.2). We further reason about the relevance of this thesis, as well as comparable papers. After that, we move on to introducing our specifically chosen setup for our experiments, define constants used and explain our test protocol (Section 2.3). Finally, we walk the reader through our upcoming chapters, building on the previously introduced information, and giving reason for certain design choices (Section 2.4).

2.1 Preliminaries

Here, we revisit the concepts that are necessary for understanding the following chapters. This section introduces basics of control theory, such as linear state-space models (Section 2.1.1), the double integrator as our example (Section 2.1.2), and the linear quadratic regulator (Section 2.1.3).

2.1.1 Linear State-Space Models

In control theory, the evolution of an agent together with its environment is described by a *state-space model*: a rule that assigns to every point of a state space the instantaneous rate at which the system moves through it. Throughout this thesis we restrict attention to the class of *linear time-invariant* systems, for which this rule is linear in both the state and the input and does not change over time [67, pp. 3–4].

Such a continuous-time system is given by

$$\dot{x}(t) = A_c\, x(t) + B_c\, u(t), \qquad x(0) = x_0, \tag{2.1a}$$

$$y(t) = C\, x(t) + D\, u(t), \tag{2.1b}$$

where $x(t) \in \mathbb{R}^n$ is the *state vector*, $u(t) \in \mathbb{R}^m$ the *input vector* chosen by the agent, and $y(t) \in \mathbb{R}^q$ the *output vector*, i.e. the output observed and thus available to the feedback controller. Accordingly, $A_c \in \mathbb{R}^{n\times n}$ is called the *system matrix*, $B_c \in \mathbb{R}^{n\times m}$ the *input matrix*, $C \in \mathbb{R}^{q\times n}$ the *output matrix*, and $D \in \mathbb{R}^{q\times m}$ the *feedthrough matrix*.

The two equations in (2.1) play fundamentally different roles, and it is helpful to explain their distinction. Equation (2.1a) is a differential equation: it does not compute the state, which is assumed to be known at every instant, but prescribes its *rate of change*. The pair (A_c, B_c) therefore governs how the system *behaves*, with A_c describing its autonomous dynamics and B_c describing how the input of the agent influences them. Equation (2.1b), in contrast, contains no derivative at all. It is an instantaneous readout, and the pair (C, D) therefore describes how the system is *observed* rather than how it evolves. We set $D = 0$ throughout, which is the standard assumption for systems in which the actuator has no direct instantaneous influence on the sensor [67, p. 4].

Given an input signal $u(\cdot)$, the trajectory of (2.1a) is obtained by integration and is given in closed form by the variation-of-constants formula [67, p. 13, Eq. (2.20)]

$$x(t) = e^{A_c t}x_0 + \int_0^t e^{A_c(t-s)}B_c\,u(s)\,\mathrm{d}s, \tag{2.2}$$

where $e^{A_c t}$ denotes the matrix exponential.

Full-state measurement. Until Section 2.3.1.1, we assume that the state is fully measurable and therefore set $C = I_n$, which reduces (2.1b) to $y(t) = x(t)$. Later, we replace C by a lift matrix U_1 and inject measurement noise (Section 2.3.1.2), such that the previous simplification no longer holds.

2.1.1.1 Discretization

Digital controllers, as in our case, do not act continuously: they sample the output at a fixed rate and hold the computed control input constant until the next sampling point. The learning algorithms studied in this thesis operate on such sampled trajectories as well, so our model has to reflect this. The standard approach is the *zero-order hold* (ZOH) [67, p. 118]. Fixing a sampling period $h > 0$ and writing $x_k \coloneqq x(kh)$, the input is assumed to be piecewise constant [67, Eq. (7.1)],

$$u(t) = u_k \qquad \text{for } t \in [kh,\,(k+1)h), \quad k \in \mathbb{N}_0. \tag{2.3}$$

Substituting (2.3) into (2.2) over a single sampling interval and applying the change of variables $\tau = (k+1)h - s$ yields the discrete-time model

$$x_{k+1} = A_d\,x_k + B_d\,u_k, \tag{2.4}$$

with [67, p. 136, Eqs. (7.26)–(7.28)]

$$A_d = e^{A_c h}, \qquad B_d = \left(\int_0^h e^{A_c\tau}\,\mathrm{d}\tau\right) B_c. \tag{2.5}$$

We use the implementation from SciPy [53] to carry out this transformation in the code. Unless stated otherwise, all experiments in this thesis are carried out in discrete time with sampling period $h = 0.01\,\mathrm{s}$, and we drop the subscript d in the following, writing simply A and B.

2.1.2 The Double Integrator

To make the comparisons across all chapters meaningful, all experiments are built on a single system: the *double integrator*, a frictionless point mass of unit mass moving along a line under an applied force u [45, pp. 87–88]. Its dynamics are Newton's second law,

$$\ddot{p}(t) = u(t), \tag{2.6}$$

where $p(t)$ denotes the position of the mass. We simplify by using a mass of one, so that force and acceleration coincide numerically. Choosing position and velocity as the state, $x = [\,p,\ v\,]^\top \in \mathbb{R}^2$, the second-order equation (2.6) decomposes into the first-order system

$$\dot{p}(t) = v(t), \qquad \dot{v}(t) = u(t), \tag{2.7}$$

in which the first equation is the definition of velocity and the second is (2.6) itself. In the notation of (2.1) this gives

$$A_c = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad B_c = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad C = I_2, \qquad D = 0. \tag{2.8}$$

The entry $(A_c)_{12} = 1$ should be read as stated in Section 2.1.1: the rate of change of position is given by the current velocity.

Because $A_c^2 = 0$, the matrix exponential in (2.5) truncates after two terms, $e^{A_c h} = I + A_c h$, and the discretization is available in closed form:

$$A_d = \begin{bmatrix} 1 & h \\ 0 & 1 \end{bmatrix}, \qquad B_d = \begin{bmatrix} h^2/2 \\ h \end{bmatrix}. \tag{2.9}$$

This system is the minimal nontrivial choice for our purposes. It is second-order, so the state is not directly actuated and the controller must account for momentum. It is *controllable* for every $h > 0$, i.e. an input sequence can navigate it from any initial state to any final state [67, p. 127, Def. 7.2].[1] Additionally, it is not open-loop stable: an unforced discrete-time system $x_{k+1} = Ax_k$ is *stable* if and only if all eigenvalues of A lie strictly inside the unit disk [67, p. 127, Thm. 7.1], i.e. if its *spectral radius* $\rho(A) \coloneqq \max_i |\lambda_i(A)|$ satisfies $\rho(A) < 1$. Here, A_c has a double eigenvalue at the origin and A_d correspondingly a double eigenvalue at 1, so $\rho(A_d) = 1$. The last property matters for the learning experiments: an uncontrolled or badly initialized policy diverges, so the assumption of a known stabilizing initial gain (Section 2.3.3) is a restriction.

[1] Controllability, with its weaker form stabilizability, and the dual notions observability and detectability are standard structural properties of linear systems. If the reader wishes to read up on their formal definitions, we kindly refer them to [67, Chs. 3–4 and pp. 127–129].

2.1.3 The Linear Quadratic Regulator

Now that we have defined our state-space model and the system we study, it is time to introduce our controller, which is based on the *linear quadratic regulator* problem.

Here, the setting is that our system starts at an arbitrary initial state in its state space, and the goal of our controller is to return to the zero state as cheaply as possible. Both distance to the origin, as well as applied control input yield certain costs, which can be combined into a single cost function, where Q describes the former, while R associates costs to control input. Over a finite horizon T, this gives the cost [68, pp. 5–6, Eqs. (1.3) and (1.5)]

$$V_T(x_0, u) = \int_0^T \left(x(t)^\top Q\, x(t) + u(t)^\top R\, u(t)\right) \mathrm{d}t, \qquad Q = Q^\top \succeq 0, \quad R = R^\top \succ 0. \tag{2.10}$$

We notice two further restrictions here: Q is required to be positive semi-definite, meaning that deviating from the origin never reduces the cost, and R must be positive definite, such that every control input results in a cost.

As the original formula relates to a continuous problem with a fixed end time T, it does not fit well with our goal of optimizing over an infinite discrete horizon. The following formula offers exactly that [67, p. 137, Eq. (7.30)]:

$$J(x_0, u) = \sum_{k=0}^{\infty} \left(x_k^\top Q\, x_k + u_k^\top R\, u_k\right). \tag{2.11}$$

This sum does not necessarily converge. If the closed loop does not decay, the costs do not vanish and J tends to $+\infty$. Using this as our definition, J becomes well defined for every gain.

2.1.3.1 The Optimal Gain

What makes the LQR problem special is that its solution is a static linear feedback law [46, Sec. 4]: the optimal input at time k depends only on the current state of the model. Introducing a gain matrix $K \in \mathbb{R}^{m\times n}$, we write it as[2]

$$u_k = -K x_k. \tag{2.12}$$

Our controller is thus reduced to choosing a matrix, and the optimal controller is given by the optimal gain matrix K^*.

As we are dealing with a discrete-time system, that gain is obtained from the *discrete algebraic Riccati equation* (DARE) [46, Sec. 4, Eq. (4.2)]

$$P = A^\top P A - A^\top P B \left(R + B^\top P B\right)^{-1} B^\top P A + Q, \tag{2.13}$$

[2]Note that the minus in front of K has been chosen as the convention for this thesis. This turns our closed loop into the format $A - BK$, while some referenced literature uses $A + BK$. Formulas cited have been adapted to our convention by replacing K with $-K$.

whose stabilizing solution $P = P^\top \succeq 0$ yields [46, Sec. 4]

$$K^* = \left(R + B^\top P B\right)^{-1} B^\top P A, \qquad u_k^* = -K^* x_k. \tag{2.14}$$

The matrix P is additionally our *cost-to-go* matrix [14, Sec. 1, Eq. (1)], meaning that we can use it to calculate the optimal cost incurred from any state x onwards with

$$J^*(x) = x^\top P x. \tag{2.15}$$

Whether (2.13) has such a solution at all depends on two structural properties of the system, stabilizability and detectability [28, p. 442, Thm. 8]:

Theorem 2.1 (Existence and uniqueness)**.** *Let (A, B) be stabilizable and let $(A, Q^{1/2})$ be detectable. Then the DARE* (2.13) *has a unique symmetric positive semi-definite solution P and the resulting closed loop satisfies $\rho(A - BK^*) < 1$.*

For the double integrator, both conditions hold: (A, B) is controllable and thus stabilizable, and with $Q = I_n$ (Section 2.3.3) the pair $(A, Q^{1/2}) = (A, I_n)$ is also observable. Thus K^* is well defined in all our experiments.

2.1.3.2 The Cost as a Function of the Gain

Equation (2.14) solves the problem, but it solves it directly and tells us nothing about the quality of non-optimal gains. As the learning algorithms of the following chapters will produce such K directly, we need to be able to calculate the cost as a function of K.

Fix a gain K and write the resulting closed loop as $x_{k+1} = A_K x_k$ with $A_K \coloneqq A - BK$. Substituting (2.12) into (2.11) shows that the cost is again a quadratic form, $J(x_0, K) = x_0^\top P_K x_0$, where P_K now solves the far simpler discrete *Lyapunov equation* [14, Sec. 2.1]

$$P_K = A_K^\top P_K A_K + Q + K^\top R K, \tag{2.16}$$

which is linear in the unknown P_K and has a unique solution whenever $\rho(A_K) < 1$. This restricts us to the set of stabilizing gains

$$\mathcal{K} \coloneqq \left\{ K \in \mathbb{R}^{m \times n} : \rho(A - BK) < 1 \right\}, \tag{2.17}$$

outside of which we set the cost to $+\infty$, consistent with the remark after (2.11).

One dependency remains, namely on the initial state. Evaluating the gain based on a single x_0 would reward gains that happen to fit that one point, so we instead draw the initial state from a distribution, $x_0 \sim \mathcal{D}$, and average [14, Sec. 3]. Writing $\Sigma_0 \coloneqq \mathbb{E}_{x_0 \sim \mathcal{D}}[x_0 x_0^\top]$ for its second moment, the objective [14, Sec. 2.1] becomes

$$f(K) \coloneqq \mathbb{E}_{x_0 \sim \mathcal{D}}\left[x_0^\top P_K x_0\right] = \operatorname{tr}\left(P_K \Sigma_0\right), \tag{2.18}$$

using the cyclic property of the trace. Equation (2.18) is the objective that the algorithms in Chapter 3 minimize and they differ only in how much of the model is known to them. Later, in Chapters 4 and 5, the RL approaches rely on a rollout-based estimate of it to optimize over.

2.2 Motivation

An agent acting in its environment receives observational input and responds to it with a commanded control input. In other words, the agent is a function in relation to its environment, and to find an optimal behavior, we need to approximate this optimal function as closely as possible. While neural networks are known to be universal approximators [5, 16], and hence able to approximate any function, in the case of an embodied agent, different constraints need to be taken into account. Ay [3, p. 392] names additionally to external embodiment constraints possible intrinsic constraints, and argues that such can even be "beneficial" for the control structure. By the core notion of cheap design, embodiment can help produce complex behaviors while only needing smaller controllers [43], which inherently requires constraints. So, for such embodied agents, the goal is to find an "embodied universal approximator" [37, Sec. 1], which restricts the neural network architecture to an intrinsic, unknown dimension, encoding physical properties belonging to the agent and the environment, while disallowing impossible movements [37, Sec. 3].

Biology motivates this approach even further: here it is clear to see that by commanding a body, some features of the control architecture can be simplified and delegated to the embodiment [38]. For example, McGeer's passive walker [35] clearly shows that some movements can be produced by the body without any control input, just by choosing the right embodiment. While obviously here being simplified to not include any control at all, it can quite simply be extended with such [34]. Müller and Hoffmann [38, p. 4] mention such a combination, a robot climbing up walls, whose design was inspired by geckos, indeed using energy for control, however very little compared to an approach not exploiting this specific embodiment design. Further, it can be seen that the complex behaviors are not only less energy-intensive to produce [38, p. 4], but also utilize muscle synergies [7] and thus fewer control parameters. And while it is difficult to clearly attribute the effect of the embodiment [70], for us, this gives reason to believe that the simplicity, and thus efficiency, comes from a low-dimensionality of the controller [37].

That this is indeed true for certain examples has been shown both in theory and practice before [6, 36, 32]. Li et al. [32] call this lower dimension "intrinsic dimension", which is the smallest dimension of a parameter subspace in which the model still achieves comparable metrics. They show, for example, that for achieving 90 % of the performance of a fully connected network on MNIST, the dimensionality can be reduced by 99.6 %. For a convolutional neural network, it is still a 99.3 % reduction. Others show that the "least complex model which is able to learn an optimal policy does learn faster" [36, Sec. 6]. For a neural network, being more parameter-efficient can be done in different ways, some of which we look at in Chapter 4. In comparison, for LQR, where the controller is a gain matrix, the corresponding approach is reducing its rank, for example via a nuclear-norm regularization [47]. However, while these approaches are more parameter-efficient at achieving similar or slightly worse performances, it was also observed that they tend to perform even worse when encountering noise [6].

Robustness to noise, however, is one of the important characteristics of controllers acting under real-world conditions [45, 48]. One approach to achieving this in nature is the principle of "redundancy and diversity" [25, p. 828], which stands in direct contrast to our goal of reducing redundant parameterizations. Another reason for producing robust controllers is the difficult transfer from a digital training environment to real-world embodiments, commonly referred to as the "reality gap" [62, 42]. Approaches to training robust controllers typically introduce noise already during training, be it by using a linear quadratic Gaussian (LQG) controller or by including variations to the dynamics [45, 42]. These approaches, however, do not look at low-dimensionality, which makes sense as Kitano [25, p. 833] proposes that robustness cannot be increased without sacrificing performance or accepting "increased resource demands".

Before taking a look at the more complex case of RL, we start simpler by using the simpler and better-understood LQR. This approach has been taken before [46] and does make sense given the guarantees regarding convergence and effectiveness of policy optimization for it [14, 60], as well as the understanding of its geometry [61]. It also offers methods which produce certain structures in its controller, like sparsity or our desired low-dimensionality, as given by structured policy iteration (S-PI) [41]. Approaches can generally be categorized into two groups if the underlying dynamics are unknown and only data is available: estimating the model and then applying a model-based solver (indirect), or using the data to estimate the gain matrix (direct) [72]. Both the indirect, as in system identification with uncertainty bounds [8], and the direct setting, with an $\mathcal{H}_\infty$ constraint that bounds the closed-loop gain from disturbance to output [71], have been studied with regard to robustness, but, to our knowledge, not in combination with low-rank policies.

Research evaluating RL on control tasks has also shown that there is room for simplification, for example using linear policies instead of complex, fully connected neural networks [33, 44]. This, together with the fact that parameters in neural networks are highly dependent, and thus redundant [11], allows us to assume that low-dimensionality can improve efficiency here. Examples of this exist, for example by factorizing a discretized low-rank policy into a thin and wide matrix [49] or by introducing an orthogonal projection inside the layer structure [63]. As a result, the dimensionality of the policy is reduced. A different approach uses "information bottlenecks", which exhibit better generalization behavior in unseen environments [20, Sec. 4.2], which again could help us with robustness.

A clear case where low-dimensional dynamics underlie a high-dimensional observational space is in perception-based tasks. Agents observing only images instead of the states of simple control tasks are faced with the challenge of extracting these states from a high-dimensional, often nonlinear encoding [66, Sec. 1]. Many approaches, based on state representation learning (SRL), exist, which use different variants to create a latent embedding of the image, used as a state [30]. The most interesting approach, especially for us looking at embodied agents, is the usage of so-called "priors", which restrict the latent space by using knowledge about the

environment [30]. This is similar to Ay's argument that the embodiment actively defines the action space [3]. Robotic priors were introduced by Jonschkowski and Brock [21] in 2015, and then further refined by Botteghi et al. [4] for continuous action spaces. A more specific version restricted to position and velocity also exists here, training a model to encode an image into a low-dimensional position state, from which the velocity is calculated by finite differences [22]. However, perception-based control is also affected by the problems of robust control, as noise induced through perception directly affects the performance [69] and therefore requires approaches for mitigating them [9].

Our goal for this thesis is therefore to look a bit deeper at the combination of low-dimensionality and robustness in an empirical way, motivated by the findings of Datar and Ay [6]. LQR serves as the perfect, well-investigated basis [46], offering low-rank approaches that have been proposed and analyzed theoretically, but have not yet been applied in practice and under noise [41, Sec. 4]. Previously investigated policies for LQR under noise in comparison did not look at low-dimensionality specifically [71]. For RL, low-dimensionality has been explored previously [1, 59], and also in comparison with robustness [31], but this research does not take into account the embodiment we are interested in. Closing this gap, our main interest is to compare robustness properties for low-dimensional policies for embodied agents under increasingly more complex and noisier environments.

2.3 Experimental Setup

We now introduce with Section 2.3.1 the modifications we apply to the double integrator loop for our experiments, followed by the metrics in Section 2.3.2 we evaluate it on. The section is concluded with Section 2.3.3, where we state our evaluation procedure.

2.3.1 The Modified Loop

2.3.1.1 An Inflated Model

To allow us to investigate a controller that has many redundant inputs and outputs, acting on a system with low-dimensional dynamics, we first need to create such a setup for our previously introduced double integrator. We do this by keeping the dynamics, so the state progression, like the loop in Section 2.1.2, but projecting the observation into a higher dimension, as well as projecting actuation down from such.

More specifically, we show the modification to our loop with two random orthonormal matrices U_1 and U_2. For an inflated input dimension d_{in} and output dimension d_{out}, the two lifts are constructed the following way:

$$U_1 \in \mathbb{R}^{d_{\text{in}} \times n}, \quad U_1^\top U_1 = I_n, \qquad U_2 \in \mathbb{R}^{m \times d_{\text{out}}}, \quad U_2 U_2^\top = I_m, \tag{2.19}$$

where U_1 is the orthonormal factor of the QR decomposition of a random matrix, whose entries are sampled from the standard normal distribution. U_2 is the transpose of such a factor sampled from a separately drawn matrix. Both are drawn once per seed, and then kept fixed throughout. A single step of the loop then becomes

$$\begin{aligned} y_k &= U_1 x_k, && \text{(2.20a)} \\ v_k &= -K y_k, \qquad K \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, && \text{(2.20b)} \\ u_k &= U_2 v_k, && \text{(2.20c)} \\ x_{k+1} &= A x_k + B u_k = A x_k + B U_2 \, v_k. && \text{(2.20d)} \end{aligned}$$

Compared with (2.1), the output matrix thus becomes $C = U_1$, so that $q = d_{\text{in}}$, and the controller acts through the input matrix $BU_2 \in \mathbb{R}^{n \times d_{\text{out}}}$.

This way, the controller only sees the inflated observation and acts in a higher dimension. As any actuation effort is penalized, its weight $\tilde{R}$ acts on the inflated actuation and therefore has the dimension $d_{\text{out}} \times d_{\text{out}}$. To simplify things, we also have the state weight $\tilde{Q}$ in this lifted dimension, acting on the observation. Here and in the following, a tilde marks the inflated counterpart of a quantity of the uninflated problem. As U_1 has orthonormal columns, and we use $\tilde{Q} = I_{d_{\text{in}}}$ throughout, this simplifies to

$$\begin{aligned} (U_1 x_k)^\top \tilde{Q} \, (U_1 x_k) &= x_k^\top U_1^\top U_1 \, x_k \\ &= x_k^\top I_n \, x_k \\ &= x_k^\top Q \, x_k, \qquad Q = I_n, \end{aligned} \tag{2.21}$$

with the uninflated Q of Section 2.1.3, so it indeed still penalizes how far our mass is from zero. The cost in the modified loop then becomes

$$\begin{aligned} J(x_0, K) &= \sum_{k=0}^{\infty} \left(y_k^\top \tilde{Q} \, y_k + v_k^\top \tilde{R} \, v_k \right) \\ &= \sum_{k=0}^{\infty} \left(x_k^\top U_1^\top \tilde{Q} \, U_1 x_k + x_k^\top U_1^\top K^\top \tilde{R} K U_1 x_k \right), \end{aligned} \tag{2.22}$$

with the closed loop $x_{k+1} = (A - BU_2 K U_1) \, x_k$.

Throughout the thesis, we use the following synonyms for different d_{in} and d_{out} dimensionalities, applied to the loop, which can be found in Table 2.1.

2.3.1.2 Noise Injection

Now, the second thing we want to take a look at is robustness to small changes or variations in the environment. We thus introduce four different types of disturbances to be included in our model, commonly known as noise. They are distinguished by the point at which they affect the loop: what the controller *observes*

Table 2.1: Preset inflation sizes.

Name	d_{in}	d_{out}
base	2	1
small	10	5
medium	50	25
large	200	100
pixel	18 432	256

(measurement noise), what its output actually *does* (control noise), what the plant *is* (parametric uncertainty), and what else *acts* on it (external disturbance). Each of them can be activated separately, so that we can clearly isolate the cause of an effect. Throughout this section k indexes the sampling instants, and σ denotes the standard deviation, or amplitude of the respective noise channel.

2.3.1.3 Measurement Noise

Here we influence the sensors used to measure the model state, such that capturing the same state twice does not lead to the same observation y_k.

Let n_k^{m} denote the measurement noise at step k. Then

$$n_k^{\text{m}} \sim \mathcal{N}\big(0, \sigma_{\text{m}}^2 I_{d_{\text{in}}}\big)\,, \qquad y_k = U_1 x_k + n_k^{\text{m}}, \qquad \sigma_{\text{m}} \in \mathbb{R}_+, \tag{2.23}$$

drawn independently at every step.

2.3.1.4 Control Noise

Control noise, as the name says, acts on the applied control input. With it, we try to model the difference between the intended movement of an actuator and its actual action due to different factors. Here we deal with *multiplicative* noise, which scales with the input. In detail this means that the applied controller output $\tilde{v}_k$ is obtained as

$$n_k^{\text{c}} \sim \mathcal{N}\big(0, \sigma_{\text{c}}^2 I_{d_{\text{out}}}\big)\,, \qquad \tilde{v}_k = (\mathbf{1}_{d_{\text{out}}} + n_k^{\text{c}}) \odot v_k, \tag{2.24}$$

where $\odot$ denotes the elementwise product, so that σ_{c} is the *relative* error. Applying this to the input, the next step becomes

$$x_{k+1} = A\, x_k + B U_2\, \tilde{v}_k. \tag{2.25}$$

2.3.1.5 Parametric Uncertainty

This noise acts directly on the matrices of the model. Here, we either decide to keep it time invariant, meaning $\Delta_k^A = \Delta_0^A$ and $\Delta_k^B = \Delta_0^B$ for all k, or to resample it at every step. The former models a wrongly identified system, while the latter

simulates one that continuously changes. As with measurement noise, it is additive. Applied, it acts on the system the following way:

$$\left(\Delta_k^A\right)_{ij}, \left(\Delta_k^B\right)_{ij} \overset{\text{i.i.d.}}{\sim} \mathcal{N}\left(0, \sigma_{\text{p}}^2\right), \qquad x_{k+1} = \left(A + \Delta_k^A\right) x_k + \left(B + \Delta_k^B\right) U_2 \, \tilde{v}_k. \tag{2.26}$$

Worth noting is that resampling every step leaves the model correct *on average*, since $\mathbb{E}[\Delta_k^A] = \mathbb{E}[\Delta_k^B] = 0$, and mainly adds variance to the trajectory. This stands in contrast to the time-invariant version, where we have a discrepancy between what we compute actions on and what actually happens.

2.3.1.6 External Disturbance

This noise models external influences, like a wind pushing on our agent, or rain hitting it with some frequency. It only affects the position, so with the first unit vector $e_1 \in \mathbb{R}^n$ and σ_{d} as the noise amplitude, we get

$$n_k^{\text{d}} = \sigma_{\text{d}} \sin(0.01\, k)\, e_1. \tag{2.27}$$

As the disturbance acts on the true state x_k, it is applied before any lifting of the observation (2.20a). The loop can thus be described as

$$x_{k+1} = A\, x_k + B U_2\, \tilde{v}_k + n_k^{\text{d}}. \tag{2.28}$$

2.3.1.7 The Full Loop

While we, until now, have only regarded each noise type on its own, there is also the possibility to include them all at once. Combining all the channels with the inflated loop, a single step becomes

$$y_k = U_1 x_k + n_k^{\text{m}}, \tag{2.29a}$$
$$v_k = -K y_k, \tag{2.29b}$$
$$\tilde{v}_k = (\mathbf{1}_{d_{\text{out}}} + n_k^{\text{c}}) \odot v_k, \tag{2.29c}$$
$$x_{k+1} = \left(A + \Delta_k^A\right) x_k + \left(B + \Delta_k^B\right) U_2\, \tilde{v}_k + n_k^{\text{d}}. \tag{2.29d}$$

2.3.2 Metrics and Robustness

In this thesis we try to answer two main questions. First, how well does a trained controller perform generally, as well as under influences unknown to it during training? Secondly, what influence does its size have on its performance, and what defines sizes? Thus we aim to introduce the metrics here to quantify these questions and our results.

2.3.2.1 Cost and Suboptimality

By Section 2.1.3.1 no gain achieves a lower cost (2.18) than K^* on the clean loop, and every controller we train is therefore in the best case only as good as it unless

noise is involved. This serves as our baseline, and we evaluate the cost produced by our controllers in relation to it.

While we can easily calculate the infinite-horizon cost of a gain via the Lyapunov equation (2.16), we can only apply it to a closed-loop system. However, when adding noise to it, this is not the case anymore. Additionally, the neural networks we train in later chapters do not admit a gain-matrix-based policy K, which would not allow cross-chapter comparison.

So, we instead evaluate them over a fixed horizon via rollout. Hence for a given controller, a noise condition c, which is a subset of channels and an intensity (Section 2.3.2.4), and a horizon T, we simulate starting from a randomly drawn $x_0 \sim \mathcal{D}$. The resulting loop (2.29) under c thus accumulates

$$J_T^{(i,s)}(K;c) = \sum_{k=0}^{T-1} \left((U_1 x_k)^\top \tilde{Q}\, (U_1 x_k) + \tilde{v}_k^\top \tilde{R}\, \tilde{v}_k \right), \tag{2.30}$$

where s indexes the seed the run was trained under and i the episode within it.

It is important to note where $\tilde{Q}$ and $\tilde{R}$ act. As $\tilde{Q}$ penalizes the difference from the origin, we apply it on the inflated state *before* any noise is applied. This way we can evaluate the actual position of the mass, as shown in (2.21). For $\tilde{R}$, we handle it differently. Here, every actuation energy adds to the cost, irrespective of its origin (so whether the controller commanded it or it is noise).

We first define the set of pairs (s,i) whose episode stayed finite under both the trained controller and the baseline,

$$\mathcal{F}_S \coloneqq \left\{ (s,i) \;:\; J_T^{(i,s)}(K;c) < \infty,\; J_T^{(i,s)}(K^*;c) < \infty \right\}. \tag{2.31}$$

Now we can state our first main metric, which is the relative excess of the cost produced by a trained controller over the cost of the optimal controller K^* under the same condition. It is taken *per episode*, meaning an episode (s,i) of a trained controller and episode (s,i) of the baseline are compared here. Both start from the same x_0 and are affected by the same noise. Overall this yields

$$S^{(i,s)}(K;c) = \frac{J_T^{(i,s)}(K;c) - J_T^{(i,s)}(K^*;c)}{J_T^{(i,s)}(K^*;c)}, \qquad \hat{S}(K;c) = \frac{1}{|\mathcal{F}_S|} \sum_{(s,i)\in\mathcal{F}_S} S^{(i,s)}(K;c), \tag{2.32}$$

where $S = 0$ means that the trained controller performed as well as our baseline and $S = 0.1$ that it did $10\,\%$ worse.

2.3.2.2 Robustness to Noise

Before introducing our second main metric, we need to define

$$\hat{J}_T(K;c) = \frac{1}{|\mathcal{F}|} \sum_{(s,i)\in\mathcal{F}} J_T^{(i,s)}(K;c), \qquad \mathcal{F} = \left\{ (s,i) \;:\; J_T^{(i,s)}(K;c) < \infty \right\}, \tag{2.33}$$

which is the cost averaged over all episodes. In the following we will also use its variation $\hat{J}_T^{(s)}(K;c)$, where we pool over all episodes of a seed s.

Our second metric relates to robustness, and helps us evaluate how much a controller is influenced by a noise condition c. It is the ratio of the test cost under noise c to the test cost under the clean loop, which we from now on will call *relative degradation*. Formalized, this gives

$$\kappa^{(s)}(K;c) \coloneqq \frac{\hat{J}_T^{(s)}(K;c)}{\hat{J}_T^{(s)}(K;\varnothing)}, \tag{2.34}$$

where $\hat{J}_T^{(s)}$ is (2.33) containing only the episodes of seed s. Thus, the closer $\kappa^{(s)}(K;c)$ is to 1, the more robust a controller is. We can also compare the relative degradation to the performance of the optimal gain, which we calibrated in (2.38) to specific ratios, by taking the ratio of the costs averaged over all seeds,

$$\kappa(K;c) \coloneqq \frac{\hat{J}_T(K;c)}{\hat{J}_T(K;\varnothing)}, \tag{2.35}$$

for which $\kappa_\ell = \kappa(K^*;c_\ell)$ holds by (2.38). As a result, this gives us the possibility to compare whether a specific trained controller is more or less robust than the baseline, or any other controller.

However, this metric is misleading when comparing a controller which has performed very badly on the clean loop with one that performed well. If we assume that both achieve the same $\kappa(K;c)$, this would equate to a much larger absolute increase in the test cost for the former, compared to the latter. To quantify this, we introduce the following metric,

$$\frac{\Delta\hat{J}}{\hat{J}^*}(K;c) \coloneqq \frac{\hat{J}_T(K;c) - \hat{J}_T(K;\varnothing)}{\hat{J}_T(K^*;\varnothing)}, \tag{2.36}$$

and call it *absolute degradation*, indicating the increase in cost caused by noise, in units of the optimal clean cost. For K^*, it equals $\kappa_\ell - 1$, i.e. 0 on the clean loop.

2.3.2.3 The Size of a Policy

We have often mentioned "low-dimensional" controllers until now; however, we want to make the number of parameters comparable, not only between different LQR controllers, but also across LQR and RL. So it is indeed a question of size. For a neural network we can simply count the number of weights and biases of the actor. For a gain matrix, however, we need to find a definition that is comparable to it.

The simplest approach for a $K \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the number of its entries, $d_{\text{out}} \cdot d_{\text{in}}$. In Chapter 3, however, we reduce the rank of K. Optimizing the storage size for K of rank r, the singular value decomposition [68, p. 85, Thm. 13.1] separates it into

$$K = U\Sigma V^\top, \qquad U \in \mathbb{R}^{d_{\text{out}} \times r}, \quad \Sigma = \operatorname{diag}(\sigma_1, \ldots, \sigma_r), \quad V \in \mathbb{R}^{d_{\text{in}} \times r}, \tag{2.37}$$

with U and V having orthonormal columns and $\sigma_1 \geq \cdots \geq \sigma_r > 0$ being the non-zero singular values. Hence, we only need to store $d_{\text{out}} \cdot r$ entries for U, r for the diagonal of Σ and $r \cdot d_{\text{in}}$ for V, which in total yields $r(d_{\text{out}} + d_{\text{in}} + 1)$. For a small enough r this is less, so we take the minimum of both:

Definition 2.1 (Policy size)**.** The size of a linear policy $K \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ of rank $r = \text{rank}(K)$ is

$$|K| \coloneqq \min\left\{d_{\text{out}} \cdot d_{\text{in}},\ r\,(d_{\text{out}} + d_{\text{in}} + 1)\right\},$$

the number of real numbers required to store it. The size of a neural policy is its total number of parameters in the actor, counting both fixed and trainable ones.

2.3.2.4 Noise Intensity

Section 2.3.1.2 introduced four channels which change the system compared to the base model. Each noise channel is separated further into five intensities,

$$\textit{none} \prec \textit{weak} \prec \textit{standard} \prec \textit{strong} \prec \textit{divergent},$$

where *none* leaves the system as it is and thus serves as our control condition. All the other intensities are calibrated by how much they increase the cost of the optimal gain K^* on the uninflated system. For each channel and intensity ℓ, we choose the standard deviation in a way such that for the condition c_ℓ under the test protocol of Section 2.3.3

$$\hat{J}_T(K^*; c_\ell) = \kappa_\ell\, \hat{J}_T(K^*; \varnothing), \qquad \kappa_\ell \in \{1.2,\ 1.5,\ 3,\ 10\} \tag{2.38}$$

applies for *weak*, *standard*, *strong* and *divergent* respectively. For K^*, the increase stays the same under inflation for all channels except control noise. As it acts elementwise on the inflated output, it is averaged out over the number of actuators: *strong* control noise raises the cost of K^* by a factor of 3.0 without inflation, but only by 1.22 and 1.05 at the sizes $d_{\text{in}} \times d_{\text{out}} = 50 \times 25$ and 200×100. For simplicity, however, we do not vary the levels by size, but instead accept this as a side effect. The concrete standard deviations per channel are listed in Table A.2.

2.3.3 Test Protocol

Having introduced our metrics, we now define a protocol for how we train and compare the different solver approaches with these metrics.

First, we note that we always use $\tilde{Q} = I_{d_{\text{in}}}$ and $\tilde{R} = I_{d_{\text{out}}}$ (so $Q = I_n$ and $R = I_m$ on the uninflated system), which weighs position, velocity and control effort equally. Initial states are drawn uniformly from the box $[-10, 10] \times [-1, 1]$. Some solvers require an initial stabilizing gain, which for the uninflated system is $K_0 = [\,0.1,\ 0.5\,]$, whose closed loop has $\rho(A - BK_0) \approx 0.9975$. On an inflated system, we use

$$\tilde{K}_0 = U_2^\top K_0 U_1^\top + K_\perp, \qquad U_2 K_\perp U_1 = 0, \qquad \|K_\perp\|_F = 0.5. \tag{2.39}$$

We create the random matrix $K_\perp$ such that $U_2 K_\perp U_1 = 0$, and thus adding it does not change the stability property of the matrix. This way, the effective gain remains $U_2 \tilde{K}_0 U_1 = K_0$, while $\tilde{K}_0$ is full rank, which offers the possibility of rank reduction.

For each algorithm, we train ten different controllers, each under a different fixed seed, until convergence. We define convergence for LQR solvers as follows:

Definition 2.2 (Convergence)**.** Let g_j be the signal that a solver observes at iteration j, and let $\bar{g}_j$ be its mean over the last N_w iterations. The solver has *converged* at the first iteration j for which

$$\bar{g}_j \geq (1 - \varepsilon_\mathrm{rel})\, \bar{g}_{j-N_\mathrm{p}},$$

i.e. once the average has not decreased by the relative tolerance ε_rel within the last N_p iterations. After an iteration with an infinite signal (caused by $K \notin \mathcal{K}$, (2.17)), the comparison restarts, such that we never compare with a window across an infinite iteration.

The signal is something the solver itself observes and optimizes for, like the cost (2.18), an estimate, or a regularized version of it for solvers introducing structure. Some also have additional early-stopping criteria. They will be introduced in detail in Chapter 3.

For all of them, we use the best iterate by $\bar{g}_j$ to identify the policy we want to use.

The parameters of Definition 2.2 for the model-based and the model-free solvers are listed in Table 2.2.

Table 2.2: Parameters of the convergence rule (Definition 2.2) for the LQR solvers.

Parameter	Model-based solvers	Model-free solvers
Window N_w	1	50
Patience N_p	50	100
Relative tolerance ε_rel	10^{-4}	10^{-3}

The number of steps the plant progressed and solver iterations until convergence are also counted and reported. An upper cap of 20 000 iterations is set to prevent looping.

For RL-based algorithms we do not use such early-stopping logic. Following our experiments in Section B.7.3, we observed that implementing such a rule would lead to worse policies in the end. Instead, the best policy is achieved close to the end, which leads us to running each algorithm for as many steps as noted in Table 2.3, and then taking the final policy. Every 25 000 environment steps throughout, we evaluate the current policy's performance, to track its training process.

Table 2.3: Environment step budget of the RL solvers per inflation size.

	small	*medium*	*large*	*pixel*
Budget (environment steps)	10^7	$1.5 \cdot 10^7$	$3 \cdot 10^7$	10^{7*}

*In the camera environment of Section 5.2, one step corresponds to one decision of the agent, which is then held for 30 environment steps. There, the step budget is 10^7 agent steps, which equals $3 \cdot 10^8$ environment steps.

After our controllers have been trained, we evaluate them per seed for *100 episodes* each, running for $T = 2000$ *steps* per episode with a sampling period of $h = 0.01\,\mathrm{s}$. For each episode we calculate the empirical cost (2.30) and its suboptimality (2.32) in comparison to the optimal gain run with the same initial state $x_0 \sim \mathcal{D}$ and sampled noise. In the end, for each solver architecture, we get a distribution of suboptimalities over a total of 1000 trajectories.

The inflated size is always the same between training and evaluation. For noise experiments, however, the controller is trained on a clean system, and then exposed to one with noise activated during testing.

On the clean system, some trained controllers might not be able to stabilize the system, and are therefore not functional. Those are therefore removed from the metric calculations and instead reported separately. To define a diverging controller, we differentiate two cases:

1. Linear gain: the controller becomes unstable, i.e. $\rho(A - BU_2KU_1) \geq 1$.
2. Neural network: the controller in seed s is not able to control the mass, i.e. contain it in the range of initial positions at the end of any of its episodes, formally once $|p_T^{(i,s)}| > 10$ for some episode i.

Under the stronger noise conditions, individual episodes might still diverge, even though their controller was stable before. To keep our mean finite, we exclude them from the calculation, and report them separately as the divergence rate.

$$D(K;c) \coloneqq \frac{1}{|\mathcal{S}|\,N} \left| \left\{ (s,i) \;:\; J_T^{(i,s)}(K;c) = \infty,\; J_T^{(i,s)}(K^*;c) < \infty \right\} \right|, \tag{2.40}$$

where $\mathcal{S}$ is the set of seeds whose controller is stable on the clean loop and $N = 100$ the number of episodes per seed.

2.4 Approach

As mentioned before, the LQR serves as the perfect starting example, which is exactly why we start our investigation here. For our investigations, we will use the double integrator, as mentioned in Section 2.1.2.

First, as a basis, we will start by comparing the standard policy gradient (PG) approach [14] with S-PI [41] on the true model (*TrP* and *TrS*). Their relative

performance to the baseline will be evaluated under the different inflation sizes introduced in Table 2.1. Then, the same controllers will be exposed to the different noise channels and intensities we introduced.

Afterwards, we will take a look at the cases where we do not have access to the real model. We compare the relative performance of both direct and indirect approaches to our baseline, which is the optimal controller calculated via the DARE (2.13) on the true model. For the indirect approaches, we will use numerical algorithms for subspace state-space system identification (N4SID) [65] to estimate the underlying model from one trajectory. A more detailed description, as well as the exact parameters selected, will be provided in the sections discussing the setup. With this estimated model as the basis, we employ three different approaches, called N4SID + DARE (*IdD*), N4SID + Model-based PG (*IdP*) and N4SID + S-PI (*IdS*). They will be complemented by Model-free Natural Policy Gradient (*MfN*) [14] and model-free S-PI (*MfS*) [41], which are direct solvers and estimate the gradient from episode rollouts. All these will also be discussed in Chapter 3.

In Chapter 4, we will then first introduce RL-based approaches. With proximal policy optimization (PPO) [52] being one of the most used architecture variants in robotics [50, 40, 24], we choose this as our main architecture, and apply different variations of it. First, by using only one layer and dropping the activation function and the biases, we force the trained controller to be linear, which allows us to extract it as a gain matrix that we can closely compare with our LQR approaches (Linear PPO, *Lin*). The fully connected, standard version is named Fully-connected PPO (*MLP*), and will also be evaluated. Additionally, we will introduce structures that force the observations through a low-dimensional latent space. Here, we will reuse the N4SID system identifier from before to identify the output matrix $\hat{C}$, before applying a change of basis to it, to achieve orthonormality in its columns. We project the high-dimensional observation through $\hat{C}^\top$ into a lower dimension, and then train PPO with this as the input. Therefore, the name for it is State encoder + PPO (*StE*). A similar approach is used for Scalable encoder + PPO (*ScE*), which is based on Jonschkowski and Brock [21] and Botteghi et al. [4], and uses robotic priors to derive a low-dimensional latent state via a trainable encoder. Again, the construction following this encoder is PPO. Lastly, we implement the orthonormal bottlenecks introduced by Todorov and Sabatelli [63] (Bottleneck PPO, *OrB*). It consists of sampling a fixed matrix U_b with d_b orthonormal columns, which is added at the end of a trainable feature extractor, and used to project the output into a d_b-dimensional space. Like with the other encoders, we follow it with our PPO architecture. The same feature extractor, but without the projection matrix U_b, followed by PPO, gives us FC-Extractor + PPO (*FcE*). From now on, we refer to all PPO variants by their short names.

All these low-dimensional approaches, if visualized, exhibit a "bow-tie" shape, which in nature can actually improve robustness [25]. So this is indeed interesting to investigate. However, first, like in Chapter 3, we start by comparing these

different approaches across the different inflation sizes, without any noise applied, to see how they generally behave. Then, we introduce noise, and compare our approaches under *large* inflation dimensions.

Taking our results from the experiments on the artificially inflated dimensions, in Chapter 5, we create a new setup, still using the double integrator as our underlying structure. Here, we will observe the mass via a simulated camera, and place a number of actuators evenly distributed among angles on a half-circle. This will give us an actual high-dimensional input, as well as a high-dimensional output, that is in reality governed by low-dimensional dynamics.

By inputting images into our existing solvers, the dimensionality increases drastically. To be able to cleanly separate effects caused by the nonlinearity of the image construction from architecture-related effects, we will first artificially inflate our inputs and outputs to exactly the sizes that will later be used by the pixelated environment. This evaluation will only be done on *ScE*, *MLP*, *OrB* and *FcE*, as the other approaches either do not scale well or are not relevant to the comparison.

Then, using the same architectural constructions for these solvers, but applied to our new environment, we compare them with the additionally introduced position-velocity encoder [22] followed by PPO (*PVE*), which uses similar robotic priors to *ScE*, but focuses more on position, velocity and acceleration. It learns positional encodings from the observations, and then calculates the velocity state based on finite differences. Here, it is quite clear that we use knowledge of the limitations of the embodiment to construct more optimal controllers, again in a bow-tie shape. Finally, we will introduce this environment to noise as well, and observe the effects of it.

Chapter 3

Robustness of LQR Solvers in Different Dimensions

3.1 A Known Model

We start by comparing two approaches that have knowledge of the underlying model, and thus do not need to estimate gradients but can calculate them exactly. This allows us to clearly separate the benefits of a low-dimensional controller from estimation errors.

3.1.1 Setting

Fazel et al. [14] introduced global convergence guarantees for policy gradient methods based on the true model in 2018. The paper describes a method for calculating the gradient of a policy from the system and cost matrices, and updating the policy accordingly. As the paper uses the same $u_k = -Kx_k$ convention as we do, we can implement the loop directly in the following way.

Starting from an initial, stabilizing gain $K_0 \in \mathcal{K}$ and the second moment Σ_0 of the initial state distribution, every iteration j first solves the discrete Lyapunov equation (2.16) for P_{K_j} using the implementation from SciPy [54]. The state correlation matrix Σ_{K_j} then follows from solving a second Lyapunov equation,

$$\Sigma_{K_j} = (A - BK_j)\Sigma_{K_j}(A - BK_j)^\top + \Sigma_0, \tag{3.1}$$

where, for initial states drawn uniformly from $[-10, 10] \times [-1, 1]$ as described in Section 2.3.3, the second moment is $\Sigma_0 = \operatorname{diag}(10^2/3,\, 1^2/3)$. The policy gradient [14, Lemma 1]

$$\nabla f(K_j) = 2\left(\left(R + B^\top P_{K_j} B\right) K_j - B^\top P_{K_j} A\right) \Sigma_{K_j} \tag{3.2}$$

is then used for a gradient step on the gain matrix. Algorithm 1 summarizes the loop. We refer to it as *TrP*.

Algorithm 1 Model-based PG

Require: stabilizing gain $K_0 \in \mathcal{K}$, second moment Σ_0, step size η

1: **for** $j = 0, 1, 2, \ldots$ **do**
2: solve (2.16) for P_{K_j}
3: solve (3.1) for Σ_{K_j}
4: compute the policy gradient $\nabla f(K_j)$ by (3.2)
5: update the gain by $K_{j+1} = K_j - \eta \nabla f(K_j)$
6: **end for**

The fixed learning rate η is a hyperparameter, which we set to $\eta = 10^{-5}$ for all inflation sizes, based on experiments whose results can be found in Section B.1. This policy gradient approach serves us as the basic, fully parameterized implementation for this experiment.

It is important to note that the above implementation is described for the uninflated loop. However, this can be easily applied to our inflated version by replacing every quantity with its inflated counterpart: $\tilde{A} = U_1 A U_1^\top$, $\tilde{B} = U_1 B U_2$, the inflated weights $\tilde{Q}$, $\tilde{R}$ in (2.22), $\tilde{\Sigma}_0 = U_1 \Sigma_0 U_1^\top$ and the initial gain $\tilde{K}_0$ from (2.39), such that the resulting gain is $K \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$.

Now we also want to implement a solver that produces a low-dimensional controller. As in our case the controller is characterized by the gain matrix, and for matrices dimensionality is "often [...] expressed by means of the rank" [47], we aim at learning a low-rank gain matrix. However, to make this comparable later to RL approaches that rely on the number of parameters, we measure the complexity of a gain by its policy size $|K|$ from Definition 2.1, which uses the more efficient version between the full and the factored parameterization.

Recht et al. [47] have shown that, given suitable conditions, nuclear norm minimization recovers low-rank solutions for affine constraints. However, as we are dealing with a non-convex optimization problem [14], we cannot transfer the results over directly. Here, Park et al. [41] offer us an approach to impose different structures on our policy during learning via regularizers based on norms, one of them being the nuclear norm. We implement the model-based approach introduced there with small adaptations in the following way.

We reuse the computation of the gradient $\nabla f(K_j)$ from Algorithm 1. For inducing the structure, we optimize the regularized cost $f(K) + \lambda \|K\|_*$, where the nuclear norm $\|\cdot\|_*$ can be reduced by eliminating singular values of K[41, Sec. 2.2], and λ is its weight in the whole cost. It further uses an adaptive learning rate, combined with a line search criterion, consisting of a stability and a sufficient decrease condition. Starting from the initial step size $\eta = 1/(2\lambda)$, following the paper's recommendation of $\mathcal{O}(1/\lambda)$, each iteration takes a gradient step and applies the proximal operator of the nuclear norm, which soft-thresholds the singular values,

$$K^+ = U \operatorname{diag}\left(\max\{\sigma_1 - \eta\lambda, 0\}, \ldots, \max\{\sigma_{\bar{r}} - \eta\lambda, 0\}\right) V^\top, \qquad \bar{r} \coloneqq \min\{d_{\text{out}}, d_{\text{in}}\}, \tag{3.3}$$

where $U \operatorname{diag}(\sigma_1, \ldots, \sigma_{\bar{r}}) V^\top$ is the singular value decomposition of the gradient step $K_j - \eta \nabla f(K_j)$. The candidate K^+ is accepted if it is stabilizing, $\rho(A - BK^+) < 1$, and satisfies the sufficient decrease condition

$$f(K^+) \leq f(K_j) - \eta \operatorname{tr}\left(\nabla f(K_j)^\top G\right) + \frac{\eta}{2}\|G\|_F^2, \qquad G \coloneqq \frac{1}{\eta}\left(K_j - K^+\right). \tag{3.4}$$

Otherwise, the step size is reduced to $\beta\eta$ and the proximal step is repeated. The learning loop further can be stopped earlier if the criterion $\|K_{j+1} - K_j\|_F < \varepsilon$ is satisfied. Algorithm 2 summarizes the loop, which, run on the true model, we refer to as *TrS*.

Algorithm 2 S-PI

Require: stabilizing gain $K_0 \in \mathcal{K}$, second moment Σ_0, weight λ, factor β, tolerance ε

1: **for** $j = 0, 1, 2, \ldots$ **do**
2: compute $\nabla f(K_j)$ as in Algorithm 1
3: set the initial step size $\eta = 1/(2\lambda)$
4: **for** at most 20 line search steps **do**
5: compute the candidate K^+ by (3.3)
6: **if** $\rho(A - BK^+) < 1$ and (3.4) holds **then**
7: **break**
8: **end if**
9: reduce the step size, $\eta \leftarrow \beta\eta$
10: **end for**
11: update the gain, $K_{j+1} = K^+$
12: **if** $\|K_{j+1} - K_j\|_F < \varepsilon$ **then**
13: **stop**
14: **end if**
15: **end for**

As one can see, there are a lot of parameters that can be modified here. We set $\varepsilon = 10^{-8}$, and use the same $\beta = 0.5$ as in the experiments section of Park et al. [41, Sec. 4.2]. For λ, as it defines the learning rate as well, we have run an experiment with various values, which can be found in Section B.2. A value of $\lambda = 5$ has shown the most promising results across the sizes.

One thing that we implemented differently than the paper is the cap on the number of line search iterations in line four, which we set to 20. Some of our previous experiments had shown that if we leave it uncapped, for unstable updates which are impossible to stabilize, the loop runs until the update produces no effect at all.

Now that we have defined our controllers and how they learn, we test them in comparison to the DARE-calculated solution (2.13) from Section 2.1.3.1.

3.1.2 Results

We apply our solvers to the *base* and inflated versions of the true model, for now noise-free, as described in Section 2.3.1.1. Specifically, as in Section 2.3.3, we train ten controllers per solver, before evaluating them on the test episodes. The results are then pooled.

The most interesting finding is how much faster *TrS* converges compared to *TrP*. While it achieves its best training suboptimality within around 50 iterations, *TrP* still falls short of it after 450 iterations. We can also see in Table 3.1 that the reduction in complexity for all inflation sizes has worked well with *TrS*. For our test suboptimality, *TrS* also outperforms *TrP* here by at least one order of magnitude. Figure 3.1 visualizes this.

Table 3.1: Size of the policy architectures on the true model, across every inflation size.

	Policy parameters				Gain rank			
Solver	base	small	medium	large	base	small	medium	large
Baseline	2	16	76	301	1.0	1.0	1.0	1.0
TrP	2	50	1250	20000	1.0	5.0	25.0	100.0
TrS	2	16	76	301	1.0	1.0	1.0	1.0

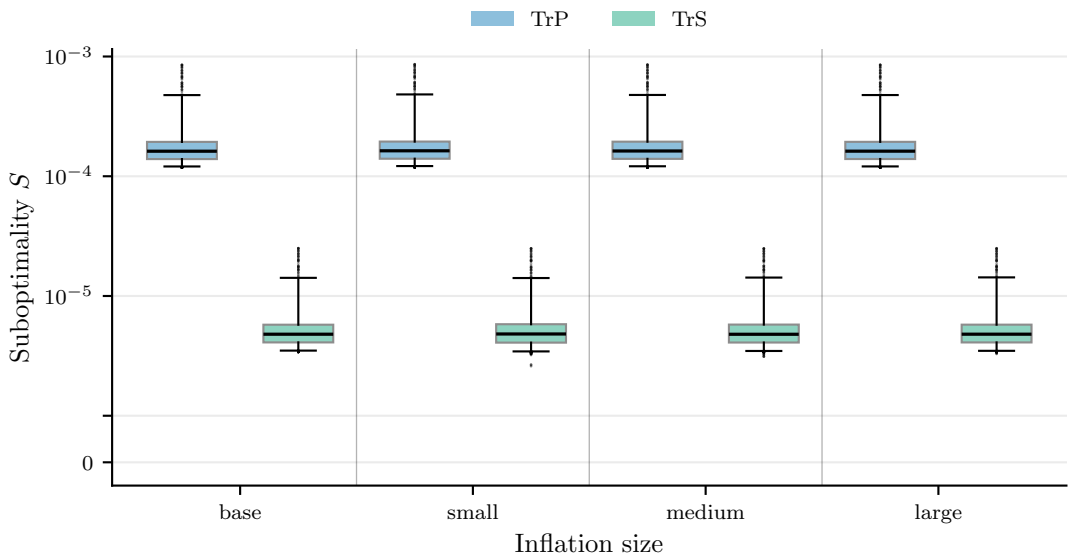


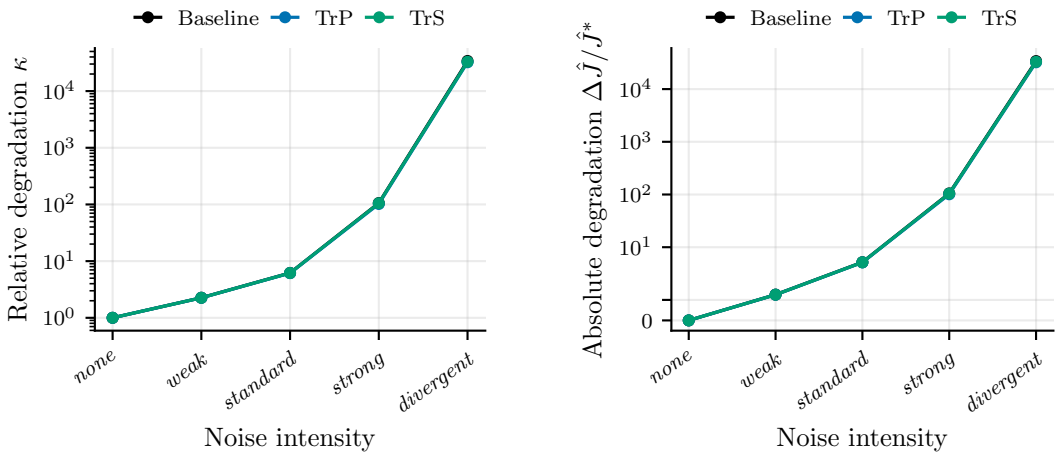
Figure 3.1: Suboptimality S of the solvers on the true model for each inflation size. The boxes indicate the interquartile range, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show episodes outside this range.

Overall, this looks very promising for *TrS*; still, our goal is to find out whether it is also a robust controller.

3.1.3 Noise Experiments

Now, we reuse the controllers we trained in Section 3.1.2 and evaluate them under the test protocol and *large* inflation with noise (Section 2.3.1.2) injected.

Looking at our trade-off plots, it might seem that each controller degrades nearly the same when all noise channels are activated at the same time, both relatively (Fig. 3.2a) and absolutely (Fig. 3.2b). However, when looking at the distribution of the seeds under relative degradation, while for some channels this might hold, there are noticeable differences for the others.



(a) Relative degradation κ.

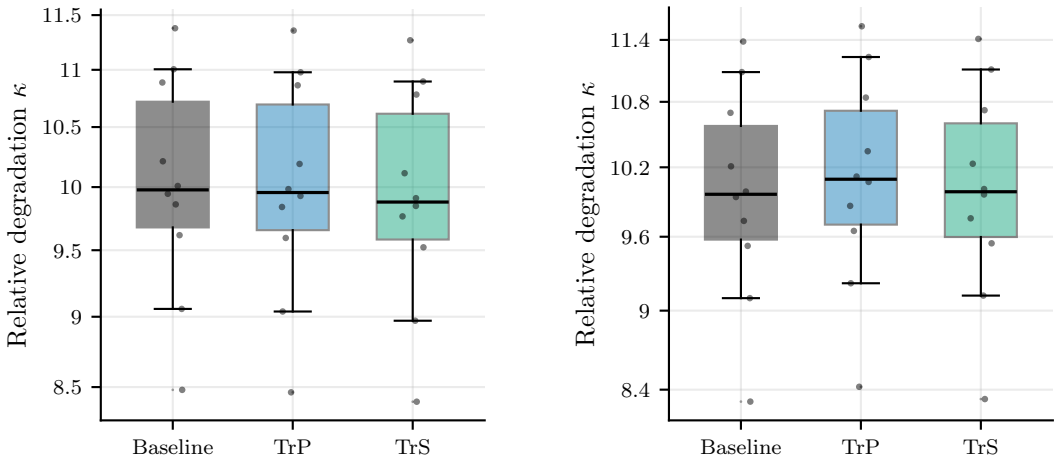
(b) Absolute degradation $\Delta\hat{J}/\hat{J}^*$.

Figure 3.2: Degradation of the solvers on the true model under *large* inflation and all noise channels at once, over the intensities.

For parametric uncertainty the distribution is also nearly identical, so here no difference can be noticed. When looking at measurement noise, however, we can notice that *TrP* degrades slightly less than the baseline (median at 9.96 vs. 9.98), and *TrS* is even further below (median at 9.88, Fig. 3.3a).

Under *divergent* external disturbance, we see that our trained controllers degrade slightly more than the baseline, with *TrP* being the worst here (median at 10.09 vs. 9.96 for the baseline, Fig. 3.3b).

The most noticeable difference is under control noise, where *TrP* noticeably degrades less than *TrS* or the baseline (median at 6.49 vs. baseline at 6.61, Fig. 3.4). *TrS* also stays below the baseline (median 6.59), but does not show as much robustness as *TrP*.



(a) Measurement noise.

(b) External disturbance.

Figure 3.3: Relative degradation κ of the solvers on the true model under *large* inflation at intensity *divergent*. The boxes indicate the interquartile range over the ten seeds, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show the individual seeds.

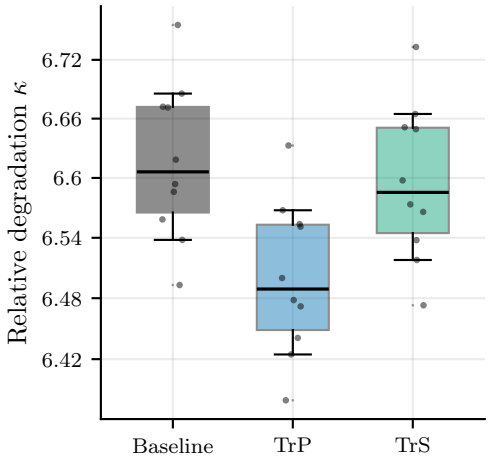


Figure 3.4: Relative degradation κ of the solvers on the true model under *large* inflation and control noise at intensity *divergent*. The boxes indicate the interquartile range over the ten seeds, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show the individual seeds.

3.1.4 Discussion

Judging from our results, it is difficult to draw significant conclusions in relation to robustness of the controllers. It seems like a trained, larger controller could be more robust to control noise; however, we need to verify this in the following experiments. What generally stands out is that *TrS* needs far fewer training iterations, and is smaller by up to a factor of 66 under *large* inflation, while achieving slightly better suboptimality and similar robustness to *TrP*. We verify whether this also holds for an unknown model in Section 3.2.

3.2 Unknown Model across Dimensions

Now we take a look at the more realistic case, where we do not know the underlying model, and compare different approaches. Before we can get into the results, we first take a deeper look at the solvers that we use and how the experiments are set up.

3.2.1 Setting

First, we introduce our basis for the indirect approaches, which is based on system identification. N4SID, introduced by Van Overschee and De Moor [65], uses "the projection of the future outputs [of the system] onto the past and future inputs and the past outputs" [65, Sec. 3]. These projections can then be described as "a function of the system matrices and the input-output block Hankel matrices" [65, Sec. 3], which allows us to reconstruct exactly that from the results. Important are further the assumptions that the systems are observable and controllable [65, Sec. 2.1]. For simplicity, we rely on the implementation of N4SID from SIPPY [55], which we integrate into our own setup. This implementation allows us to recover estimates A_{id}, B_{id} and C_{id} of the system matrices, whereas the cost matrices $\tilde{Q}$ and $\tilde{R}$ are choices of the controller design (Section 2.3.3), and thus already known beforehand. To the identified matrices we then apply the following change of basis, based on the QR decomposition $C_{\mathrm{id}} = \hat{C}\Theta$ with $\hat{C} \in \mathbb{R}^{d_{\mathrm{in}} \times \hat{n}}$ and $\Theta \in \mathbb{R}^{\hat{n} \times \hat{n}}$ upper triangular with a positive diagonal,

$$\hat{A} = \Theta A_{\mathrm{id}} \Theta^{-1}, \qquad \hat{B} = \Theta B_{\mathrm{id}}, \qquad \hat{C} = C_{\mathrm{id}} \Theta^{-1}, \tag{3.5}$$

such that $\hat{C}$ has orthonormal columns. This makes sense, as we have the knowledge that the output matrix C of our true model is also of this construction, but also allows us to get the pseudoinverse of $\hat{C}$ as $\hat{C}^\top$, since $\hat{C}^\top \hat{C} = I_{\hat{n}}$.

The collection of observation and action data runs the plant for T steps under the initial gain, with a randomly sampled excitation added on top of it,

$$v_k = -\tilde{K}_0 y_k + \sigma_{\mathrm{e}} w_k, \qquad w_k \sim \mathcal{N}(0, I_{d_{\mathrm{out}}}), \qquad k = 0, \ldots, T-1. \tag{3.6}$$

The paper mentions that a part of the result becomes exact as the number of block rows of the Hankel matrices grows [65, Sec. 5.3]. With the number of columns being

dependent on our trajectory length T, we aim to find the best combination of both. Denoting the number of block rows as the "Hankel horizon", we evaluated different combinations to find the optimal setting to use in our experiment. The full results of the selection can be found in Section B.3. Specifically, we use the settings in Table 3.2.

Table 3.2: Identification settings per inflation size.

Name	Trajectory length T	Hankel horizon
base	500	5
small	500	5
medium	500	5
large	2000	5

The resulting identified system then serves as a surrogate for the true model for the model-based approaches from Section 3.1.1, in which we assume a latent second moment of $I_{\hat{n}}$, i.e. $\tilde{\Sigma}_0 = \hat{C}\hat{C}^\top$. We refer to these as N4SID + Model-based PG (*IdP*) and N4SID + S-PI (*IdS*), and use for *IdP* an $\eta = 10^{-4}$ and for *IdS* $\lambda = 1$ across all sizes. This thus produces results without ever having access to the real model.

An attentive reader might now ask why we would even want to use these iterative approaches, if we could just solve the DARE (2.13) directly on this estimation. And indeed we include this N4SID + DARE controller (*IdD*) as an indicator for how well N4SID identifies the system in our following comparisons.

Additionally, we also use direct model-free approaches, which are variations of the model-based approaches from Section 3.1. Unlike before, they do not calculate the gradient directly, but estimate it based on the returns of N_{traj} pairs of perturbed versions $K \pm \Xi_i$ of the current policy K, each simulated for l steps. As one might assume, more "data" could conversely lead to better estimation and thus better policy updates, but it requires more computation. So, to settle this, we evaluated different combinations of them in Section B.4. The best trade-off between computational efficiency and quality gave $N_{\text{traj}} = 200$ and $l = 500$.

We use the model-free natural policy gradient (NPG) estimation described by Fazel et al. [14], which at its core consists of the loop in Algorithm 3. Instead of the proposed single-point estimator, we use a two-point estimator inspired by Duchi et al. and Mania et al. [12, 33], as this drastically reduced the variance of the estimates, and thus the instability of the controller.

We further aim to clarify a couple of things about the given loop. Subtracting the mean difference $\bar{\delta}$ does not change the expected value of the estimate, since $\mathbb{E}[\Xi_i] = 0$, but lowers its variance. We count a rollout as diverged once its observation exceeds $\|y_k\| = 10^5$. It is therefore ended early and charged a penalty cost of 10^{10}.

Algorithm 3 Two-point gradient estimation

Require: gain $K \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, number of perturbation pairs N_{traj}, rollout length l, smoothing radius r

for $i = 1, \dots, N_{\text{traj}}$ **do**
  sample Ξ_i uniformly from the matrices with $\|\Xi_i\|_F = r$
  sample an initial state $x_0 \sim \mathcal{D}$
  simulate $K + \Xi_i$ and $K - \Xi_i$ for l steps from x_0, with stage costs $c_k^{\pm}$ and observations $y_k^{\pm}$
  $\hat{J}_i^{\pm} = \sum_{k=0}^{l-1} c_k^{\pm}, \quad \hat{\Sigma}_i^{\pm} = \sum_{k=0}^{l-1} y_k^{\pm} \left(y_k^{\pm}\right)^\top, \quad \delta_i = \hat{J}_i^{+} - \hat{J}_i^{-}$
end for
$\bar{\delta} = \frac{1}{N_{\text{traj}}} \sum_{i=1}^{N_{\text{traj}}} \delta_i$
$\widehat{\nabla f}(K) = \frac{1}{N_{\text{traj}}} \sum_{i=1}^{N_{\text{traj}}} \frac{d_{\text{out}} d_{\text{in}}}{2r^2} \left(\delta_i - \bar{\delta}\right) \Xi_i$
$\hat{\Sigma}_K = \frac{1}{N_{\text{traj}}} \sum_{i=1}^{N_{\text{traj}}} \left(\hat{\Sigma}_i^{+} + \hat{\Sigma}_i^{-}\right)$
$\hat{f}(K) = \frac{1}{N_{\text{traj}}} \sum_{i=1}^{N_{\text{traj}}} \frac{1}{2} \left(\hat{J}_i^{+} + \hat{J}_i^{-}\right)$
return $\widehat{\nabla f}(K)$, $\hat{\Sigma}_K$, $\hat{f}(K)$

Additionally, for inflated dimensions, $\hat{\Sigma}_K$ is singular, so we precondition with its truncated pseudoinverse $\hat{\Sigma}_K^{\dagger}$ instead, which discards singular values below 10^{-6} times the largest one.

Our full learning algorithm, which we refer to as Model-free NPG (*MfN*), is then given in Algorithm 4.

Algorithm 4 Model-free NPG

Require: stabilizing gain $K_0 \in \mathcal{K}$, N_{traj}, l, r, step size η

for $j = 0, 1, 2, \dots$ **do**
  compute $\widehat{\nabla f}(K_j)$ and $\hat{\Sigma}_{K_j}$ by Algorithm 3
  update the gain by $K_{j+1} = K_j - \eta\, \widehat{\nabla f}(K_j)\, \hat{\Sigma}_{K_j}^{\dagger}$
end for

We use the natural gradient $\widehat{\nabla f}\, \hat{\Sigma}^{\dagger}$ here, as it preconditions the update with the pseudoinverse of the state correlation instead of applying the update directly like the standard model-free version [14]. In initial experiments it consequently showed better stability, which is why we selected it. In the same process we decided on a smoothing parameter of $r = 0.5$, and, based on separate results in Section B.5, have chosen a learning rate of $\eta_{\text{base}} = 0.1$, $\eta_{\text{small}} = 0.03$, $\eta_{\text{medium}} = 0.01$ and $\eta_{\text{large}} = 0.01$.

Next, we implement a model-free version of S-PI (*MfS*) that relies on the same gradient estimation (Algorithm 3) as *MfN*. Another crucial difference, apart from the estimation of the gradient compared to the calculation, is the "usage of a fixed stepsize η, rather than an adaptive stepsize from a backtracking linesearch" [41, Sec. 3.1], as that would require access to the function value $f(K) = \operatorname{tr}(P_K \Sigma_0)$.

Before we look at how we select this step size and λ, we define our learning loop in Algorithm 5.

Algorithm 5 S-PI (model-free)

Require: stabilizing gain $K_0 \in \mathcal{K}$, N_{traj}, l, r, fixed step size η, weight λ, tolerance ε

1: **for** $j = 0, 1, 2, \ldots$ **do**
2: compute $\widehat{\nabla f}(K_j)$ by Algorithm 3
3: compute K_{j+1} by (3.3), with $\widehat{\nabla f}(K_j)$ in place of $\nabla f(K_j)$
4: **if** $\|K_{j+1} - K_j\|_F < 10^{-8}$ **then**
5: **stop**
6: **end if**
7: **end for**

Now to the selection of the parameters. Park et al. already note the difficulty of finding a fixed step size that does not produce divergence of the method [41, Sec. 3.2]. Also, the example shown only uses the model-based approach and the regularizer to induce sparse structure[41, Sec. 4], not the model-free variant combined with low dimensionality. So during our first tests, we often ran into either unstable controllers, or ones that did not induce any structure at all, hinting at a λ that is too low. If we remember the proximal step (3.3), we can see that it depends on both the step size and λ, as every singular value is mapped to

$$\sigma_i \mapsto \max\{\sigma_i - \eta\lambda, 0\}. \tag{3.7}$$

Even after some improvement by using the two-point estimator (Algorithm 3), it remained quite a difficult topic. So rather extensive experiments on the variation of λ and the step size can be found in Section B.6. Based on those, we selected $\lambda_{\text{base}} = 4$ and $\eta_{\text{base}} = 3.5 \times 10^{-6}$, $\lambda_{\text{small}} = 400$ and $\eta_{\text{small}} = 3.5 \times 10^{-5}$, $\lambda_{\text{medium}} = 400$ and $\eta_{\text{medium}} = 3.5 \times 10^{-5}$; and $\lambda_{\text{large}} = 40\,000$ and $\eta_{\text{large}} = 3.5 \times 10^{-8}$.

3.2.2 Results

Looking at the time until convergence, as well as the achieved training suboptimality, we notice that *MfS* produced under *large* inflation a very expensive gain, while still not achieving a strong reduction in rank. As this was the same problem we observed during our parameter search (Section B.6), we accept it as an unsuccessful attempt and difficult to implement. Apart from this, it is noticeable that *MfN* takes longer to converge the larger the inflation gets, with an increase from 167 at *base* to 610 at *large* (Table 3.3). This makes it slower than *IdS* and *IdP*, where *IdS* (at most 90 across the sizes) again is still much faster than *IdP* (around 350). The fastest approach here is *IdD*, as it only requires the identification step, and no iteration.

Table 3.3: Iterations until convergence of the solvers without the model, as mean and standard deviation over the ten seeds, for each inflation size.

Solver	*base*	*small*	*medium*	*large*
IdD	1	1	1	1
IdP	346	346 ± 3	346 ± 1	346
IdS	34 ± 1	90 ± 5	76 ± 0.3	68
MfN	167 ± 9	216 ± 43	356 ± 46	610 ± 104
MfS	368 ± 58	216 ± 32	186 ± 116	451 ± 5

Looking at the policy sizes, we can see that *IdS* indeed produces the smallest policy, with *IdD* being slightly larger in some cases (Fig. 3.5), but still a lot smaller than the full-rank gains.

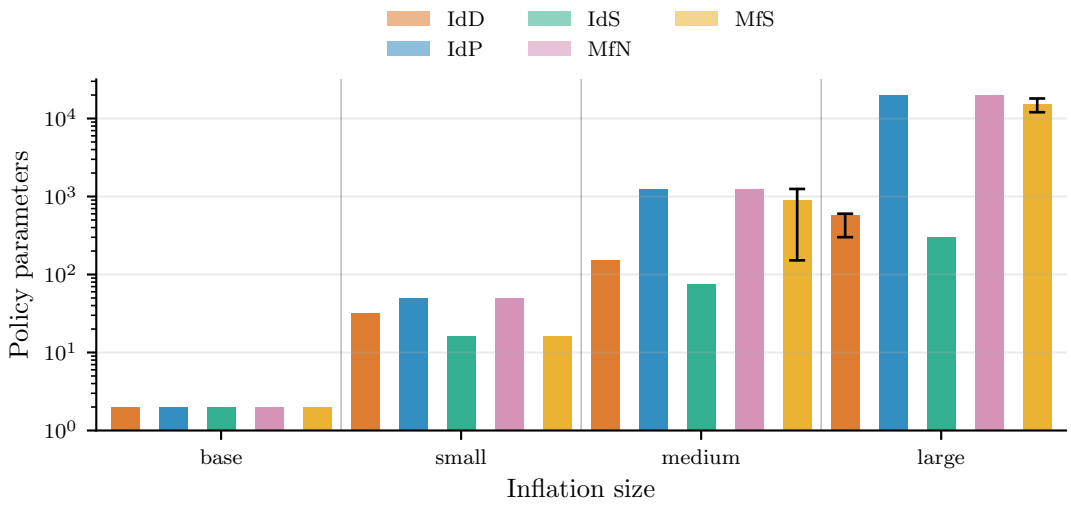


Figure 3.5: Policy size $|K|$ (Definition 2.1) learned without the model, for each inflation size.

Analyzing the resulting test suboptimality, we see the expected worse performance of *MfS* from *medium* onwards, but also that the direct approaches underperform the indirect (identification) approaches. What also jumps out is that *IdD* matches the optimal performance within $2 \cdot 10^{-7}$ across every size (Fig. 3.6), indicating that N4SID is able to almost perfectly identify the underlying system.

3.2.3 Noise Experiments on Large Inflation

Again, we investigate our system inflated to *large* size and repeat our experiments from the true model section (Section 3.1.3).

We first start by taking a look at parametric uncertainty. As it affects the model

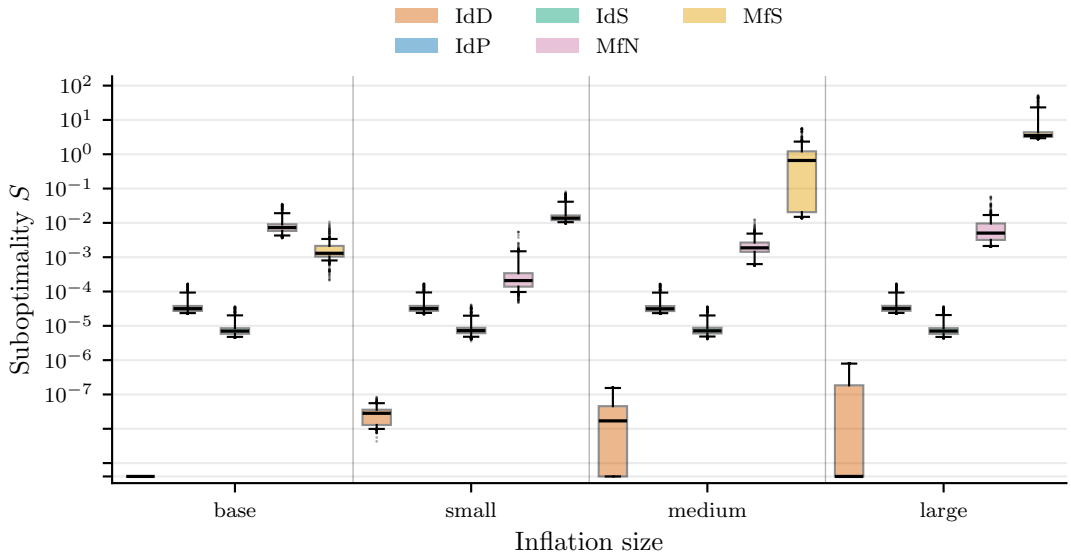


Figure 3.6: Suboptimality S of the solvers without the model for each inflation size. The boxes indicate the interquartile range, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show episodes outside this range.

matrices directly, already worse performance on the clean plant is magnified even more if the dynamics change. For *MfS*, this can clearly be seen, as under *divergent* intensity, even the least relative degradation amounts to 7714, with the majority being above 10^5 (Table 3.4).

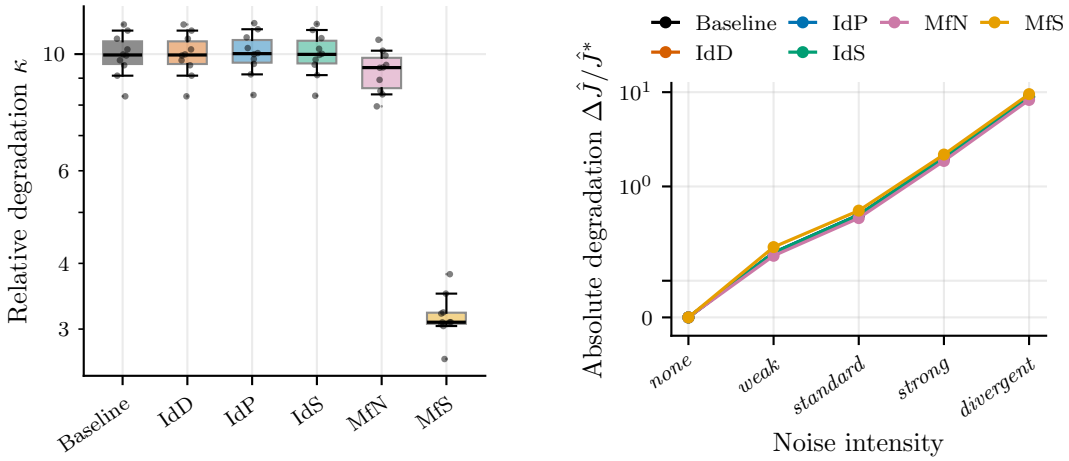
Table 3.4: Median relative degradation κ over the ten seeds under *large* inflation and parametric uncertainty, and the range over the seeds at intensity *divergent*.

Intensity	Baseline	IdD	IdP	IdS	MfN	MfS
weak	1.195	1.195	1.194	1.194	1.197	13.66
standard	1.451	1.451	1.450	1.450	1.455	134.5
strong	2.422	2.422	2.418	2.420	2.435	7033
divergent	5.747	5.747	5.735	5.740	6.297	1.654×10^5
divergent, minimum	4.673	4.673	4.623	4.649	5.028	7714
divergent, maximum	25.69	25.69	26.13	25.94	41.91	9.147×10^5

Looking at the other architectures, we can see that all controllers relying on N4SID estimation achieve almost identical relative degradation compared to the baseline, with their medians only differing by at most 0.3 %. Taking the direct approach of *MfN* into account as well, we can see that at *divergent* intensity its median relative degradation is around 10 % higher than that of the other approaches (6.3 vs. the

others at 5.74). Looking at the worst degradation, the gap is even wider (41.9 vs. 26.1).

Next, we analyze robustness against external disturbances. Here, we again notice that indirect approaches are affected similarly to the baseline, while the directly trained controllers degrade in relative terms slightly less (in the case of *MfN*) or a lot less (*MfS*, Fig. 3.7a). This is also visible across all intensities. However, when looking at the absolute degradation, one can see that the low value for *MfS* stems from the fact that its nominal performance was poor, and its actual absolute increase is similar to all the other controllers, or even slightly above (Fig. 3.7b). For *MfN*, however, it also lies below the others, hinting at an ever so slightly improved robustness.



(a) Relative degradation κ at intensity *divergent*.

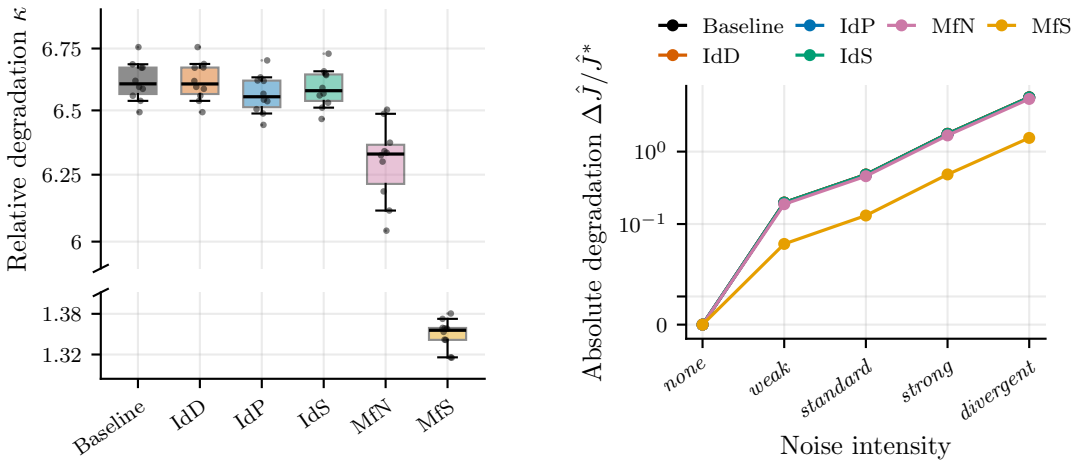
(b) Absolute degradation $\Delta\hat{J}/\hat{J}^*$ over the intensities.

Figure 3.7: Degradation of the solvers without the model under *large* inflation and external disturbance. In (a), the boxes indicate the interquartile range over the ten seeds, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show the individual seeds.

Looking at control noise next, we can again see the issue from before, where in relative terms *MfS* barely degrades, while the other solvers show values much closer to the calibrated increase κ_ℓ of the baseline (Fig. 3.8a). Still, as *MfS* also shows less absolute degradation here (Fig. 3.8b), it seems to be more robust to control noise than the other architectures. Again, we can also notice that *MfN* is slightly better than all the indirect approaches or the baseline, however not by much.

Noticeable from the spread, however, is that control noise seems to have mostly the same effect across all seeds, compared to the previous noise channels, where relative degradation affected seeds differently.

Finally, we investigate measurement noise. Here, from visual inspection, it seems



(a) Relative degradation κ at intensity *divergent*.

(b) Absolute degradation $\Delta\hat{J}/\hat{J}^*$ over the intensities.

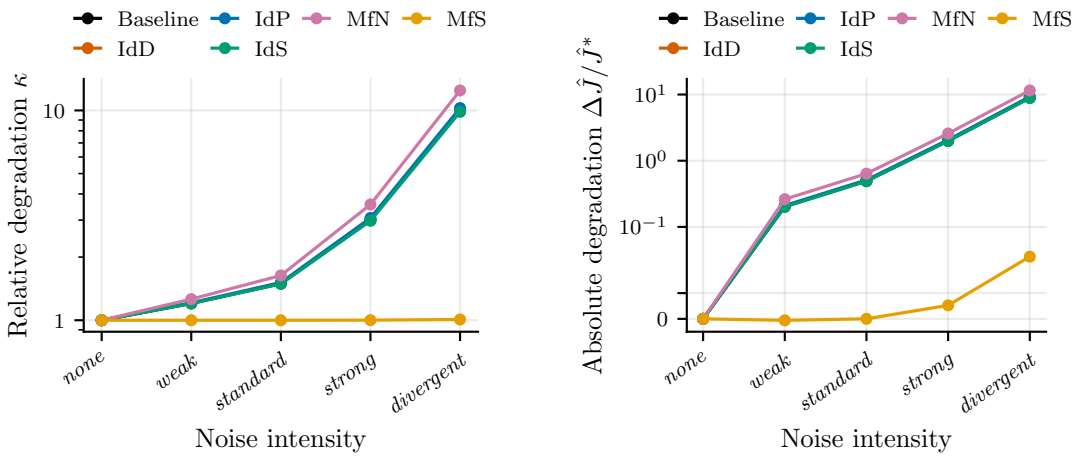
Figure 3.8: Degradation of the solvers without the model under *large* inflation and control noise. In (a), the boxes indicate the interquartile range over the ten seeds, the line marks the median, and the whiskers span the 5th to 95th percentile. Points show the individual seeds.

like *MfS* is completely robust against it (Fig. 3.9a). While the other architectures experience an around tenfold increase in cost (with *MfN* even slightly higher) similar to the baseline, it sits consistently across the intensities at a relative degradation of one. However, this time we cannot fully attribute this to its higher nominal cost. Under *weak* and *standard* intensity, measurement noise does not increase its cost measurably (Fig. 3.9b) and under higher intensities it remains much more robust than the other approaches.

When now taking a look at the other architectures in more detail, we notice that the larger controllers are indeed more affected by the noise. *MfN* sits above all the model-based approaches, and *IdP* also degrades slightly more than *IdS*, with the baseline sitting between both. We already noticed a similar pattern in the previous experiment, where *TrS* performed slightly better than *TrP*, indicating that fewer parameters might make a controller more robust against measurement noise.

3.2.4 Discussion

From our results, we can see that on the clean loop the identification-based approaches perform much better than the direct approaches, achieving a performance similar to that of the optimal controller. Here, *IdD* seems to be the best solution, as it does not require any training iterations, achieves the lowest suboptimality, and is of a similar size as *IdS*. When looking at our model-free approaches, nothing speaks in favor of them—they require more iterations and, in the case of *MfS* under *large* inflation, perform poorly on the clean loop, while not even achieving



(a) Relative degradation κ.

(b) Absolute degradation $\Delta\hat{J}/\hat{J}^*$.

Figure 3.9: Degradation of the solvers without the model under *large* inflation and measurement noise of increasing intensity.

a strong rank reduction.

This worse performance is then amplified under parametric uncertainty, penalizing them even more, while *IdD*, *IdP* and *IdS* degrade the same as the baseline. External disturbance affects all indirect approaches the same, while among the direct approaches, *MfN* shows more and *MfS* less robustness than those, judging from the absolute degradation.

When looking at control noise, it might seem like this is where *MfS* is much more robust than the others, and *MfN* slightly as well. But when taking a look at Fig. 3.10, we can see that *MfN* applies slightly less acceleration than the others, and *MfS* applies it at a much smaller amplitude. As control noise is in our case multiplicative, a lower value is affected by much less noise. Therefore, the savings in cost come from the fact that *MfS* is not functioning well, and does not achieve the goal of steering the mass towards the origin.

If we extend the same reasoning to measurement noise, it also seems logical that *MfS* is showing less degradation here. If the gain matrix was not able to control the mass without noise well, hence not mapping inputs to effective outputs, then a change to these named inputs could have any effect on the cost. In this case, it seems to help reduce its degradation.

One interesting thing however remains. Again, for measurement noise, *IdS* seems to sit slightly below *IdP*, hinting again at a better resistance of smaller policies to measurement noise, like we hypothesized in Section 3.2.3. One might say that *MfS* also shows this; however, this is incorrect. Based on the small size reduction, and the difficulties controlling the mass under the clean loop, it would be an inconsistent comparison. But, comparing *IdS* to *IdP* and *MfN*, we can indeed see that *IdS* is

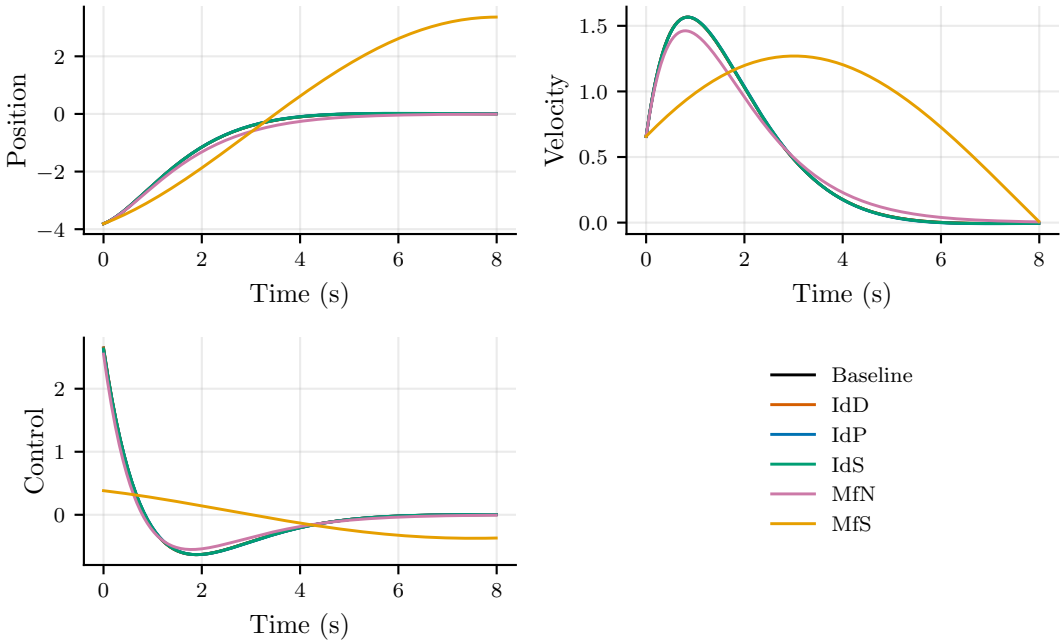


Figure 3.10: Position, velocity and control input on the clean loop under *large* inflation.

slightly more robust to measurement noise, while being much smaller than those other two.

3.3 Discussion

This chapter has shown us that smaller-sized policies can indeed achieve performances similar to fully parameterized controllers, while maintaining similar or partially better robustness properties. The main side note here, though, is that it depends on the choice of the algorithm that creates such a policy. As we have noticed, direct approaches perform generally worse, and require more training time, compared with a mixed approach of system identification and policy iteration. Unbeaten is clearly *IdD*, which requires just the system identification step, and performs on par with the optimal controller throughout.

As this system is a very simple case, and system identification is not scalable to larger inflation (as we will mention in Chapter 5), we cannot rely on these approaches for real-world applications. We therefore take these learnings and indications with us, and try to use them for choosing RL-based algorithms in Chapter 4.

Chapter 4

Robustness of Reinforcement Learning Approaches

Having seen how low dimensionality affects robustness with a simple, LQR-based controller, we now want to take a look at a setting which allows us to judge architectures used in more complex tasks as well. Still, we will stay with our introduced double-integrator system, but change the controllers for now.

4.1 PPO and Its Variants

4.1.1 Proximal Policy Optimization

Before getting into our implementation details, we want to explain the basic construction of PPO. Introduced by Schulman et al. [52] in 2017, it is an actor-critic algorithm, which means it consists of an actor, guided by a policy, that is simulated in an environment, and a critic, which tries to estimate the underlying value function based on the current policy and the actor's returns [26]. In the case of PPO, as we are trying to optimize a policy via policy gradient optimization, we "[construct] an objective function whose gradient is the policy gradient estimator" [52, Sec. 2.1]. What makes PPO special is the construction of this surrogate function, based on ideas from Trust Region Policy Optimization [51]. In the paper, two approaches are proposed: a Clipped Surrogate Objective or an Adaptive Kullback–Leibler (KL) Penalty Coefficient. The former clips the probability ratio past a limit set by a hyperparameter, imposing constraints on the policy update size [52, Sec. 3], while the latter, rather than setting a fixed KL for an update, sets a target KL divergence [52, Sec. 4]. In case further interest arises, we refer to the original paper [52]. With the probability ratio $r_k(\theta)$ between the current policy π_θ

and the policy $\pi_{\theta_{\text{old}}}$ that collected the data, the clipped objective becomes

$$\begin{aligned} r_k(\theta) &= \frac{\pi_\theta(v_k \mid y_k)}{\pi_{\theta_{\text{old}}}(v_k \mid y_k)}, \\ L^{\text{CLIP}}(\theta) &= \hat{\mathbb{E}}_k \left[\min \left\{ r_k(\theta)\hat{A}_k,\ \text{clip}\left(r_k(\theta), 1-\varepsilon_{\text{clip}}, 1+\varepsilon_{\text{clip}}\right)\hat{A}_k \right\} \right]. \end{aligned} \tag{4.1}$$

4.1.2 Training Setup

However, as has been noted before [13, 17], the actual implementation of PPO matters, and can influence the effectiveness of it in practice. For our construction, we use the architecture provided by Stable-Baselines3 (SB3) [57]. As the standard architecture, for our Fully-connected PPO (*MLP*), we use the standard two-layer multilayer perceptron (MLP) with 64 neurons per layer for both the actor and the critic [56]. Table B.5 in Section B.7 lists all deviations from the default parameters. After evaluating different values, we settled on the learning rates in Table B.6, which differ by inflation size.

After initial runs with this basic setup, we could observe some divergence behavior, which led us to consider additional implementation improvements, as suggested by Huang et al. [19]. Specifically, we modified the discount factor to $\gamma = 0.999$ instead of 0.99 and used the reward normalization implementation from SB3 [58] based on a discounted moving average [19, continuous-action Detail 8], which also clips the normalized reward to $[-10, 10]$. Two additional helpful changes were:

1. Using learning-rate annealing instead of a fixed rate [19, Detail 4]. Now the learning rate is decreased with every update, such that at the end of the step budget (Table 2.3) it is equal to 0. So this is

$$\eta(N_{\text{env}}) = \eta_0 \max\left\{1 - \frac{N_{\text{env}}}{N_{\max}},\ 0\right\}. \tag{4.2}$$

2. Separating the feature extractors for actor and critic [19, Detail 13]. As described in the SB3 documentation [56], to reduce computation, in the standard implementation, the extractor is shared by default. For extractors that are trained together with PPO, we separate actor and critic extractors, such that they are trained separately. Later, when introducing pretrained extractors which are fixed post-training, those will be shared again.

Algorithm 6 summarizes the resulting training loop.

4.1.3 Policy Architectures

Now that we have introduced the base construction, it is easier for us to describe variations of it. We start with Linear PPO (*Lin*), which allows us to compare the performance of PPO to the previous chapter. Here, our actor becomes a single layer, with the bias set to and kept at 0, such that we can rewrite this network as

$$\mu(y_k) = W y_k = -K y_k, \qquad K \coloneqq -W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, \tag{4.3}$$

Algorithm 6 PPO implemented here

Require: initial actor π_θ and critic V_ϕ, initial learning rate η_0, step budget $N_{\max}$

$N_{\text{env}} \leftarrow 0$
while $N_{\text{env}} < N_{\max}$ **do**
  run π_θ for 2048 steps, sampling $v_k \sim \pi_\theta(\cdot \mid y_k)$; store y_k, v_k and the rewards
  normalize the rewards by a running estimate of the return's standard deviation
  compute the advantages $\hat{A}_k$ and value targets with V_ϕ
  $\theta_{\text{old}} \leftarrow \theta$, set the learning rate $\eta(N_{\text{env}})$ by (4.2)
  for epoch $= 1, \ldots, 10$ **do**
    for each minibatch of 64 stored steps **do**
      take an Adam step on $-L^{\text{CLIP}}(\theta) + 0.5 \cdot$ (value loss of V_ϕ), with the gradient norm clipped to 0.5
    end for
    if the approximate KL divergence to $\pi_{\theta_{\text{old}}}$ exceeds $1.5 \cdot 0.02$ **then**
      break
    end if
  end for
  $N_{\text{env}} \leftarrow N_{\text{env}} + 2048$
  if 25 000 steps have passed since the last evaluation **then**
    evaluate π_θ for the learning curve
  end if
end while
return the final policy π_θ

which allows us to extract a gain matrix K that is comparable to the previous chapter. The critic remains as described in Section 4.1.2. We further initialize our actor with the weights taken from the stable $\tilde{K}_0$ of Section 2.3.3.

A second, already known, component is used for State encoder + PPO (*StE*). Here, we reuse N4SID (Section 3.2) to identify the system with the same settings as before (Table 3.2) and then encode an observation into a latent state:

$$\hat{x}_k = \hat{C}^\top y_k, \qquad \hat{C} \in \mathbb{R}^{d_{\text{in}} \times \hat{n}}, \quad \hat{C}^\top \hat{C} = I_{\hat{n}}. \tag{4.4}$$

Due to the construction of $\hat{C}$ in Section 3.2.1, its transpose $\hat{C}^\top$ is equal to its pseudoinverse, projecting an observation back to the identified latent state dimension. The inputs of both the actor and the critic are thus projected to dimension $\hat{n}$, which is the order of the identified model.

However, as we will later scale our inputs to larger dimensions (Chapter 5), N4SID will become infeasible for such dimensions. Still, we want to stay in line with the idea of encoding our observations into a latent state, as this exactly introduces the bow-tie structure we want. So, we introduce another encoder, which offers this scalability, and call the combination Scalable encoder + PPO (*ScE*). This feature extractor is based on the results of Jonschkowski and Brock [21] and Botteghi

et al. [4], which use robotic priors. Robotic priors, simply described, are prior knowledge, like physical laws, from which we define loss functions. By adding them to the training process, and optimizing for them, we can ensure that the learned representation is in line with these laws. As mentioned in Section 2.4, this allows us to make use of the embodiment to improve our controller architecture. Here, we will use the following priors [4]:

- simplicity: the relevant information is lower in dimensionality than the observation,
- temporal coherence: state changes are rather slow,
- proportionality: similar actions should show similar output magnitudes,
- repeatability: similar actions in similar states should show similar state changes,
- causality: prevents the trivial solution of mapping every observation to the same point.

The translation of these principles into loss calculations can be found in detail in the paper [4]. We modify their total loss calculation slightly, by dropping the $\mathcal{L}_{\text{reg}}$ term, and thus obtain

$$\mathcal{L} = \omega_1 \mathcal{L}_{\text{temp}} + \omega_2 \mathcal{L}_{\text{prop}} + \omega_3 \mathcal{L}_{\text{rep}} + \omega_4 \mathcal{L}_{\text{caus}}, \tag{4.5}$$

with $\omega_1 = \omega_2 = \omega_3 = 1$ and $\omega_4 = 2$.

As this encoder must be trained first, we pretrain it for 20 epochs on 20 000 collected environment steps, before freezing its parameters and prepending it to the actor and the critic. The data is collected by running a policy using the initial gain $\tilde{K}_0$ or a randomly drawn action in a 70 %/30 % split on the environment for at most 800 consecutive steps.

An important parameter is the dimension of the latent state, which we set to the true state dimension $n = 2$ here. In practice, one might not know the underlying dimension of the dynamics, but as N4SID correctly identifies the true dimension in most runs, we use the same dimension here to keep the solvers comparable.

Algorithm 7 summarizes the pretraining.

The second trainable extractor, which we use in Bottleneck PPO (*OrB*), is based on the paper by Todorov and Sabatelli [63], using orthogonal bottlenecks. It consists of a fully connected MLP, with 512 neurons in the first, and 256 neurons in the second and third layer, using a ReLU activation function. Following it is a matrix with orthonormal columns, which projects the output into a state of bottleneck dimension d_{b}, which we here set to 2. Like U_1 in (2.19), the matrix is the orthonormal factor of the QR decomposition of a random matrix with standard normal entries, so that the feature extractor output $z_k \in \mathbb{R}^{d_{\text{z}}}$ is mapped to

$$U_{\text{b}}^{\top} z_k \in \mathbb{R}^{d_{\text{b}}}, \qquad U_{\text{b}} \in \mathbb{R}^{d_{\text{z}} \times d_{\text{b}}}, \quad U_{\text{b}}^{\top} U_{\text{b}} = I_{d_{\text{b}}}. \tag{4.6}$$

Algorithm 7 Pretraining of the scalable encoder

Require: initial gain $\tilde{K}_0$, action bound $a_{\max}$, encoder $f_\psi \colon \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^2$, weights $\omega_1, \ldots, \omega_4$

for $k = 0, \ldots, 20\,000 - 1$ **do**
  with probability 0.3, draw v_k uniformly from $[-a_{\max}/10, a_{\max}/10]^{d_{\text{out}}}$, otherwise set $v_k = -\tilde{K}_0 y_k$
  apply v_k, store (y_k, v_k, y_{k+1}), and reset the plant after 800 steps
end for
for epoch $= 1, \ldots, 20$ **do**
  for each minibatch of 128 stored transitions **do**
    pair every transition with a randomly drawn second transition
    take an Adam step on $\omega_1\mathcal{L}_{\text{temp}} + \omega_2\mathcal{L}_{\text{prop}} + \omega_3\mathcal{L}_{\text{rep}} + \omega_4\mathcal{L}_{\text{caus}}$ of these pairs
  end for
end for
standardize the latent state with its mean and standard deviation over the stored observations
freeze ψ and place f_ψ in front of the actor and the critic

Unlike the previous encoder, this one is trained together with PPO, so the actor and critic train them separately, while the projection matrix U_{b} stays the same for both, and across seeds to ensure comparability.

Lastly, we introduce FC-Extractor + PPO (*FcE*), which uses the same three-layer extractor, but passes its 256-dimensional output to the actor and the critic directly, without projecting it via U_{b}. The goal here is to differentiate whether the projection of (4.6) has any positive effect, or just the size of its policy.

Figure A.1 in Section A.4 visualizes the architectures of all our solvers.

4.2 Comparison across Dimensions

Looking at achieved suboptimality during training, we overall see a learning process in almost all of our architectures (Fig. 4.1). Noticeable, however, is the long stagnation in the beginning of the learning, before decreasing towards the end. Significant is also *Lin*, which does not achieve an improvement under *large*, but this was expected, given its unsuccessful learning-rate search results (Fig. B.8). Also, when looking at the spread, we see that in some seeds *ScE* also rather increases in cost.

This then leads to some episodes in which the mass cannot be contained by this policy during testing, which we can see in Table 4.1. We exclude the affected seeds from the following analysis. The instability of the policies trained by *Lin*, where six out of the ten seeds experience an unstable gain during training, out of which three remain unstable in the end, is also noted there.

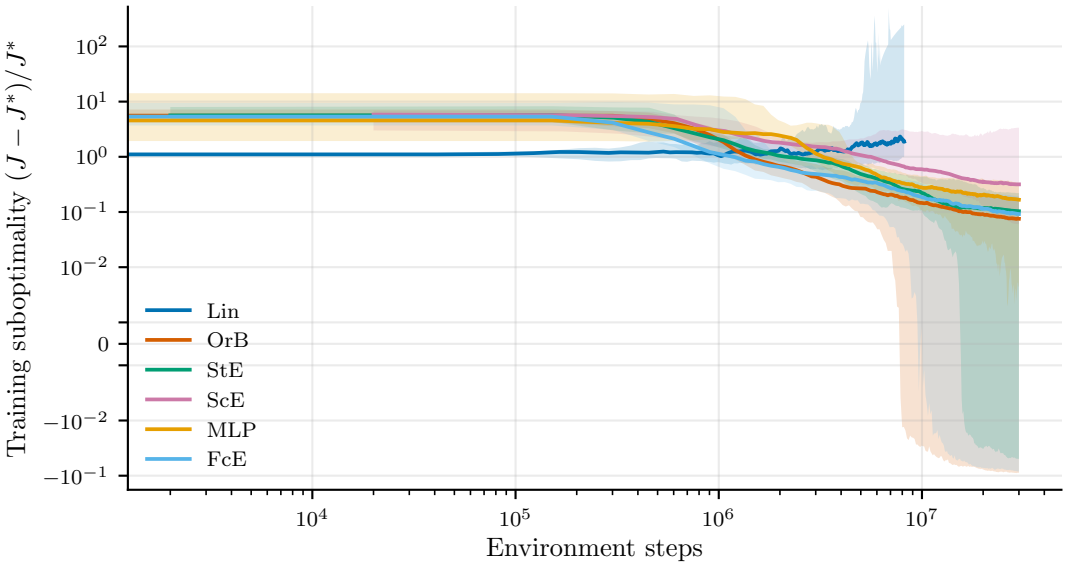


Figure 4.1: Training suboptimality $(J - J^*)/J^*$ of the PPO variants under *large* inflation. Values below 0 can occur as the cost J is only estimated from rollouts.

Table 4.1: Stability of the PPO variants for each inflation size, without noise. Stable counts the seeds whose controller kept the plant stable and contained the mass in every test episode, runaway the test episodes (out of 1000) that end with the mass outside the range of initial positions. A linear gain is evaluated by its spectral radius only.

	small		*medium*		*large*	
Solver	Stable	Runaway	Stable	Runaway	Stable	Runaway
Lin	10/10	–	10/10	–	7/10	–
OrB	10/10	0	10/10	0	10/10	0
StE	10/10	0	10/10	0	10/10	0
ScE	9/10	2	10/10	0	9/10	4
MLP	10/10	0	10/10	0	10/10	0
FcE	10/10	0	10/10	0	10/10	0

Looking at the complexity of our trained policies, *Lin* is significantly the smallest under *small* and *medium* inflation. Under *large*, however, we can see that *StE* has the smallest parameter count, but is of a similar magnitude as *Lin* or *MLP*, which are roughly double its size. The group of much larger policies (ten- to fifteenfold) consists of *OrB*, the non-dimensionally reduced *FcE* and *ScE*. For the latter, most of its parameters (around 94 %) are already frozen after pretraining, differentiating it from the rest, which are almost all trained via PPO, and making its trainable parameters comparable to *StE* (Table 4.2).

Table 4.2: Actor parameters of the PPO variants for each inflation size, split into those trained by PPO and those frozen after pretraining.

	small			*medium*			*large*		
Solver	Trainable	Frozen	Total	Trainable	Frozen	Total	Trainable	Frozen	Total
Lin	50	0	50	1 250	0	1 250	20 000	0	20 000
OrB	207 429	0	207 429	229 209	0	229 209	310 884	0	310 884
StE	4 677	20	4 697	5 977	100	6 077	10 852	400	11 252
ScE	8 645	137 666	146 311	9 945	158 146	168 091	14 820	234 946	249 766
MLP	5 189	0	5 189	9 049	0	9 049	23 524	0	23 524
FcE	223 685	0	223 685	245 465	0	245 465	327 140	0	327 140

Regarding the performance, we can see that under *small* inflation *Lin*, *OrB* and *FcE* achieve a similar median, while *StE* and *MLP* have a slightly worse performance, with *ScE* being the worst of them all. However, when looking at the spreads, *StE* has some runs in the 100 to 1000 suboptimality range, which are the largest ones, while the worst case of *MLP* is similar to that of *FcE*. The best one here is *Lin*, which shows little spread.

Looking at *medium*, here *MLP* becomes worse regarding its median, followed by *ScE*, even though the spread of *ScE* still has worse runs. *FcE* maintains its median from *small*, only increasing its spread a bit, and, together with *OrB*, achieves the best median overall. *StE* in comparison shows a good median similar to *Lin*, however has a wider spread.

Now under *large* it becomes interesting, as the performance of *Lin* has deteriorated, becoming the worst controller of them all, judging from the median. Under worst-case evaluation of its stable seeds, it is still the best; however, three of the ten seeds already diverged. *ScE* is still the second worst, now again worse than the fully connected version. However, the other large-parameter architectures, *OrB* and *FcE*, show a similar performance as under *medium*, while most others (except for *MLP*) have shifted upwards. The simple *StE* also remains in a good spot, even though it has worsened in comparison to *OrB* (Fig. 4.2).

4.3 Noise on Large Inflation

Similar to what we have seen before in Section 3.2.3, parametric uncertainty seems to affect most trained controllers similarly and generally increases the cost by multiple orders of magnitude. Here, the baseline remains the most robust one, with only its calibrated increase by a factor of 10 under *divergent*, while for the majority it is one between 10^5 and 10^6 in the median case. However, *Lin* seems to be more robust than the other trained policies, "only" increasing by 10^3. When only looking at Fig. 4.3a, one might question whether *Lin* is actually more robust than the others, given its worse performance on the nominal plant. A small relative

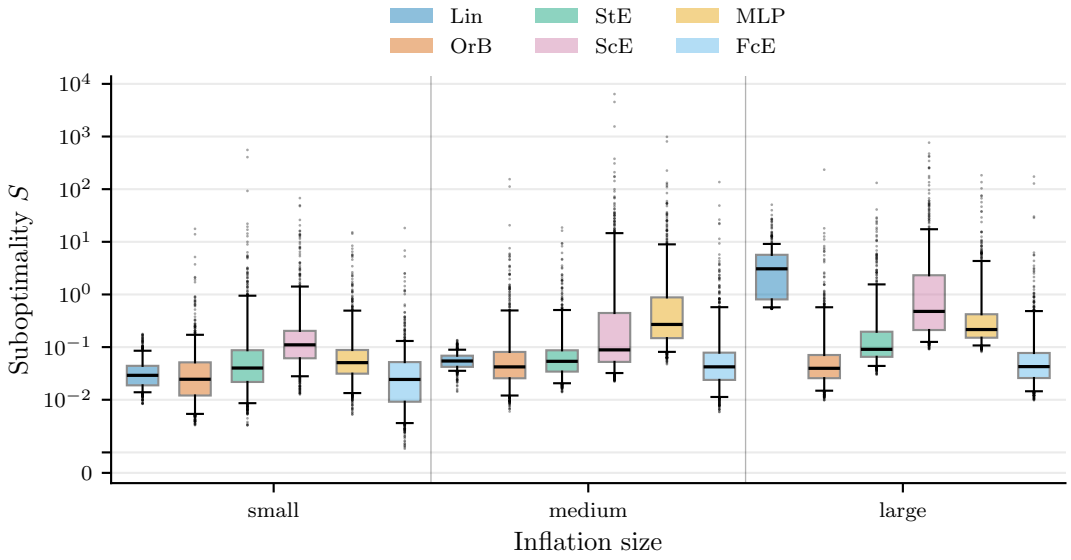
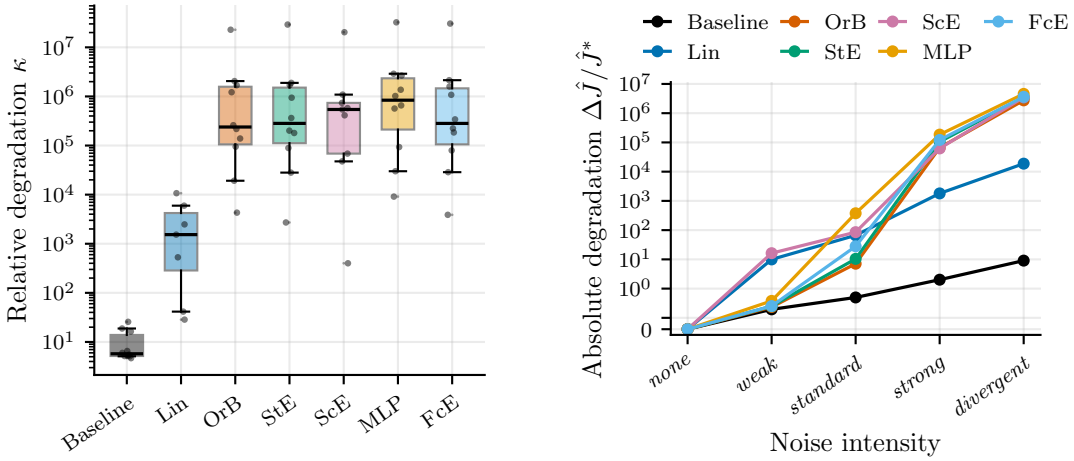


Figure 4.2: Suboptimality S of the PPO variants for each inflation size. The boxes indicate the interquartile range, while the central line marks the median, and the whiskers span the 5th to 95th percentile. Points show episodes outside this range.

increase could then still mean a large absolute one. However, checking Fig. 4.3b, we notice that indeed it also degrades less than the other controllers absolutely.



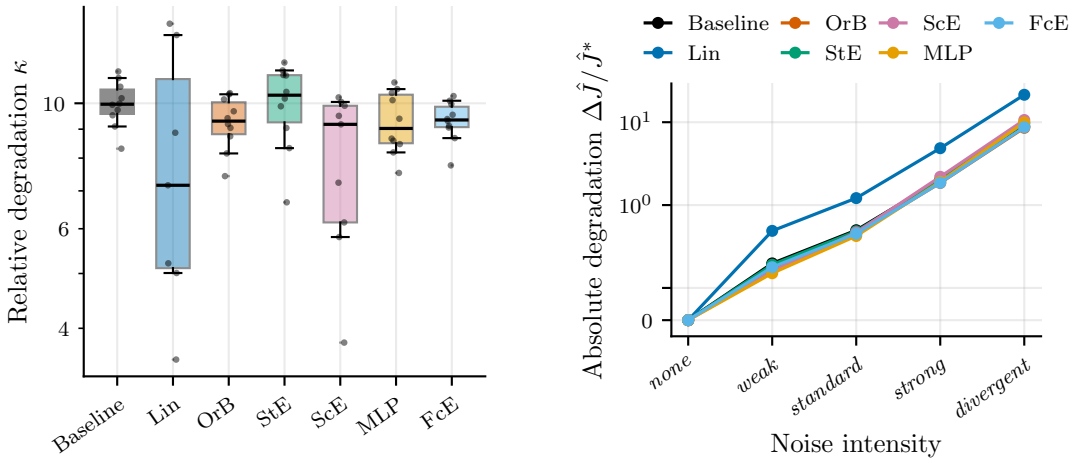
(a) Relative degradation κ at intensity *divergent*.

(b) Absolute degradation $\Delta\hat{J}/\hat{J}^*$ over the intensities.

Figure 4.3: Degradation of the PPO variants under *large* inflation and parametric uncertainty.

For external disturbance, not much can be said, as here all controllers are affected relatively and absolutely in a similar fashion (Fig. 4.4). The only noticeable differ-

ence is a wider range of values for *Lin*, with two noticeably worse seeds pushing its absolute degradation above the others. However, for *Lin* and *ScE*, there also exists one seed each which experiences only a fourfold relative cost increase, compared to the tenfold increase measured at the baseline.



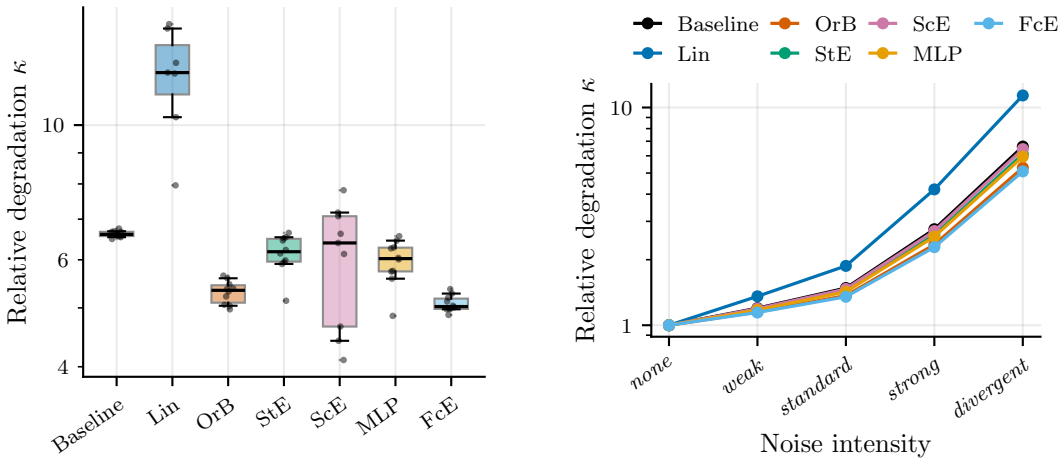
(a) Relative degradation κ at intensity *divergent*.

(b) Absolute degradation $\Delta\hat{J}/\hat{J}^*$ over the intensities.

Figure 4.4: Degradation of the PPO variants under *large* inflation and external disturbance.

Under control noise, we can observe that trained controllers degrade in relation to how well they performed on the nominal loop. This can be seen from the fact that the order of relative degradation in Fig. 4.5a is roughly the same as for the clean plant, which is the exact opposite of what we found in Chapter 3. Generally, however, we can see that all trained controllers except *Lin* are more robust on average than the baseline in relative terms (Fig. 4.5b). In absolute terms, *ScE* shows a worse degradation, which mainly stems from one seed. Also, in relative degradation, its seeds are spread further apart than those of the other controllers. There, in three out of the nine stable seeds, *ScE* degrades less than the other controllers (excluding *Lin*), but then in three others it shows more degradation, making it overall worse on average. More robust are *StE* and *MLP*, as well as *OrB* and *FcE*, showing slightly better robustness properties for the larger controllers.

For measurement noise, we can see that the smaller controller *Lin* remains largely more robust in comparison to the baseline across intensities, both relatively and absolutely (Figs. 4.6a and 4.6b). *ScE* is slightly worse, however far below the baseline in relative degradation, and even beating *Lin* in absolute degradation under *divergent* noise. While *ScE* indeed has a good performance here, its curve is heavily influenced by one outlier seed, which seems to not be affected by noise at all. We can further notice that *MLP* shows the worst degradation here. Also interesting is that *OrB*, *FcE* and *StE* all show slightly worse degradation than the

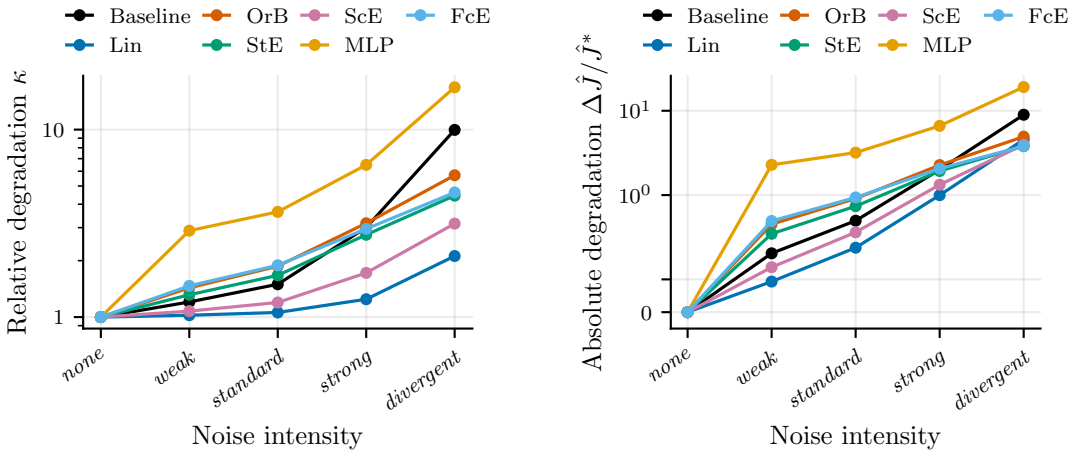


(a) Relative degradation κ at intensity *divergent*.

(b) Relative degradation κ over the intensities.

Figure 4.5: Degradation of the PPO variants under *large* inflation and control noise.

baseline under *weak* and *standard* intensities; however, with increasing strength, they continue to degrade less than the baseline. Under *divergent* noise, in absolute degradation *FcE* and *StE* even reach levels better than *Lin*, which grows similarly to the baseline from *strong* onwards. *OrB* is positioned slightly above (4.93 vs. *Lin* at 4.55). It is still very interesting to see that *StE* performs at the same level as *FcE*, and better than *OrB*, as it uses far fewer parameters than both.



(a) Relative degradation κ.

(b) Absolute degradation $\Delta\hat{J}/\hat{J}^*$.

Figure 4.6: Degradation of the PPO variants under *large* inflation and measurement noise of increasing intensity.

4.4 Discussion

What we notice in our experiments is that under *large* inflation, the largest policies (by the number of trainable parameters) generally perform better than smaller ones on the clean loop, with *StE* being an exception. *FcE* and *OrB* are nearly identical here, and also better than the other controllers if we look at the median, which tells us that the low-dimensional latent state neither improves nor hurts performance here. What is indeed noticeable is that *Lin*, which is much smaller than those, becomes more unstable the larger the inflation grows. However, if we take a look at *StE*, which has only half the parameters of *Lin* or *MLP*, we can see that its performance beats both under the clean loop, and performs only slightly worse in absolute degradation under control noise than *MLP*, while across the other noise channels it remains more robust than it. Noticeable is also *ScE*, which also produces one unstable test seed, and performs worse than the others on the clean loop (excluding *Lin*). If looking at its parameters, its total is similar to *FcE* or *OrB*, but the trainable parameters are comparable to *StE*. Taking a look at its robustness, we can actually see that in absolute degradation, it does not perform much differently than the other architectures, except for control noise, where it does seem that robustness correlates here with the number of parameters available for training.

Overall, we can see that a low-dimensional latent state does not generally guarantee performance or robustness improvement. Instead, under control noise more parameters generally help the controller stay slightly more robust. However, certain low-dimensional structures, as seen with *StE*, can perform similarly to fully parameterized controllers, while containing far fewer parameters (a reduction of more than 90 % compared to *FcE* and *OrB*). Especially under measurement noise and external disturbance, this low-dimensionality offers good robustness properties which are similar to larger controllers, and at or below the baseline. Under parametric uncertainty, all policies degrade similarly, except for *Lin*, which degrades less.

Comparing that to our previous chapter, where we also already noticed that controllers with more parameters are more robust, this explains also why the order in comparison to the clean loop was inverted there—as the worse controllers did not apply any control, this order was flipped, while in our experiment all controllers are acting, and *OrB* and *FcE* were already the best on the clean loop, and thus kept their position. We can also see that the robustness in relation to measurement noise for low-dimensional controllers does indeed still hold; however, it seems as if a very large number of parameters can compensate that. What also stands out is that a hybrid approach of pretraining a large extractor and then a smaller model is comparable with the indirect approaches of our previous chapter. Even closer is using the pseudoinverse of the output matrix like we did in *StE*, which also performed great in relation to its parameter count.

So, to conclude this chapter, we again emphasize that the low-dimensional archi-

tecture one chooses indeed plays a major role in whether the controller can achieve comparable performances, and how robust it is. However, as we have only validated our hypotheses on an artificially inflated example, in the next chapter we create a more realistic environment in which we can compare our approaches.

Chapter 5

Perception-Based Experiments

From the previous artificially inflated inputs, we aim to move to a more realistic example, which shows the robustness of low-dimensional controllers in a scenario involving an image observation as input. We do this by first using our setup from the previous chapter to scale it to the dimension of the images (Section 5.1). Afterwards, we build a new environment (Section 5.2), which combines perception-based observation with multiple actuators acting on our double integrator, before comparing the controllers' performance on it (Section 5.3). Finally, we again apply noise and analyze its effects (Section 5.4).

5.1 Artificial Inflation to Pixel Size

5.1.1 Setting

As mentioned, we take our loop from before (Section 2.3.1.1) and scale it up further. The input dimension in this case becomes 18 432, while we set the output dimension to 256. These numbers were specifically selected to match the dimensions in Section 5.2, such that we can compare both more closely. For the input dimension, this is a 96×96 image stack of two images, and for the output dimension, the number of thrusters. The goal is to separate effects caused by image observation from the increased dimensions later in Section 5.3.

We further note that *StE* cannot be applied at this scale, as the identification for such large matrices is computationally infeasible, since the matrix decompositions would not fit into memory. Additionally, we excluded *Lin*, due to its already weak performance under *large*.

Due to computational complexity and time restrictions, we reduced the training steps to 10^7 and did not run experiments to determine the optimal learning rate here. Instead, we used the optimal learning rate for *large*, and scaled it via

$$\eta_0^{\text{pixel}} = \eta_0^{\text{large}} \sqrt{\frac{200}{18\,432}} \approx 0.104\, \eta_0^{\text{large}}, \tag{5.1}$$

rounded to two significant digits as in (B.1), which leaves us with the values in Table 5.1.

Table 5.1: Initial learning rate η_0 at the *pixel* size, extrapolated from *large* with (5.1).

Solver	*large*	*pixel*
OrB	$6.7 \cdot 10^{-6}$	$7 \cdot 10^{-7}$
ScE	$2.2 \cdot 10^{-6}$	$2.3 \cdot 10^{-7}$
MLP	$6.7 \cdot 10^{-6}$	$7 \cdot 10^{-7}$
FcE	$6.7 \cdot 10^{-6}$	$7 \cdot 10^{-7}$

5.1.2 Results

When we now take a look at the training results, we can see that only for *OrB* and *FcE* we can actually notice a learning process (Fig. 5.1). Here it is interesting to see that *FcE* performs noticeably better than *OrB* in the median (1.37 vs. 2.57), even though they share nearly the same architecture. *ScE* in comparison is mostly flat, with a small improvement at $2 \cdot 10^6$ steps, before flattening again (3.77), while *MLP* only shows a slight decrease of suboptimality towards the end (ending at 5.11). Potentially, the shorter training length and rescaled learning rates could be at fault here, as the original ones were calibrated for triple the step budget.

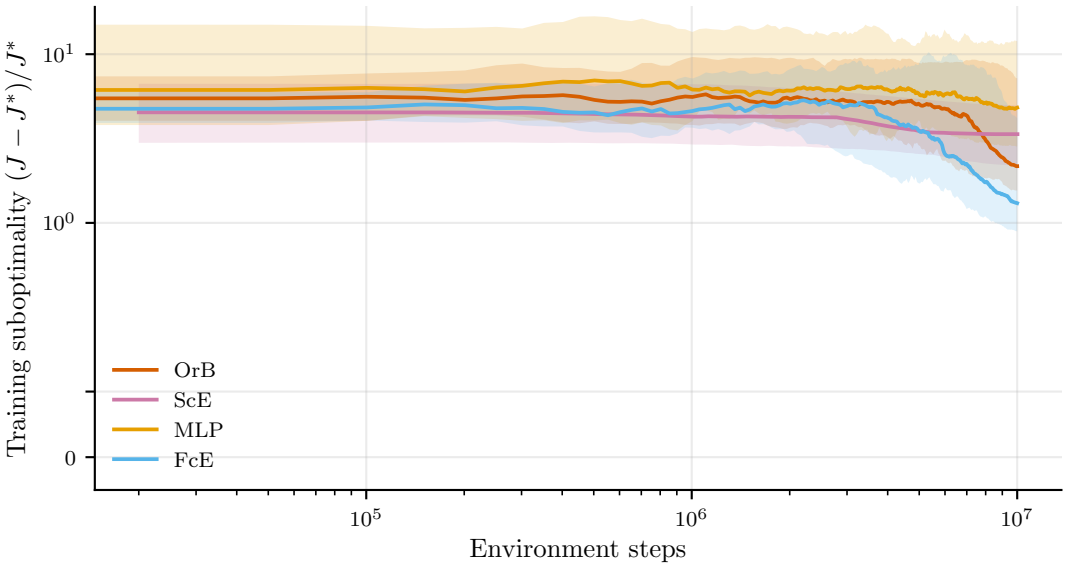


Figure 5.1: Training suboptimality $(J - J^*)/J^*$ of the PPO variants under *pixel* inflation.

Examining an example trajectory, and analyzing how each controller acts on the mass, we notice that, compared to the Baseline, they barely apply any control

input, or are generally more careful with it (Fig. 5.2). This observation indicates that it is difficult for them to propagate any directed actuation through the output lift, which is costly, so that in turn it is cheaper to just accept the position penalty, without wasting additional energy.

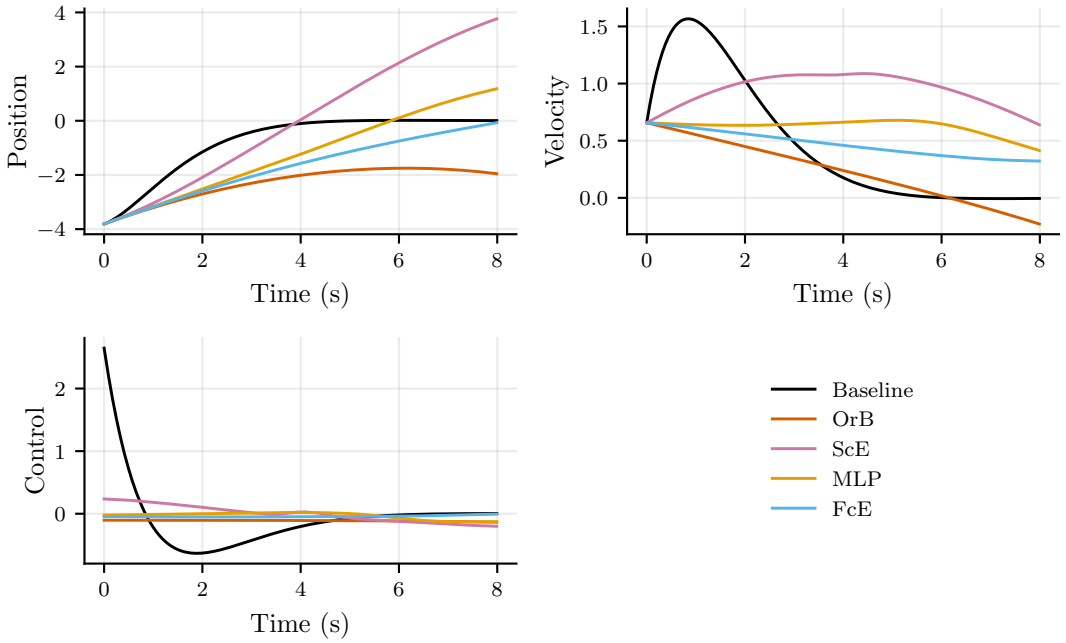


Figure 5.2: Trajectories of the Baseline and the PPO variants under *pixel* inflation from the same initial state.

Finally, we want to investigate how the test suboptimality here compares to that of the previous chapter (Fig. 5.3). Generally, we notice that it has increased strongly across all architectures, which is caused by both the much larger input dimension and the changed step budget and learning rates. However, it can be observed that the policies with a larger total parameter count perform better (judging by the median). *ScE*, which had been worse than *MLP* under *large* inflation, has overtaken it and now sits below it with a median S of 17.5 (35.4 for *MLP*, Table 5.2).

Another noticeable thing, which we also observed in the previous chapter, is the large spread across all architectures, where the worst episode is multiple orders of magnitude larger than the best one (Table 5.2). In the context of some controllers applying little force and thus not steering the mass, this makes sense, as then the initial velocity direction has a much larger influence on the incurred cost. However, there is indeed still a noticeable difference, as *OrB* and *ScE* have a similar spread, but *OrB* has a better median and interquartile range. Both are nevertheless beaten by *FcE*, showing us that the projection in *OrB* is negatively affecting it.

We will revisit these results in the context of the new environment (Section 5.5).

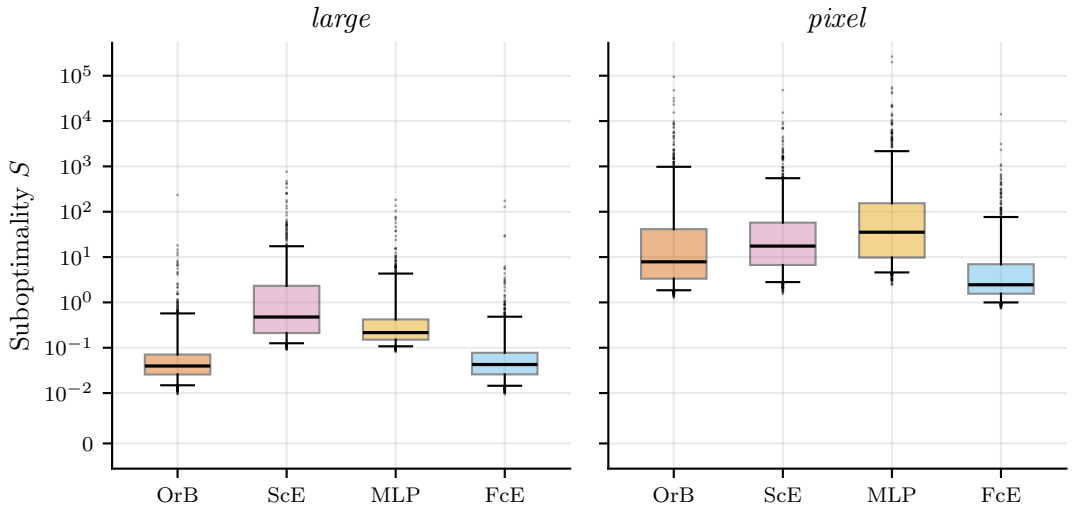


Figure 5.3: Suboptimality S of the PPO variants under *large* (Section 4.2) and *pixel* inflation.

Table 5.2: Median, best and worst test episode of the suboptimality S of the PPO variants under *pixel* inflation, without noise, pooled over the stable seeds.

Solver	OrB	ScE	MLP	FcE
Median	7.86	17.5	35.4	2.45
Best episode	1.25	1.53	2.48	0.738
Worst episode	9.45×10^4	4.8×10^4	2.65×10^5	1.41×10^4

5.2 The New Environment and the Position-Velocity Encoder

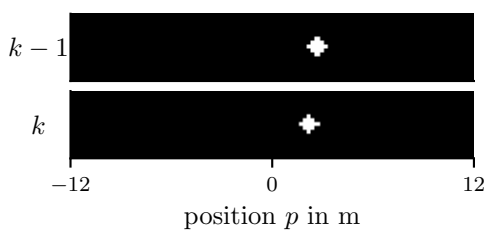
5.2.1 Pixelated Environment

For our perception setup, we still use the underlying double integrator as a basis, however now simulate its actual movement. This simulation is observed by a monochromatic camera producing 96×96-pixel images of a mass with a radius of two pixels moving horizontally in front of a black background. The visible range is from -12 to 12 meters, so a total of 24 meters, corresponding to 0.25 meters per pixel. To be able to see changes between the pictures, we hold any action commanded by the agent for 30 environment steps before capturing another frame and call this interval an *agent step*. The agent then receives a stack of two images as the input. Figure 5.4a shows an example of such an image.

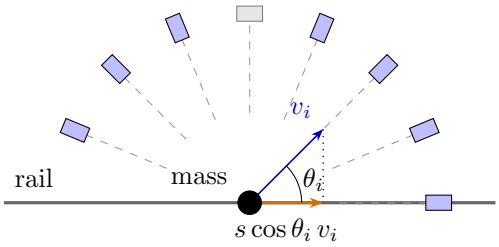
In addition to the input being high-dimensional compared to the actual state, we also modify the way our mass is actuated. We replace the single actuator with 256 thrusters, distributed equally around a half circle, so thruster $i \in \{0, \dots, 255\}$

sits at an angle $\theta_i = \pi i/256$. Each thruster can actuate in the range $v_i \in [-1, 1]$, however, due to the angle, only $s \cos\theta_i$ effective acceleration is produced on the mass, where s is a scaling factor.[1]

This leads to some thrusters being more effective than others, while all are charged for the commanded output, so some energy might be lost. For example, the only vertically acting thruster $\theta = \pi/2$ cannot affect the mass in any way, and would thus only waste energy if used. Figure 5.4b visualizes the thrust directions for eight thrusters.



(a) The two frames of one observation, cropped to rows around the mass.



(b) Schematic of the effect of eight thrusters. Created with Claude Code.

Figure 5.4: Observation and actuation in the camera environment.

With our chosen frame and action repeat, a pixel difference between two frames can only be guaranteed at velocities above 0.83 m/s. Otherwise, motion is only visible if the mass crosses a pixel boundary, making velocity generally hard to identify. For a better approximation the number of frames stacked could be increased, or anti-aliasing could be used to display changes better.

5.2.2 Position-Velocity Encoder

For this new environment, we introduce another feature extractor, which is based on robotic priors used in *ScE* [21, 4]. Called Position-Velocity Encoders [22], it tries to encode a position and velocity state from an observation, by learning a latent position state, and then reconstructing the velocity state using temporal differences. As we specifically look for position and velocity in our states, we can construct even more specific priors, again in line with our reasoning that knowledge about the embodiment helps in designing the controller architecture (Section 2.4). The priors used in [22, Sec. III-B] are:

1. Variation: "Positions of relevant things vary."
2. Slowness: "Positions change slowly."

[1] The most energy-efficient and thus cheapest way to produce an acceleration a is the least-norm allocation, which uses the thrusters based on their output effectiveness $v_i = a\cos\theta_i/(128{\cdot}s)$, where $128 = \sum_i \cos^2\theta_i$. The horizontal, and thus most effective, thruster saturates at $a = 128 \cdot s$. We thus choose s here such that 1.5 times the maximum acceleration demanded by our optimal controller can be reached without clipping, which is $26\,\mathrm{m/s^2}$, so $s = 26/128$.

3. Inertia: "Velocities change slowly."
4. Conservation: "Velocity magnitudes change slowly."
5. Controllability: "Controllable things are relevant."

For our implementation, we follow the paper's recommended approach. Each frame is input into a convolutional neural network (CNN), which encodes a latent position state $s_k^{(\mathrm{p})}$ from it. Its dimension is a hyperparameter, but is set to five throughout the paper [22, Sec. III-C], which we follow. The velocity is then calculated as the difference between two frames, or $s_k^{(\mathrm{v})} = \alpha \left(s_k^{(\mathrm{p})} - s_{k-1}^{(\mathrm{p})} \right)$, where α is the inverse of the time between two agent steps. For the weighting of our prior loss terms, we use the combination suggested for similar tasks in [22, Tab. I], and thus exclude the controllability prior. For the training process we further follow the one proposed in [22, Sec. III-C]. For details on how exactly the total loss function is constructed, we refer the reader to the original paper [22]. We train this extractor with data of 20 000 agent steps, which is split into episodes of 810 environment steps, which at our action repeat corresponds to 27 agent steps. Similar to before, we use K_0 acting on the true state in 70 % with least-norm allocation through the thrusters and random, uniformly distributed actuation drawn from $[-1, 1]^{256}$ in 30 % of the time, before freezing the extractor's weights and using it in front of our PPO construct (Section 4.1). We refer to it as Position-Velocity Encoder + PPO (*PVE*).

ScE is pretrained with a similar data split for this setting.

5.3 Effect of Visual Inputs on the Solvers

5.3.1 Setting

For the experiments here, we use the optimal gain under our action repeat (Section 5.2.1) as our reference and compare it with different approaches, which here consist of *FcE*, *ScE*, *OrB* as well as our *PVE* (Section 5.2.2). As they reuse the already introduced architectures, they will not be repeated here. The only difference is that we increase the latent dimension of *ScE* to 5, which is the same as *PVE*, after initial results showed a weak performance with 2. The optimal controller for this setup actually does not act based on the perception, but instead has access to the real state, while the output remains our thrusters, for which it uses the least-norm allocation. It is therefore optimal, and serves as the lower bound for our comparison. Given the knowledge it has about the true state, we call it "Oracle (true state)" in our visualizations. For the other approaches, except for *PVE*, which takes images as an input, we flatten the images, such that they receive an 18 432-dimensional input vector. The learning rates we use here are identical to the ones extrapolated in Table 5.1. As we did not use *PVE* (Section 5.2.2) before, we copy the learning rate from *ScE* for it, as it is the most similar (both use robotic priors).

However, we also modify some of our PPO parameters for this setting. Not mentioned parameters equate to their Chapter 4 values. Specifically, we

- set $\gamma = 0.97$, which gives us around $1/(1-\gamma) \approx 33$ effective agent steps. Multiplying this with our agent step interval of 0.3 s gives us roughly 10 s of effective horizon, equal to what we used before;
- raise the reward clipping threshold from 10 to ∞ (Section 4.1.2), so the reward is never clipped;
- charge the input cost on the command before clipping, though only enact the clipped command;
- end the episode with a penalty if the mass exits the observable area. The penalty is set to twice the cost that a mass sitting at the edge of the screen would incur over a whole episode.

The last three points relate a bit to each other, and were changes that were implemented after initial test failures. Here, we noticed that by charging the cost only after clipping, the controller never learned to actually control the mass in some cases. For these, it would produce increasingly larger command inputs, but could never return from this instability, as every action above this threshold had the same effect. Also, clipping the reward like in Chapter 4 had the side effect that leaving the screen became an economically reasonable choice, so the model chose this behavior more often. Combined with this, we started charging the penalty, which promotes staying on screen even more.

We also need to slightly adapt our test protocol (Section 2.3.3) to this new environment. To maintain similarity to the previous experiments, we choose 27 agent steps per episode during training, corresponding to 810 environment steps. The overall training step budget is chosen as 10^7 agent steps. For one test episode we use $\lceil 2000/30 \rceil = 67$ agent steps, corresponding to 2010 environment steps.

5.3.2 Additional Metrics

With an increase in environment complexity, it becomes increasingly difficult to trace back performance to certain behaviors. To change that, we introduce two additional metrics for this chapter, which allow us to assess the quality of the encoded latent state, as well as a controller's ability to actuate effectively.

Inspired by Jonschkowski et al. [22, Sec. IV-C] and Lesort et al. [30, Sec. 4.3], we implement a regression-based probe on the latent state. To not replicate the exact setup from the PVE paper [22], which uses a neural network as the probe, we instead rely on a stricter, yet simpler, construction similar to the one used by Alain and Bengio [2]. We use ridge regression, which keeps the system well conditioned for strongly correlated predictors [15], and collect pairs of the true and the latent state over 20 episodes with 27 agent steps each post-training. We train with a ridge of 10^{-3} on the first 14 episodes of the data, before computing the coefficient

of determination on the steps $\mathcal{T}$ of the remaining six episodes,

$$R^2 = 1 - \frac{\sum_{k\in\mathcal{T}}(y_k - \hat{y}_k)^2}{\sum_{k\in\mathcal{T}}(y_k - \bar{y})^2}, \tag{5.2}$$

where y_k is the true value, $\hat{y}_k$ the prediction from the latent state and $\bar{y}$ its mean over the training steps. Similarly, we compute it for control, using the command from the oracle as the target. Overall, an $R^2 = 1$ tells us that the latent state has encoded this feature perfectly linearly, while a value close to 0 means that it is not better than just predicting the training mean.

A second metric we use is wasted energy via the control input. As mentioned in Section 5.2.1, the resulting actuation u_k is calculated via $u_k = c^\top v_k$, where v_k is the commanded actuation, and $c \coloneqq s\,[\cos\theta_0, \ldots, \cos\theta_{255}]^\top$, due to the construction of the thrusters. We compare the least-norm allocation $u_k\, c/\|c\|^2$ with the commanded input, to determine what amount of energy is directed into the null space of $c^\top$, and thus never reaches the mass. The resulting equation, calculated for each step, is

$$\|v_k\|^2 - \frac{u_k^2}{\|c\|^2}, \tag{5.3}$$

which is then averaged over the steps of all 100 episodes.

5.3.3 Results

Looking at the evolution of the training suboptimality, we can again differentiate between two groups. While the fully trainable architectures achieve suboptimalities below 1 with a small spread, the ones that mostly consist of frozen parameters do show a decrease in the beginning, but then flatten out over the last $9 \cdot 10^6$ steps. *FcE* and *OrB* do the exact opposite, and rather increase their learning slope over this range (Fig. 5.5).

Table 5.3 shows us how we can differentiate our architectures in relation to low-dimensionality. *FcE*, *OrB* and *ScE* are in total parameters of the same magnitude; however, of those, *ScE* consists of $\approx 99.74\,\%$ frozen parameters, and its trainable parameters are rather comparable to *PVE*, of which the extractor is around eight times smaller. For *FcE* and *OrB*, the main difference is only the projection into the two-dimensional latent space for *OrB*.

If we now take a look at the test suboptimality for our architectures (Fig. 5.6), we notice the same order as in the training suboptimality, and that *FcE*, being the largest in trainable parameters, performs the best here, while the projection of *OrB* seems to only increase the cost. *ScE* and *PVE* achieve similar medians; however, the interquartile range of *PVE* is further spread out, having both episodes performing better than *ScE*, but also some which are worse.

Putting the suboptimality into the context of example trajectories observed in Fig. B.10, we can see that *PVE* is not controlling the mass at all and *ScE* only slightly, with both failing to effectively move the mass.

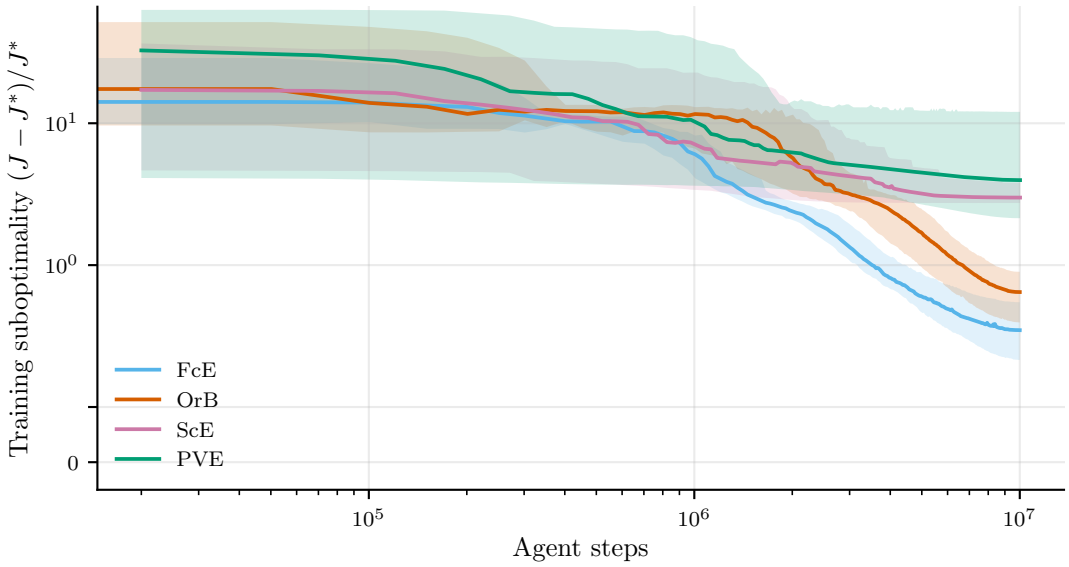


Figure 5.5: Training suboptimality $(J - J^*)/J^*$ of the PPO variants.

Table 5.3: Actor parameters of the PPO variants in the camera environment, split into those trained by PPO and those frozen after pretraining.

Solver	Trainable	Frozen	Total
FcE	9 672 064	0	9 672 064
OrB	9 655 808	0	9 655 808
ScE	24 960	9 570 693	9 595 653
PVE	21 504	1 261 445	1 282 949

Table 5.4 shows us that *FcE* encodes both position and velocity fairly well from the observations, but the control less so ($R^2 \approx 0.60$). Calculating a control input from these is then probably delegated to the PPO network, and the extractor is therefore mainly an encoder. For *OrB*, we notice a difference here: The latent state partially encodes the oracle's control (0.68), but barely position (0.10) and velocity (-0.02), indicating that its extractor moves the mentioned calculation before the latent projection. As we see that *OrB* wastes some thrust energy in Fig. 5.7b, the size of the PPO head could be too little to perfectly translate this signal into effective thrust. We can notice a similar pattern in both *ScE* and *PVE*, where the latent state indeed contains both control and position correlation, at even higher levels than *FcE*, indicating that the encoding is effective. However, *ScE* is missing velocity observations, which makes it difficult for it to see the effects of its control. As a result, it loses the mass outside the screen in 6.4 % of the test episodes, getting charged the exit penalty (Fig. 5.7a). Encoding-wise, *PVE* captures the position of its point almost perfectly, and indicates well how the controller should steer the mass. But similar to the previous cases, however the most severe, the

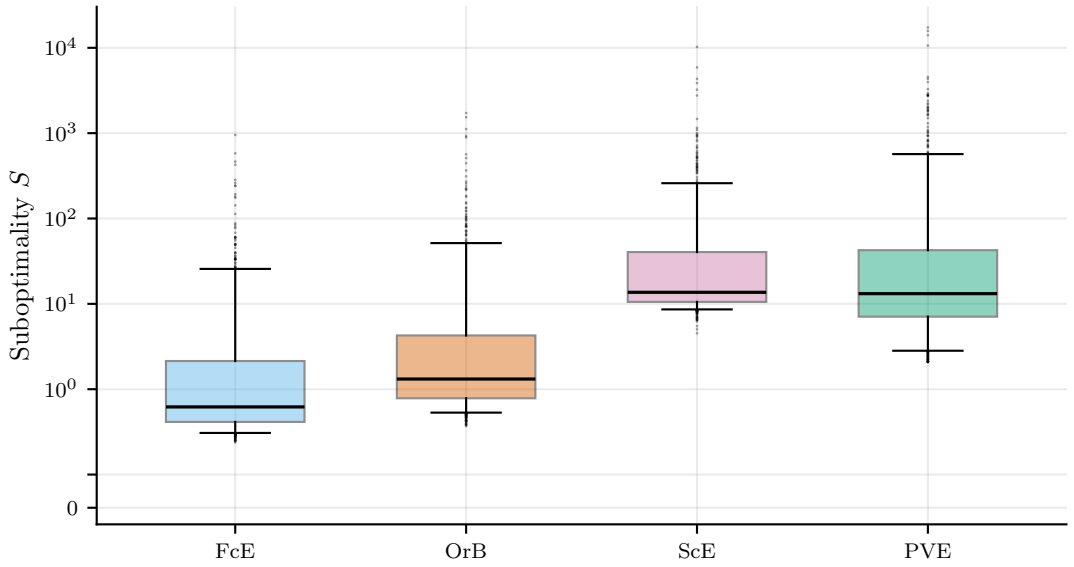


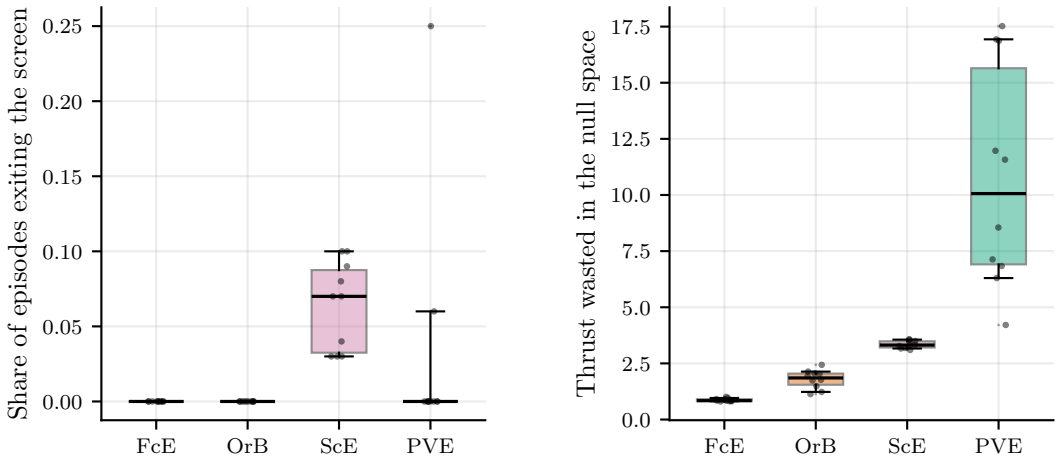
Figure 5.6: Suboptimality S of the PPO variants. The boxes indicate the interquartile range, while the central line marks the median, and the whiskers span the 5th to 95th percentile. Points show episodes outside this range.

complexity of PPO here seems to be too reduced to effectively translate it onto the thrusters. Judging by the major waste in the null space, *PVE* cannot differentiate well between effective and wasteful thrusters, and therefore does not move at all in some cases (Fig. B.10). As a result, it also loses the mass in 3.1 % of the episodes. So, for both *ScE* and *PVE*, a higher suboptimality than *FcE* or *OrB* is produced, where the latter do not exit any episode early.

Table 5.4: Held-out R^2 of a linear probe from the latent state of each PPO variant to the true position, the true velocity and the control of the oracle, together with the final training suboptimality.

	FcE	OrB	ScE	PVE
Probe R^2, position	0.895 ± 0.046	0.103 ± 0.238	0.941 ± 0.075	0.992 ± 0.021
Probe R^2, velocity	0.794 ± 0.192	-0.023 ± 0.164	0.014 ± 0.149	0.684 ± 0.383
Probe R^2, optimal control	0.600 ± 0.234	0.676 ± 0.145	0.691 ± 0.098	0.954 ± 0.034
Training suboptimality $(J - J^*)/J^*$	0.337 ± 0.0264	0.641 ± 0.139	3.07 ± 0.215	5.17 ± 3.51

As a summary, we note that a low-dimensional latent layer can indeed produce effective encodings of the true state, even with far fewer parameters. However, the complexity of the following network must allow this encoding to be translated back into action, which in our case it fails to do, possibly because of its size. This effect is even more enlarged if the extractor is frozen, and thus both parts are trained separately, and the majority of the training happens only on 0.3 % (*ScE*) or 1.7 % (*PVE*) of the parameters. Increasing the complexity of the actor until performances comparable to *FcE* can be achieved is left for future research.



(a) Share of test episodes in which the mass exited the screen.

(b) Thruster energy per agent step exceeding the least-norm allocation.

Figure 5.7: Terminated episodes and wasted thrust of the PPO variants.

5.4 Noisy Perception

5.4.1 Setting

As our environment setup changed drastically, the cost increases associated with the noise levels calculated from before no longer apply. Thus, we again ran a calibration with the optimal controller to set them to respective levels (Section A.3). With these new values, we can use them like before to change how the loop behaves. Parametric uncertainty still changes the underlying model, external disturbance still acts on the position of our mass, and control noise changes the commanded actuation outputs. Measurement noise now affects the visual input, as shown in Fig. 5.8, by applying Gaussian noise to all the pixels in the image. Here, we set the levels ourselves, as the optimal controller is not acting from the camera image, but rather from the true state, so this noise does not affect it. Still, the chosen values are found in the appendix (Table A.3).

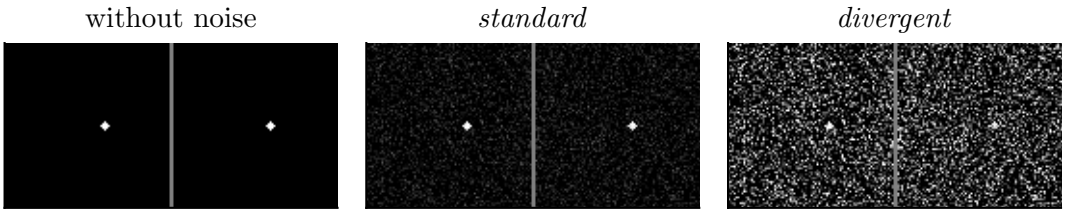


Figure 5.8: An observation without measurement noise, and with measurement noise of intensity *standard* and *divergent*.

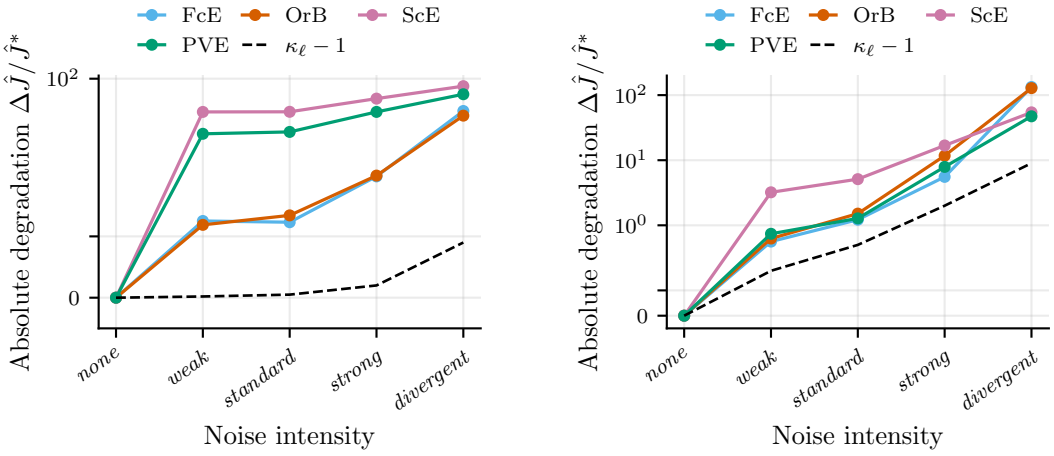
We now run the trained controllers from Section 5.3 on noise-affected trajectories

at different strengths under the test protocol introduced in Section 5.3.1. For evaluation, we use the absolute degradation (2.36) of each controller in relation to its clean run in Section 5.3 and display it in units of the cost of the oracle on the clean plant. For all noise channels, except measurement noise, we also display the calibrated level $\kappa_\ell - 1$ as a dashed line for reference.

5.4.2 Results

As we have already noted in Section 5.3.3, *ScE* and *PVE* perform far worse than *FcE* and *OrB*, and are barely acting on the mass. We keep this in mind for our analysis of the results, and drop relative degradation as a metric for this section, as it is heavily skewed.

Looking now at parametric uncertainty, we do notice that *FcE* and *OrB* seem to be more robust under lower intensities, but sharply increase in absolute degradation from *standard* onwards, while *PVE* and *ScE* already increase a lot under *weak*, but then remain relatively stable (Fig. 5.9a). While initially suspected to indicate robustness, the plateauing is actually caused by penalties for leaving the screen. Under *weak*, for *ScE* the mass exits 52 % of the time, for *PVE* it sits at 36 %. As under *divergent* those values sit at 74 % and 62 % respectively, we can attribute this cap on the increase to this.



(a) Parametric uncertainty.

(b) External disturbance.

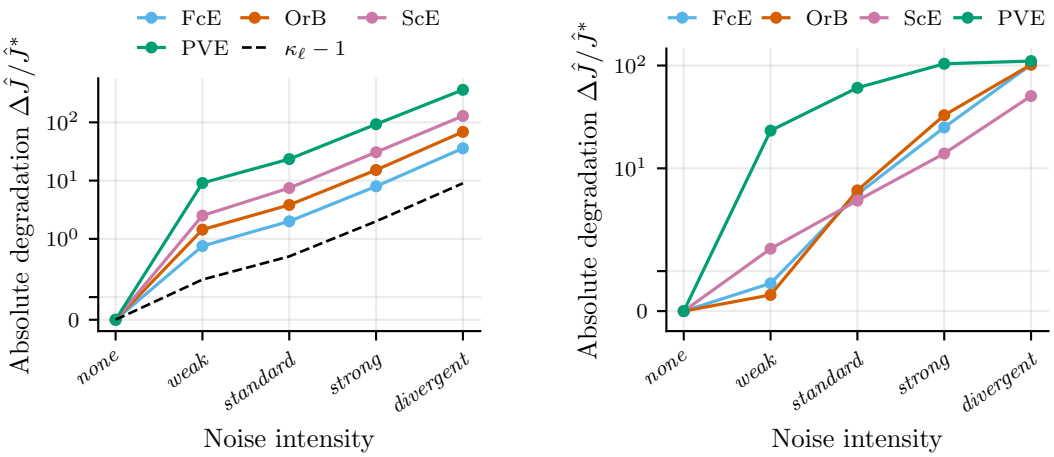
Figure 5.9: Absolute degradation $\Delta\hat{J}/\hat{J}^*$ of the PPO variants in the camera environment under parametric uncertainty and external disturbance.

If we take a look at external disturbance now, we can also see that *PVE* seems to be similarly affected as *FcE* and *OrB* under lower intensities, with *ScE* performing worse, while under *divergent* our largely pretrained controllers outperform the others (Fig. 5.9b). What is generally noticeable is that the oracle does outperform all of them; however, this is expected, as it is not affected by the high complexity

of the environment.

Looking at the effect of control noise again, we find support for our previous hypothesis that *PVE* does not manage to correctly address its thrusters. Similarly to *ScE*, which wastes more thrust in the null space than the other architectures, this leads to a higher increase in absolute degradation under control noise, as overall more control is applied (Fig. 5.10a). Again, we notice overall that the larger architectures handle control noise much better than the smaller ones. For *OrB* compared with *FcE*, the influence of only encoding the control signal in its latent state also seems to produce an adverse effect.

Taking now a look at measurement noise, it is quite difficult to interpret the result. On the one hand, the lower effect of it on *ScE* and the flattening of the curve for *PVE* (Fig. 5.10b) could be an indicator that those low-dimensional controllers are less affected by measurement noise, which is something we suspected from the results in our previous chapters. However, this is not the case here. Instead, for *ScE* it is the effect we noticed before, where controllers that are already non-functional on the clean plant are generally less affected by noise on their inputs. For *PVE*, any noise intensity applied leads to a high percentage of the mass exiting the screen (18, 43, 73 and 79 %), which consequently ends the episode early with our previously discussed penalty (Section 5.3.1). As a result, the curve flattens at the higher intensities, as the cost cannot increase much more. Therefore, as *FcE* and *OrB* do not show any difference under *divergent*, there is no establishable basis for the claim that low-dimensional latents improve robustness against measurement noise, at least for this setting.



(a) Control noise.

(b) Measurement noise.

Figure 5.10: Absolute degradation $\Delta\hat{J}/\hat{J}^*$ of the PPO variants in the camera environment under control and measurement noise.

5.5 Discussion

Concluding this chapter, we first of all note that, on the clean loop, the fully connected approach, *FcE*, is unbeaten in terms of performance, and a low-dimensional latent layer as in *OrB* does not show any improvement; instead, it performs worse. This was already prominent under our pixel-size inflation, and has now persisted in our visual input example. Similarly, we also were not able to effectively implement *ScE*, such that it could show strong improvements over the other controllers. While it still functioned under *large*, even though worse than the other controllers, with only a fraction of trainable parameters, under visual input it became practically useless, most likely due to its inability to encode velocity. *PVE*, however, does encode velocity ($R^2 \approx 0.68$), and position and control even better than *FcE*, with a much smaller extractor. A previously mentioned increase in the number of frames stacked or improved imaging (anti-aliasing, Section 5.2.1) could potentially improve these encodings for *FcE*. In the end, the actor head failed to translate those encodings into the correct actions, which hints at the PPO network being too small, if the extractor is frozen before. This could have also contributed to *ScE*'s worse performance.

Looking at the effect of noise, we again notice that parametric uncertainty affects already worse performing controllers generally more. For external disturbances, we observe a pattern under *divergent* intensity, where *ScE* and *PVE* both degrade noticeably less than *FcE* and *OrB*, which are not able to contain the mass in most of the episodes. A possible robustness stemming from the frozen extractors could be the case, but cannot be clearly attributed by our data. Under control noise, we find that controllers which cannot coordinate their thrusters well, and waste more thrust, are consequently more affected by it. Also, the trend that fully parameterized controllers are less affected by it continues in our experiments. Finally, we notice, contrary to our previous findings, that controllers carrying a low-dimensional latent state do not show more robustness to measurement noise, as the only lower absolute degradation of *ScE* can be explained by ineffectiveness of the controller on the clean plant.

Overall, based on our observations in this chapter, controllers containing a low-dimensional latent state, as well as far fewer trainable parameters, can be constructed using pretrained extractors. Making them perform at suboptimalities similar to fully connected controllers however requires a careful selection of the actor size, as well as an effective encoding of all relevant properties. The latter is especially what matters when it comes to robustness. If both are fulfilled, it is probable that such a controller can outperform a full network, while being more space efficient.

Chapter 6

Conclusion

6.1 Conclusions

In this thesis, we have shown that simply enforcing low-dimensional structures or latent layers does not improve the performance of controllers for overparameterized systems; instead, it might even hurt performance. However, when combining sufficiently pretrained latent projectors with the right controller architecture, they can achieve similar performances to fully connected networks, while being more efficient in their parameter count and training time. While this could be clearly shown for LQR, under RL only tendencies were observed, requiring further investigation.

We further found conflicting results on whether low-dimensional latent states can improve robustness to measurement noise. While LQR solvers and model-informed *StE* present a tendency here, it is insignificant compared to the spread of the seeds. For the camera environment, we could not observe any of that. This, however, might be related to the first point, where our RL approaches already showed bad performance under the clean loop.

Regarding the latent state, it is also important to ensure that not only control information is carried here, but also information about the observed state. Otherwise, robustness to control noise might worsen.

In relation to that, we also note that a low degradation under control noise can also hint at a barely acting controller, and is not always a sign of robustness. Still, in our experiments, low-dimensional controllers were affected slightly more by it than fully parameterized ones, which we observed across all experiments.

6.2 Summary of Contributions

We have presented that combining pretrained encoders with a trainable network can reduce the parameter count effectively on a toy example, while noting common problems in relation to architecture design. Failing to translate its success to a

vision-based problem brought additional pitfalls to light. Further, we presented that fully parameterized controllers offer, if only slightly, more robustness to control noise, while for the other noise types we could only observe tendencies, which open possibilities for further research here.

6.3 Limitations

Throughout our experiments, we noticed multiple times how variable metrics across seeds are. While we intended to mitigate this by using multiple seeds, in some cases relying only on the median might not be as expressive as it should be. Additionally, by excluding diverging seeds, we might have unintentionally improved the values for these controllers.

What is also important to note is the setup for Chapter 5. Here, due to the simplicity of the camera setup, as well as the extrapolation of learning rates instead of calibration, the quality of the results has suffered. Given the high percentage of exits during episodes here, as well as non-functional controllers using pretrained encoders, it is difficult to draw much from these experiments. While these decisions were mostly caused by time constraints of this thesis, it would be interesting to repeat these experiments at a more appropriate complexity in the future.

6.4 Future Research

From where this thesis ends, there are different directions, which we have left for future research and discussion. These include, but are not limited to, whether:

1. and how the number of parameters can be distributed more effectively between the extractor and the actor, to achieve performances comparable to a fully connected network;
2. improving the imaging setup leads to better position and velocity encodings for fully trained networks;
3. our results hold for more complex systems, e.g., a walking humanoid robot;
4. these findings are transferable to real-world applications, i.e., if this setup were built and tested.

Finally, we also want to mention that, while these are interesting directions, the data collected in our experiments permits additional analysis. For example, while we captured the compute time and environment steps needed, a different angle could include this in the evaluation of the approaches. Taken together with the limitations mentioned, we believe that while we found low-dimensional controllers to perform worse, there is more research to be done on this topic.

Appendix A

Nomenclature

A.1 Notation

The following table contains the conventions regarding symbols used throughout this thesis, together with the place where each is defined:

Symbol	**Meaning**	**Defined in**
Sets and operators		
$\mathbb{R}$, $\mathbb{C}$, $\mathbb{N}_0$	real numbers, complex numbers, natural numbers including zero	
I_n, $\mathbf{1}_n$, e_1	identity matrix and all-ones vector of dimension n, first unit vector	
$M^\top$, $M^\dagger$	transpose and (truncated) pseudoinverse of M	Algorithm 4
tr, rank, diag	trace, rank, diagonal matrix	
$\lambda_i(M)$, $\sigma_i(M)$	i-th eigenvalue and i-th singular value of M	
$U\Sigma V^\top$	singular value decomposition, $\Sigma = \operatorname{diag}(\sigma_1, \ldots, \sigma_r)$	(2.37)
$\rho(M)$	spectral radius, $\max_i \lvert\lambda_i(M)\rvert$	Section 2.1.2
$M \succeq 0$, $M \succ 0$	M symmetric positive semi-definite / positive definite	
$\lVert\cdot\rVert$, $\lVert\cdot\rVert_F$, $\lVert\cdot\rVert_*$	Euclidean (spectral) norm, Frobenius norm, nuclear norm	
$\lvert\cdot\rvert$	absolute value, cardinality of a set, size of a policy	Definition 2.1
$\odot$	elementwise product	(2.24)

Symbol	Meaning	Defined in
$\mathbb{E}$, $\hat{\mathbb{E}}_k$	expectation, empirical average over the collected steps	(4.1)
$\mathcal{N}(\mu, \Sigma)$	Gaussian distribution with mean μ and covariance Σ	
∇f, $\widehat{\nabla f}$	gradient of f and its estimate	(3.2), Algorithm 3
System description		
n, m, q	dimension of the state, input and output	(2.1)
t, k, h	continuous time, discrete step index, sampling period	(2.3)
$x_k \in \mathbb{R}^n$	state	(2.4)
$u_k \in \mathbb{R}^m$	control input of the plant	(2.4)
$y_k \in \mathbb{R}^q$	observed output	(2.1)
p, v	position and velocity of the double integrator, $x = [\, p,\ v\,]^\top$	(2.7)
A_c, B_c	continuous-time system and input matrix	(2.1)
A, B	their zero-order-hold discretizations	(2.5)
C, D	output and feedthrough matrix; $D = 0$ throughout	(2.1)
A_K	closed-loop matrix, $A_K = A - BK$	(2.16)
Optimal control		
Q, R	state and input cost matrices, $Q \succeq 0$, $R \succ 0$	(2.10)
$V_T(x_0, u)$	continuous-time cost over the finite horizon T	(2.10)
$J(x_0, u)$, $J(x_0, K)$	infinite-horizon cost of an input sequence or of a gain	(2.11)
$K \in \mathbb{R}^{m \times n}$	feedback gain, $u_k = -Kx_k$	(2.12)
K^*, u_k^*	optimal gain and optimal input	(2.14)
P, $J^*(x)$	stabilizing solution of the DARE and optimal cost-to-go $x^\top P x$	(2.13), (2.15)
P_K	solution of the Lyapunov equation for a fixed K	(2.16)
$\mathcal{K}$	set of stabilizing gains	(2.17)
K_0	stabilizing initial gain of the uninflated system, $K_0 = [\, 0.1,\ 0.5\,]$	Section 2.3.3

Symbol	**Meaning**	**Defined in**
$\mathcal{D}$, Σ_0	distribution of x_0 and its second moment $\mathbb{E}[x_0 x_0^\top]$	(2.18)
$f(K)$	expected cost of a gain, $\mathrm{tr}(P_K \Sigma_0)$	(2.18)
Inflated system		
d_{in}, d_{out}	inflated dimension of the controller input and output	(2.19)
U_1, U_2	lifts with orthonormal columns and rows	(2.19)
$K \in \mathbb{R}^{d_{\mathrm{out}} \times d_{\mathrm{in}}}$	inflated feedback gain, $v_k = -K y_k$	(2.20)
v_k, $\tilde{v}_k \in \mathbb{R}^{d_{\mathrm{out}}}$	controller output, before and after control noise	(2.20), (2.24)
$\tilde{Q}$, $\tilde{R}$	inflated state and input cost, $\tilde{Q} = I_{d_{\mathrm{in}}}$, $\tilde{R} = I_{d_{\mathrm{out}}}$; $U_1^\top \tilde{Q} U_1 = Q = I_n$	(2.22)
$\tilde{K}_0$, $K_\perp$	inflated initial gain $U_2^\top K_0 U_1^\top + K_\perp$ and its full-rank component, $U_2 K_\perp U_1 = 0$	(2.39)
$\tilde{A}$, $\tilde{B}$, $\tilde{\Sigma}_0$	model in the inflated coordinates, $U_1 A U_1^\top$, $U_1 B U_2$, $U_1 \Sigma_0 U_1^\top$	Section 3.1
Noise		
n_k^{m}, σ_{m}	measurement noise and its standard deviation	(2.23)
n_k^{c}, σ_{c}	control noise (relative) and its standard deviation	(2.24)
Δ_k^A, Δ_k^B, σ_{p}	parametric perturbations and their standard deviation	(2.26)
n_k^{d}, σ_{d}	external disturbance and its amplitude	(2.27)
c, $\varnothing$	noise condition (channels and intensity), noise-free condition	(2.30)
ℓ, c_ℓ, κ_ℓ	noise intensity, a channel at intensity ℓ, its cost ratio	(2.38)
Evaluation and training protocol		
T	horizon of a rollout	(2.30)
s, i	index of the seed and of the episode	(2.30)
M, N	number of seeds and of episodes per seed	(2.40)
$\mathcal{S}$	set of seeds whose controller is stable on the clean loop	(2.40)
$J_T^{(i,s)}(K; c)$	empirical cost of one episode	(2.30)
$\hat{J}_T(K; c)$, $\mathcal{F}$	mean empirical cost, set of finite episodes	(2.33)

Symbol	Meaning	Defined in
$S^{(i,s)}(K;c)$, $\hat{S}(K;c)$	suboptimality of one episode and its mean	(2.32)
$\hat{J}_T^{(s)}(K;c)$, $\kappa^{(s)}(K;c)$	mean empirical cost of seed s, its ratio to the clean loop	(2.34)
$\kappa(K;c)$	cost ratio to the clean loop over all seeds, $\kappa_\ell = \kappa(K^*;c_\ell)$	(2.35)
$\Delta\hat{J}/\hat{J}^*(K;c)$	absolute degradation, cost increase under noise in units of the clean cost of K^*	(2.36)
$D(K;c)$	divergence rate	(2.40)
$\lvert K\rvert$, r	size and rank of a linear policy	Definition 2.1
j	iteration index of a solver	Definition 2.2
g_j, $\bar{g}_j$	convergence signal and its moving mean	Definition 2.2
N_w, N_p, ε_rel	window, patience and relative tolerance of the convergence rule	Definition 2.2
LQR solvers		
η	step size (learning rate)	Algorithm 1
Σ_K	state correlation matrix of a gain	(3.1)
λ	weight of the nuclear-norm regularizer	(3.3)
K^+, $\bar{r}$	candidate gain of S-PI, number of its singular values	(3.3)
G	gradient mapping	(3.4)
β, ε	step-size reduction factor and stopping tolerance of S-PI	Algorithm 2
σ_e, w_k	excitation amplitude and excitation noise for identification	(3.6)
A_id, B_id, C_id	system matrices returned by N4SID	(3.5)
$\hat{A}$, $\hat{B}$, $\hat{C}$, $\hat{n}$	identified model in the basis with orthonormal $\hat{C}$, and its order	(3.5)
Θ	upper triangular factor of the QR decomposition $C_\mathrm{id} = \hat{C}\Theta$	(3.5)
$\hat{x}_k$	encoded state $\hat{C}^\top y_k$	(4.4)
N_traj, l, r	number of perturbation pairs, rollout length, smoothing radius	Algorithm 3
Ξ_i	perturbation of the gain	Algorithm 3
$c_k^\pm$, $\hat{J}_i^\pm$	stage cost and rollout cost of $K \pm \Xi_i$	Algorithm 3
δ_i, $\bar{\delta}$	cost difference of a pair and its mean	Algorithm 3

Symbol	Meaning	Defined in
$\hat{\Sigma}_K$, $\hat{f}(K)$	estimated state correlation and cost	Algorithm 3
Reinforcement learning		
θ, π_θ, μ	actor parameters, policy, mean of the policy	(4.1), (4.3)
ϕ, V_ϕ	critic parameters and critic	Algorithm 6
$r_k(\theta)$, $\hat{A}_k$	probability ratio and advantage estimate	(4.1)
L^{CLIP}, $\varepsilon_{\mathrm{clip}}$	clipped surrogate objective and clip range	(4.1)
γ	discount factor	Section 4.1.2
η_0, N_{env}, $N_{\max}$	initial learning rate, environment steps so far, step budget	(4.2)
η_0^{large}, η_0^{pixel}	initial learning rate at the *large* and *pixel* size	(5.1)
W	weight matrix of *Lin*, $W = -K$	(4.3)
f_ψ, ψ	scalable encoder and its parameters	Algorithm 7
$a_{\max}$	action bound	Algorithm 7
$\mathcal{L}$, $\omega_1, \ldots, \omega_4$	prior loss and its weights	(4.5)
z_k, d_{z}	output of the bottleneck extractor and its dimension	(4.6)
U_{b}, d_{b}	orthonormal bottleneck projection and bottleneck dimension	(4.6)

A.2 Noise Intensities

The settings used for the different noise intensities, obtained by the calibration (2.38) on the non-inflated system. It is important to note that parametric noise is resampled at every step, and that the disturbance noise uses its sinusoidal variant (2.27).

Table A.2: Standard deviation per noise channel and intensity, calibrated to the cost ratio κ_ℓ of (2.38). Every value reaches its ratio within 0.9 %.

Channel	*none*	*weak*	*standard*	*strong*	*divergent*
Cost ratio κ_ℓ	1	1.2	1.5	3	10
Measurement σ_m	0	0.389	0.607	1.22	2.58
Control σ_c	0	1.07	1.67	3.19	5.68
Parametric σ_p	0	0.0211	0.0304	0.0451	0.0582
Disturbance σ_d	0	0.00767	0.0122	0.0246	0.0519

A.3 Noise Intensities of the Perception Environment

The settings used for the different noise intensities in the perception-based environment of Section 5.2, obtained by the calibration (2.38) with the optimal controller acting on the true state as reference. As measurement noise is not calibrated but set by hand (Section 5.4.1), its values indicate the standard deviation σ_m of the noise drawn. Pixel brightness is scaled to $[0, 1]$, so after applying the noise, the image will be clipped to this range. We chose a value of 0.5 for *divergent* based on visual inspection, and then halved the value for each lower intensity.

Table A.3: Standard deviation per noise channel and intensity, calibrated to the cost ratio κ_ℓ of (2.38). Every value reaches its ratio within 9.4 %. Measurement noise is set by hand and not calibrated.

Channel	*none*	*weak*	*standard*	*strong*	*divergent*
Cost ratio κ_ℓ	1	1.2	1.5	3	10
Control σ_c	0	1.31	2.13	4.24	8.96
Parametric σ_p	0	0.0158	0.0167	0.0232	0.0359
Disturbance σ_d	0	0.00737	0.0115	0.0231	0.0494
Measurement σ_m	0	0.0625	0.125	0.25	0.5

A.4 Policy Architectures

The actor architectures of the PPO variants introduced in Section 4.1.3.

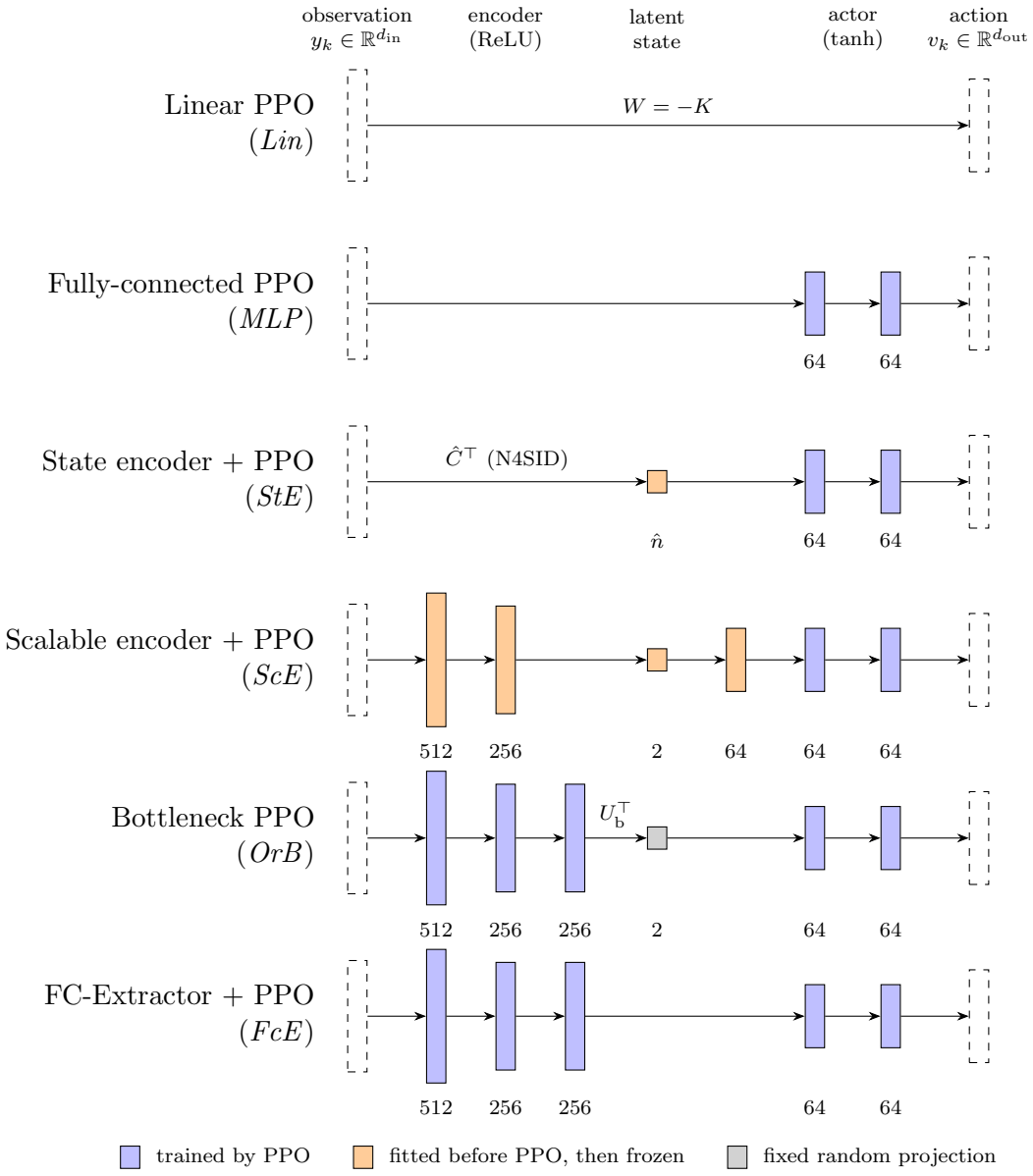


Figure A.1: Actor architectures of our PPO variants. For each layer, the number below indicates its neurons. The critic receives the same encoder structure and uses its own 64-64 tanh perceptron with a scalar output. Visualization created with Claude Code.

Appendix B

Complete Simulation Results

Here the results from some hyperparameter tuning are shown. For simplicity, mostly the ones under *large* inflation are shown, unless named otherwise, as this is the most challenging setting from what we have observed. The remaining plots and tables can be found in the repository [27].

B.1 Learning Rate of *TrP*

Figure B.1 and Table B.1 show the results for *large* inflation.

Table B.1: TrP under *large* inflation, mean and standard deviation over the seeds.

Learning rate η	Training suboptimality	Iterations	Iterations to 1 %
10^{-6}	0.00173	2 390	1 039
3×10^{-6}	6.05×10^{-4}	1 123	342
10^{-5}	1.63×10^{-4}	454	84
3×10^{-5}	$(4.04 \pm 0.02) \times 10^{-5}$	$1\,306 \pm 1$	$1\,040 \pm 1$
10^{-4}	1.13	64	–
1.2×10^{-3}	1.13	20 000	–

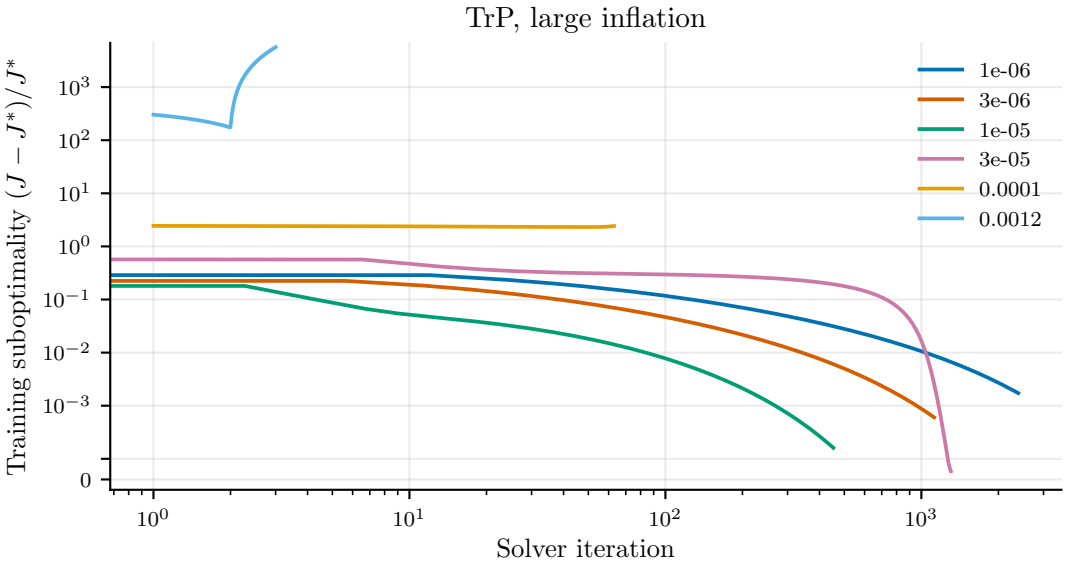


Figure B.1: Training suboptimality of *TrP* over the solver iterations for each learning rate under *large* inflation. Median and range over ten seeds.

B.2 Nuclear-Norm Weight of S-PI

Figure B.2 show the results for *large* inflation.

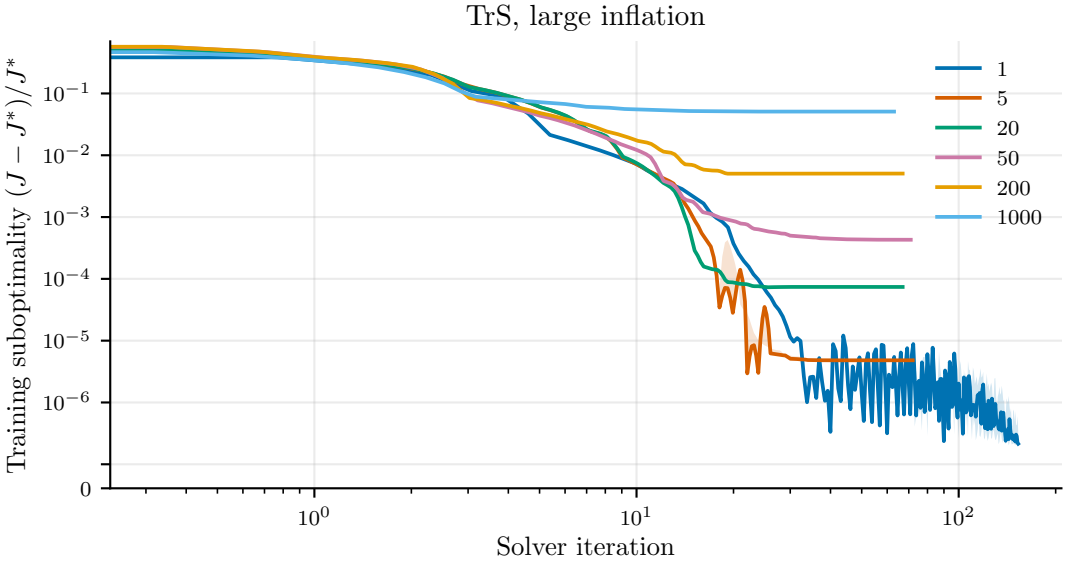


Figure B.2: Training suboptimality of *TrS* over the solver iterations for each nuclear-norm weight λ under *large* inflation. Median and range over ten seeds.

B.3 N4SID Identification Length

Figures B.3 and B.4 show the results for *large* and *medium* inflation over all trajectory lengths and Hankel horizons.

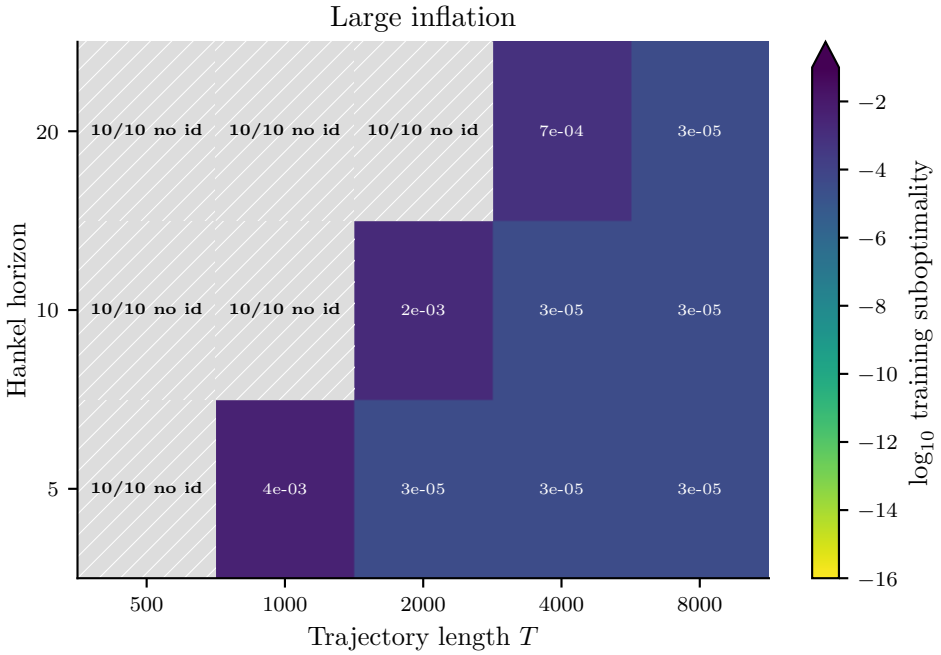


Figure B.3: Median training suboptimality of *IdP* per trajectory length T and Hankel horizon under *large* inflation.

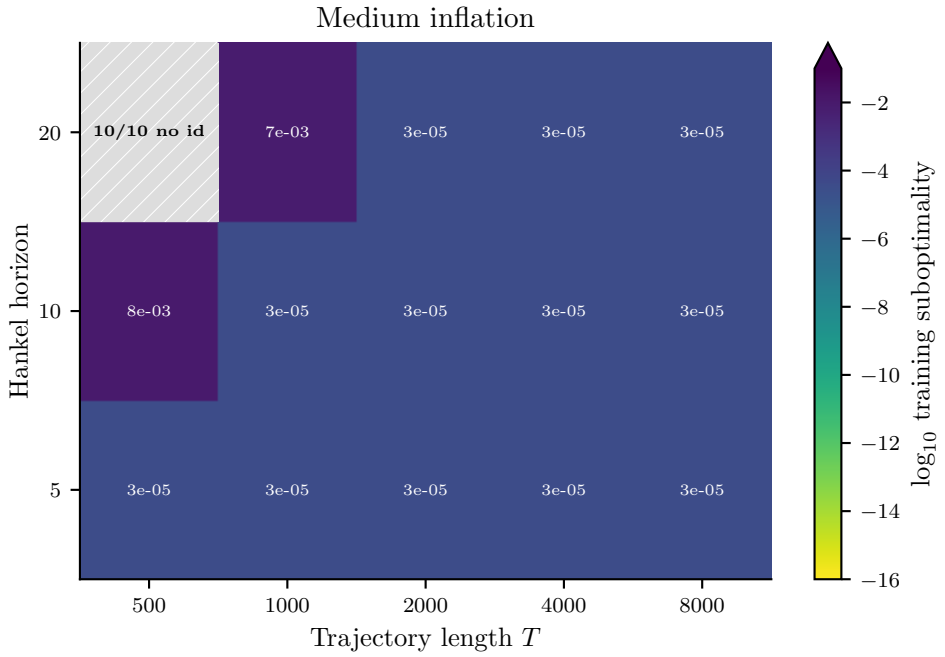


Figure B.4: Median training suboptimality of *IdP* per trajectory length T and Hankel horizon under *medium* inflation.

B.4 Sample Size

Figures B.5 and B.6 show the results of *MfN* for *large* inflation, Table B.2 the training suboptimality for all inflation sizes.

Table B.2: MfN per rollout length l and number of trajectories N_{traj} for each inflation size, mean and standard deviation over the stable seeds.

		Training suboptimality			
l	N_{traj}	*base*	*small*	*medium*	*large*
200	100	0.187 ± 0.329	0.351 ± 0.290	0.498 ± 0.241	0.998 ± 0.189
	200	0.0780 ± 0.0033	0.218 ± 0.064	0.282 ± 0.099	0.631 ± 0.224
500	100	0.00253 ± 0.00125	0.00346 ± 0.00252	0.00741 ± 0.00727	0.337 ± 0.445
	200	0.00334 ± 0.00091	0.00226 ± 0.00255	0.00216 ± 0.00135	0.00658 ± 0.00482
800	100	0.00297 ± 0.00121	0.00296 ± 0.00249	0.117 ± 0.357	0.273 ± 0.437
	200	0.00384 ± 0.00146	0.00183 ± 0.00128	0.228 ± 0.477	0.00566 ± 0.00462

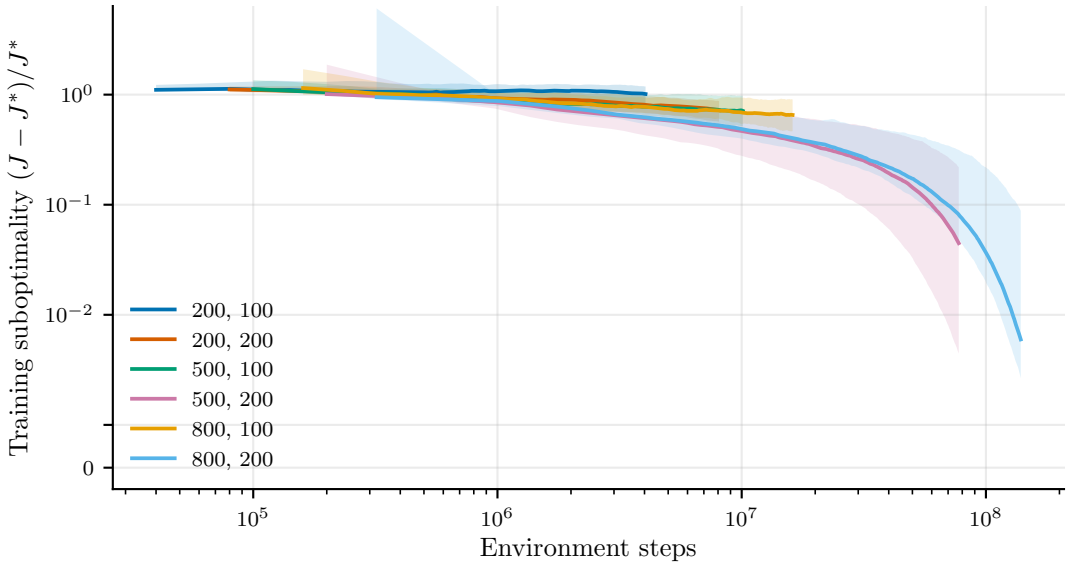


Figure B.5: Training suboptimality of *MfN* over the environment steps for each rollout length l and number of trajectories N_{traj} under *large* inflation. Median and range over ten seeds.

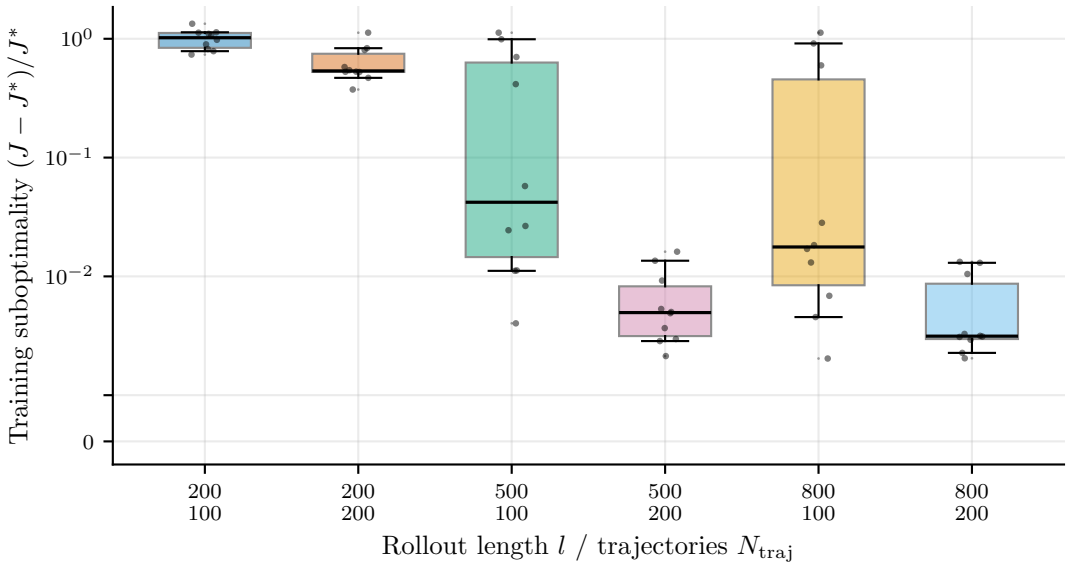


Figure B.6: Final training suboptimality of *MfN* for each rollout length l and number of trajectories N_{traj} under *large* inflation.

B.5 Learning Rate of *MfN*

Figure B.7 and Table B.3 show the results for *large* inflation.

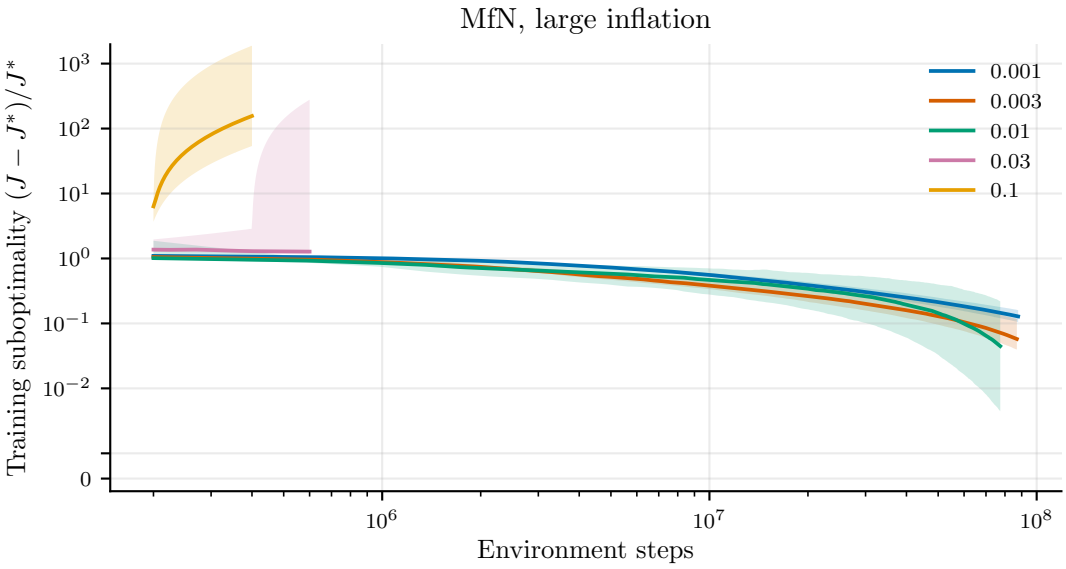


Figure B.7: Training suboptimality of *MfN* over the environment steps for each learning rate under *large* inflation. Median and range over ten seeds.

Table B.3: MfN under *large* inflation, mean and standard deviation over the seeds. Times to 1 % average over the seeds that reach it. Stable counts the seeds whose controller left the plant stable and kept the mass contained in every test episode.

Learning rate η	Stable	Training suboptimality	Env. steps	Env. steps to 1 %
0.001	10/10	0.0982 ± 0.0257	$(1.23 \pm 0.22) \times 10^8$	–
0.003	10/10	0.0386 ± 0.0171	$(1.23 \pm 0.16) \times 10^8$	–
0.01	10/10	0.00658 ± 0.00482	$(1.22 \pm 0.21) \times 10^8$	$(1.02 \pm 0.22) \times 10^8$
0.03	10/10	0.694 ± 0.599	$(7.41 \pm 7.83) \times 10^7$	$(1.40 \pm 0.07) \times 10^8$
0.1	10/10	1.13	$(6.87 \pm 0.74) \times 10^5$	–

B.6 Step Size and Nuclear-Norm Weight of *MfS*

Here, we show different combinations of values for the step size and λ under *large* inflation (Table B.4). For $\lambda = 400$, no rank reduction could be observed (λ too low), while for $\lambda = 4 \times 10^6$ every controller except those with the smallest step size was unstable (λ too high). From the remaining values, we picked a combination that exhibits rank reduction, while at the same time having a low suboptimality.

Table B.4: MfS per nuclear-norm weight λ and step size η under *large* inflation, mean and standard deviation over the stable seeds. Every stable run takes 50 iterations and 10^7 environment steps. Stable counts the seeds whose controller left the plant stable and kept the mass contained in every test episode.

λ	Step size η	Stable	Training suboptimality	Gain rank
4×10^2	3.5×10^{-10}	10/10	1.12	100.0
	3.5×10^{-9}	10/10	1.09 ± 0.03	100.0
	3.5×10^{-8}	10/10	0.682 ± 0.156	100.0
	3.5×10^{-7}	10/10	0.308 ± 0.003	100.0
	3.5×10^{-6}	10/10	1.13	100.0
4×10^3	3.5×10^{-10}	10/10	1.12	100.0
	3.5×10^{-9}	10/10	1.07 ± 0.04	100.0
	3.5×10^{-8}	10/10	0.638 ± 0.005	100.0
	3.5×10^{-7}	10/10	0.373 ± 0.003	96.7 ± 2.1
	3.5×10^{-6}	10/10	0.531 ± 0.018	81.7 ± 5.0
4×10^4	3.5×10^{-10}	10/10	1.12	100.0
	3.5×10^{-9}	10/10	1.04	100.0
	3.5×10^{-8}	10/10	0.698 ± 0.006	53.3 ± 4.5
	3.5×10^{-7}	10/10	11.9 ± 2.1	58.8 ± 4.9
	3.5×10^{-6}	10/10	6.05 ± 2.13	28.1 ± 10.7
4×10^5	3.5×10^{-10}	10/10	1.12	100.0
	3.5×10^{-9}	10/10	1.13	15.6 ± 0.5
	3.5×10^{-8}	0/10	–	–
	3.5×10^{-7}	0/10	–	–
	3.5×10^{-6}	0/10	–	–
4×10^6	3.5×10^{-10}	10/10	1.22	15.4 ± 0.7
	3.5×10^{-9}	0/10	–	–
	3.5×10^{-8}	0/10	–	–
	3.5×10^{-7}	0/10	–	–
	3.5×10^{-6}	0/10	–	–

B.7 Parameters for the RL Solvers

B.7.1 Deviations from SB3 Standard Parameters

Table B.5 lists the parameters of the PPO solvers in Chapter 4 that differ from the defaults of SB3 (version 2.9.0) [57, 56, 58]. All other parameters keep their default values.

Table B.5: Parameters of the PPO solvers that differ from the SB3 defaults.

Parameter	SB3 default	Used here
Discount factor γ	0.99	0.999
Learning rate	3×10^{-4}, constant	Table B.6, annealed by (4.2)
Target KL divergence	none	0.02
Initial log standard deviation	0	-2
Orthogonal initialization	yes	no

B.7.2 Learning Rates for the RL Solvers

For each solver and inflation size, we train three seeds with each initial learning rate

$$\eta_0 \in 3 \times 10^{-4} \sqrt{\frac{10}{d_{\text{in}}}} \cdot \left\{0.01,\ \tfrac{1}{30},\ 0.1,\ \tfrac{1}{3},\ 1\right\}, \tag{B.1}$$

rounded to two digits. The final rates were then chosen based on inspection of the training curves (Table B.6). For *Lin* under *large* every gain becomes unstable during training (Fig. B.8).

Table B.6: Initial learning rate η_0 of the PPO solvers per inflation size.

Solver	*small*	*medium*	*large*
Lin	10^{-4}	4.5×10^{-6}	2.2×10^{-6}
OrB	3×10^{-5}	4.5×10^{-5}	6.7×10^{-6}
StE	10^{-4}	4.5×10^{-5}	6.7×10^{-6}
ScE	10^{-4}	1.3×10^{-5}	2.2×10^{-6}
MLP	3×10^{-5}	1.3×10^{-5}	6.7×10^{-6}
FcE	3×10^{-5}	1.3×10^{-5}	6.7×10^{-6}

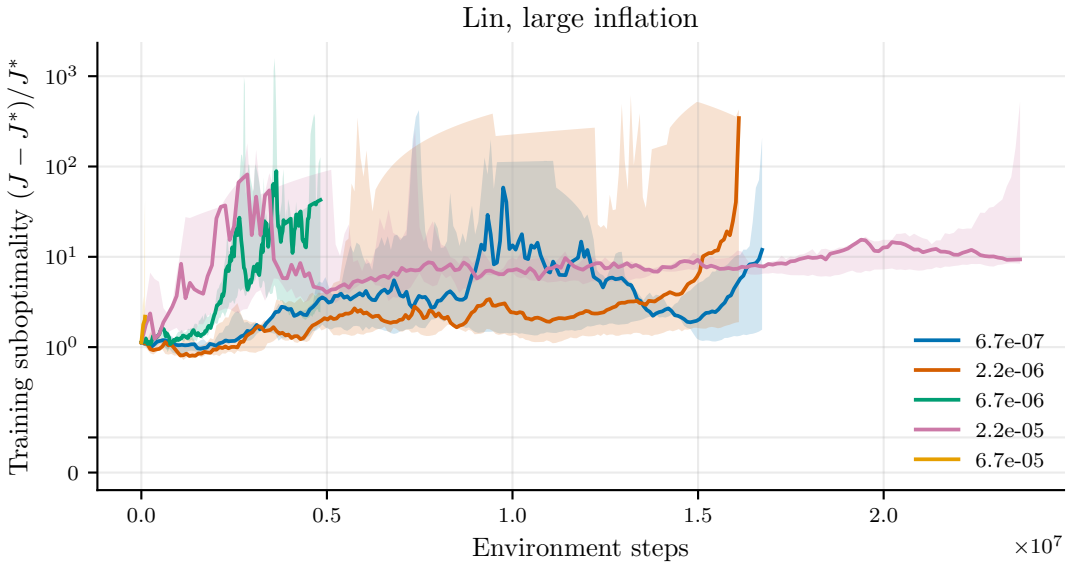


Figure B.8: Training suboptimality for different learning rates for *Lin* under *large* inflation.

B.7.3 Stopping Rule of the RL Solvers

Table B.7: Stopping-rule settings with the lowest median regret.

ε_{rel}	N_{p}	N_{w}	Budget used	Median regret	Max. regret
0.001	40	10	20 %	19 %	279 %
0.003	40	10	20 %	20 %	279 %
0.01	40	10	20 %	20 %	279 %
0.03	40	10	19 %	23 %	279 %
0.001	40	5	16 %	26 %	279 %
0.003	40	5	16 %	26 %	279 %
0.01	40	5	16 %	26 %	279 %
0.03	40	5	16 %	27 %	279 %
0.001	40	3	15 %	29 %	288 %
0.003	40	3	15 %	29 %	288 %

Table B.8: RL runs best policy, as a share of the step budget, and the regret of the policy after 75 % of the budget, of the final policy, and of the policy with the best $\bar{g}_j$.

Solver	Size	Best policy at	Regret at 75 %	Final regret	Regret by $\bar{g}_j$
Lin	*small*	98 %	0.79 %	0.018 %	0.11 %
OrB	*small*	99 %	1.2 %	0.034 %	1.9 %
StE	*small*	87 %	1.6 %	0.22 %	0.2 %
ScE	*small*	80 %	2.1 %	0.94 %	1.4 %
MLP	*small*	87 %	1.9 %	0.43 %	0.64 %
FcE	*small*	99 %	1.4 %	0.019 %	0.97 %
Lin	*medium*	98 %	1.4 %	0.14 %	0.077 %
OrB	*medium*	94 %	2.2 %	0.046 %	0.13 %
StE	*medium*	98 %	2.7 %	0.066 %	1.8 %
ScE	*medium*	98 %	2.1 %	0.17 %	8.1 %
MLP	*medium*	57 %	3.7 %	1.9 %	3.7 %
FcE	*medium*	98 %	0.98 %	0.047 %	0.72 %
Lin	*large*	3.7 %	1664 %	391 %	391 %
OrB	*large*	97 %	0.94 %	0.084 %	0.084 %
StE	*large*	98 %	1 %	0.03 %	0.044 %
ScE	*large*	99 %	6.5 %	0.002 %	1.2 %
MLP	*large*	94 %	3.2 %	0.55 %	1.2 %
FcE	*large*	99 %	1 %	0.022 %	1.3 %

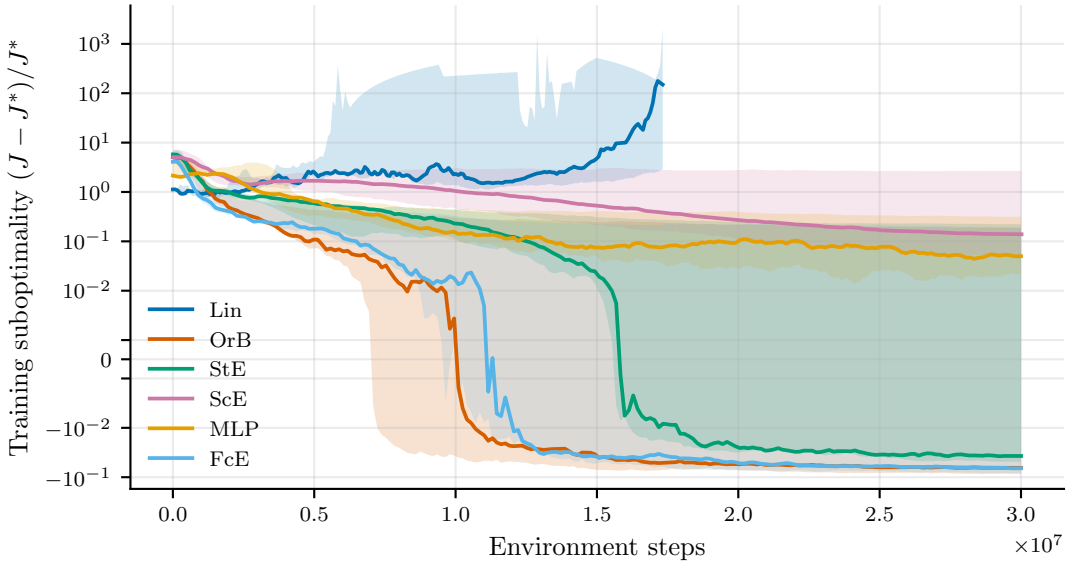


Figure B.9: Training suboptimality of the RL solvers over the environment steps under *large* inflation.

B.7.4 Additional Plots for the Perception Experiments

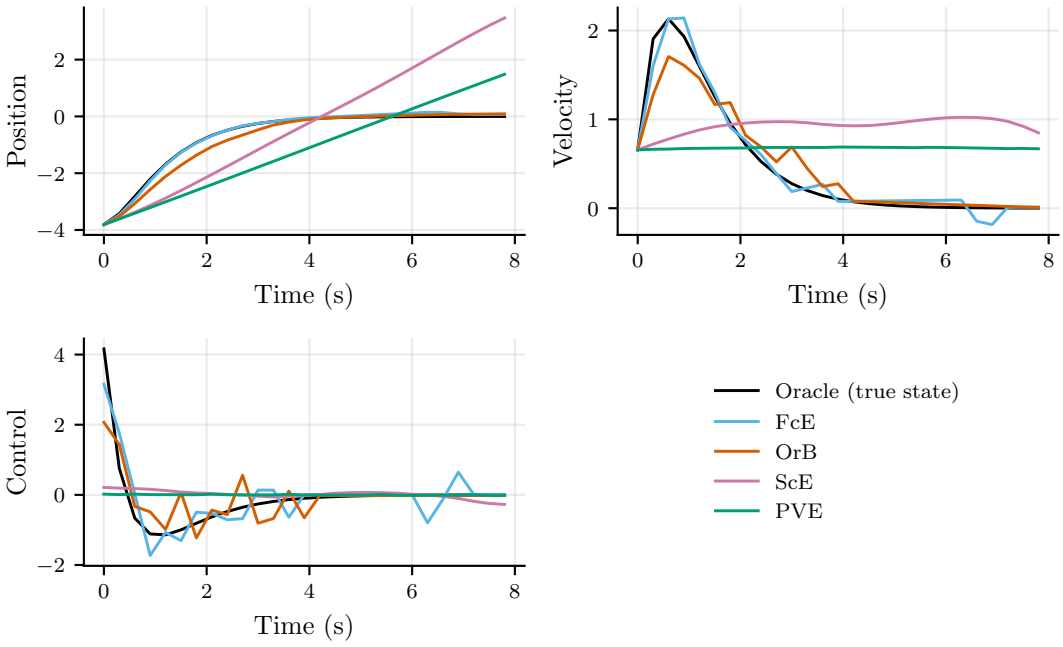


Figure B.10: Trajectories of the PPO variants for one sampled episode under one seed.

Bibliography

[1] Alekh Agarwal, Sham Kakade, Akshay Krishnamurthy, and Wen Sun. FLAMBE: Structural complexity and representation learning of low rank MDPs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 20095–20107, 2020.

[2] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *5th International Conference on Learning Representations (ICLR), Workshop Track*, 2017. arXiv:1610.01644.

[3] Nihat Ay. Geometric design principles for brains of embodied agents. *KI – Künstliche Intelligenz*, 29(4):389–399, 2015.

[4] Nicolò Botteghi, Khaled Alaa, Mannes Poel, Beril Sirmacek, Christoph Brune, Abeje Mersha, and Stefano Stramigioli. Low dimensional state representation learning with robotics priors in continuous action spaces. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021.

[5] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

[6] Adwait Datar and Nihat Ay. Universal and cheap control architectures for embodied agents. Poster, Institute for Data Science Foundations, Hamburg University of Technology, 2025.

[7] Andrea d'Avella, Philippe Saltiel, and Emilio Bizzi. Combinations of muscle synergies in the construction of a natural motor behavior. *Nature Neuroscience*, 6(3):300–308, 2003.

[8] Sarah Dean, Horia Mania, Nikolai Matni, Benjamin Recht, and Stephen Tu. On the sample complexity of the linear quadratic regulator. *Foundations of Computational Mathematics*, 20(4):633–679, 2020.

[9] Sarah Dean, Nikolai Matni, Benjamin Recht, and Vickie Ye. Robust guarantees for perception-based control. In *Proceedings of the 2nd Conference on Learning for Dynamics and Control (L4DC)*, volume 120 of *PMLR*, 2020. arXiv:1907.03680.

[10] Mostafa Dehghani et al. Scaling vision transformers to 22 billion parameters. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 7480–7512, 2023. arXiv:2302.05442.

[11] Misha Denil, Babak Shakibi, Laurent Dinh, Marc'Aurelio Ranzato, and Nando de Freitas. Predicting parameters in deep learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, 2013. arXiv:1306.0543.

[12] John C. Duchi, Michael I. Jordan, Martin J. Wainwright, and Andre Wibisono. Optimal rates for zero-order convex optimization: The power of two function evaluations. *IEEE Transactions on Information Theory*, 61(5):2788–2806, 2015.

[13] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on PPO and TRPO. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:2005.12729.

[14] Maryam Fazel, Rong Ge, Sham M. Kakade, and Mehran Mesbahi. Global convergence of policy gradient methods for the linear quadratic regulator. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 1467–1476, 2018.

[15] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[16] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[17] Chloe Ching-Yun Hsu, Celestine Mendler-Dünner, and Moritz Hardt. Revisiting design choices in proximal policy optimization. arXiv:2009.10897 [cs.LG], 2020.

[18] Bin Hu, Kaiqing Zhang, Na Li, Mehran Mesbahi, Maryam Fazel, and Tamer Başar. Toward a theoretical foundation of policy optimization for learning control policies. *Annual Review of Control, Robotics, and Autonomous Systems*, 6, 2023.

[19] Shengyi Huang, Rousslan Fernand Julien Dossa, Antonin Raffin, Anssi Kanervisto, and Weixun Wang. The 37 implementation details of proximal policy optimization. In *ICLR Blog Track*, 2022. `https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/`, accessed 2026-10-07.

[20] Maximilian Igl, Kamil Ciosek, Yingzhen Li, Sebastian Tschiatschek, Cheng Zhang, Sam Devlin, and Katja Hofmann. Generalization in reinforcement learning with selective noise injection and information bottleneck. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. arXiv:1910.12911.

[21] Rico Jonschkowski and Oliver Brock. Learning state representations with robotic priors. *Autonomous Robots*, 39(3):407–428, 2015.

[22] Rico Jonschkowski, Roland Hafner, Jonathan Scholz, and Martin Riedmiller. PVEs: Position-velocity encoders for unsupervised learning of structured state representations. arXiv:1705.09805 [cs.RO], 2017.

[23] Rudolf E. Kalman. Contributions to the theory of optimal control. *Boletín de la Sociedad Matemática Mexicana*, 5:102–119, 1960.

[24] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Champion-level drone racing using deep reinforcement learning. *Nature*, 620:982–987, 2023.

[25] Hiroaki Kitano. Biological robustness. *Nature Reviews Genetics*, 5(11):826–837, 2004.

[26] Vijay R. Konda and John N. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems 12 (NIPS 1999)*, pages 1008–1014. MIT Press, 2000.

[27] Alexander Michael Kranz. Source code: `low-dimensional-control-from-lqr-to-rl`. GitLab repository, `https://collaborating.tuhh.de/ccf6681/low-dimensional-control-from-lqr-to-rl`, 2026. Accessed 2026-10-25.

[28] Vladimír Kučera. The discrete Riccati equation of optimal control. *Kybernetika*, 8(5):430–447, 1972.

[29] Hojoon Lee, Dongyoon Hwang, Donghu Kim, Hyunseung Kim, Jun Jet Tai, Kaushik Subramanian, Peter R. Wurman, Jaegul Choo, Peter Stone, and Takuma Seno. SimBa: Simplicity bias for scaling up parameters in deep reinforcement learning. In *13th International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.09754.

[30] Timothée Lesort, Natalia Díaz-Rodríguez, Jean-François Goudou, and David Filliat. State representation learning for control: An overview. *Neural Networks*, 108:379–392, 2018.

[31] Chenliang Li, Junyu Leng, Jiaxiang Li, Youbang Sun, Shixiang Chen, Shahin Shahrampour, and Alfredo Garcia. AdaRL: Adaptive low-rank structures for robust policy learning under uncertainty. arXiv:2510.11899 [cs.LG], 2025.

[32] Chunyuan Li, Heerad Farkhoor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes. In *6th International Conference on Learning Representations (ICLR)*, 2018.

[33] Horia Mania, Aurelia Guy, and Benjamin Recht. Simple random search provides a competitive approach to reinforcement learning. In *Advances*

in Neural Information Processing Systems (NeurIPS), volume 31, 2018. arXiv:1803.07055.

[34] Tad McGeer. Stability and control of two-dimensional biped walking. Technical Report CSS-IS TR 88-01, Simon Fraser University, Burnaby, BC, Canada, 1988.

[35] Tad McGeer. Passive dynamic walking. *The International Journal of Robotics Research*, 9(2):62–82, 1990.

[36] Guido Montúfar, Keyan Ghazi-Zahedi, and Nihat Ay. Geometry and determinism of optimal stationary control in partially observable markov decision processes. *arXiv:1503.07206 [math.OC]*, 2015.

[37] Guido Montúfar, Keyan Ghazi-Zahedi, and Nihat Ay. A theory of cheap control in embodied systems. *PLOS Computational Biology*, 11(5):e1004427, 2015.

[38] Vincent C. Müller and Matej Hoffmann. What is morphological computation? on how the body contributes to cognition and control. *Artificial Life*, 23(1):1–24, 2017.

[39] Michal Nauman, Mateusz Ostaszewski, Krzysztof Jankowski, Piotr Miłoś, and Marek Cygan. Bigger, regularized, optimistic: Scaling for compute and sample-efficient continuous control. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. arXiv:2405.16158.

[40] OpenAI, Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Józefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, Jonas Schneider, Szymon Sidor, Josh Tobin, Peter Welinder, Lilian Weng, and Wojciech Zaremba. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.

[41] Youngsuk Park, Ryan A. Rossi, Zheng Wen, Gang Wu, and Handong Zhao. Structured policy iteration for linear quadratic regulator. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, 2020. arXiv:2007.06202.

[42] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810, 2018.

[43] Rolf Pfeifer and Josh C. Bongard. *How the Body Shapes the Way We Think: A New View of Intelligence*. MIT Press, Cambridge, MA, 2006.

[44] Aravind Rajeswaran, Kendall Lowrey, Emanuel Todorov, and Sham Kakade. Towards generalization and simplicity in continuous control. In *Advances in*

Neural Information Processing Systems (NeurIPS), volume 30, pages 6550–6561, 2017.

[45] Venkatesh G. Rao and Dennis S. Bernstein. Naive control of the double integrator. *IEEE Control Systems Magazine*, 21(5):86–97, 2001.

[46] Benjamin Recht. A tour of reinforcement learning: The view from continuous control. *Annual Review of Control, Robotics, and Autonomous Systems*, 2:253–279, 2019.

[47] Benjamin Recht, Maryam Fazel, and Pablo A. Parrilo. Guaranteed minimum-rank solutions of linear matrix equations via nuclear norm minimization. *SIAM Review*, 52(3):471–501, 2010.

[48] Nicholas Roy, Ingmar Posner, Tim Barfoot, Philippe Beaudoin, Yoshua Bengio, Jeannette Bohg, Oliver Brock, Isabelle Depatie, Dieter Fox, Dan Koditschek, Tomas Lozano-Perez, Vikash Mansinghka, Christopher Pal, Blake Richards, Dorsa Sadigh, Stefan Schaal, Gaurav Sukhatme, Denis Thérien, Marc Toussaint, and Michiel Van de Panne. From machine learning to robotics: Challenges and opportunities for embodied intelligence. arXiv:2110.15245 [cs.RO], 2021.

[49] Sergio Rozada and Antonio G. Marques. Matrix low-rank trust region policy optimization. arXiv:2405.17625 [cs.LG], 2024.

[50] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100, 2022. arXiv:2109.11978.

[51] John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, and Pieter Abbeel. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *PMLR*, pages 1889–1897, 2015. arXiv:1502.05477.

[52] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347 [cs.LG], 2017.

[53] SciPy documentation: Continuous-to-discrete conversion, `cont2discrete`. `https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.cont2discrete.html`, 2026. Accessed 2026-10-17.

[54] SciPy documentation: Discrete Lyapunov solver, `solve_discrete_lyapunov`. `https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.solve_discrete_lyapunov.html`, 2026. Accessed 2026-10-15.

[55] SIPPY Python library: Subspace identification for linear systems, N4SID. `https://github.com/CPCLAB-UNIPI/SIPPY`, 2026. Accessed 2026-10-15.

[56] Stable-Baselines3 documentation: Policy networks. `https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html`, 2026. Accessed 2026-10-07.

[57] Stable-Baselines3 documentation: PPO. `https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html`, 2026. Accessed 2026-10-07.

[58] Stable-Baselines3 documentation: Vectorized environments, VecNormalize. `https://stable-baselines3.readthedocs.io/en/master/guide/vec_envs.html`, 2026. Accessed 2026-10-14.

[59] Stefan Stojanovic, Yassir Jedra, and Alexandre Proutiere. Model-free low-rank reinforcement learning via leveraged entry-wise matrix estimation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

[60] Shokichi Takakura and Kazuhiro Sato. Structured output feedback control for linear quadratic regulator using policy gradient method. arXiv:2203.00289 [math.OC], 2023.

[61] Shahriar Talebi, Yang Zheng, Spencer Kraisler, Na Li, and Mehran Mesbahi. Policy optimization in control: Geometry and algorithmic implications. arXiv:2406.04243 [math.OC], 2024.

[62] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017.

[63] Aleksandar Todorov and Matthia Sabatelli. Learning in low-dimensional subspaces: Orthogonal bottlenecks for reinforcement learning. *Reinforcement Learning Journal*, 2026. arXiv:2605.26012.

[64] Emanuel Todorov and Michael I. Jordan. Optimal feedback control as a theory of motor coordination. *Nature Neuroscience*, 5(11):1226–1235, 2002.

[65] Peter Van Overschee and Bart De Moor. N4SID: Subspace algorithms for the identification of combined deterministic-stochastic systems. *Automatica*, 30(1):75–93, 1994.

[66] Manuel Watter, Jost Tobias Springenberg, Joschka Boedecker, and Martin Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015. arXiv:1506.07365.

[67] Herbert Werner. Control systems theory and design. Lecture notes, Institute of Control Systems, Hamburg University of Technology, 2021.

[68] Herbert Werner. Optimal and robust control. Lecture notes, Institute of Control Systems, Hamburg University of Technology, 2021.

[69] Jingxi Xu, Bruce Lee, Nikolai Matni, and Dinesh Jayaraman. How are learned perception-based controllers impacted by the limits of robust control? In *Proceedings of the 3rd Conference on Learning for Dynamics and Control (L4DC)*, volume 144 of *PMLR*, 2021. arXiv:2104.00827.

[70] Keyan Zahedi and Nihat Ay. Quantifying morphological computation. *Entropy*, 15(5):1887–1915, 2013.

[71] Kaiqing Zhang, Bin Hu, and Tamer Başar. Policy optimization for $\mathcal{H}_2$ linear control with $\mathcal{H}_\infty$ robustness guarantee: Implicit regularization and global convergence. *SIAM Journal on Control and Optimization*, 59(6):4081–4109, 2021.

[72] Feiran Zhao, Florian Dörfler, Alessandro Chiuso, and Keyou You. Data-enabled policy optimization for direct adaptive learning of the LQR. arXiv:2401.14871 [math.OC], 2024.

Verwendete Hilfsmittel

Für die Erstellung dieser Bachelorarbeit wurde Generative KI innerhalb der im Folgenden genannten Hilfsmittel für die angeführten Zwecke verwendet. Der Autor hat jegliche produzierten Ausgaben ausführlich geprüft, und übernimmt Verantwortung für diese. Die Selbständigkeit der produzierten Ideen, sowie deren Umsetzung, waren zu jeder Zeit gewährleistet.

Im Detail wurde genutzt:

GitHub Copilot, integriert in Visual Studio Code, für:

- Automatische Vervollständigung von Codezeilen ("Inline Suggestions") zur Unterstützung bei Latex Formatierung, sowie der python-basierten Experimente

Claude Code, mit den Modellen Opus 4.8, Opus 5 und Opus 5.5, für:

- Korrektur von Grammatik, Zeichensetzung und Tippfehlern
- Formatierung von Latex Code
- Quellenvorschläge
- Automatisches Einpflegen meiner Quellen in die Latex Bibliografie
- Erstellung von Latex-basierten Schema Visualisierungen von Code Implementierungen
- Unterstützung bei der Erstellung von Skripten für die automatische Generierung von Figuren und Tabellen basierend auf den Experiment Ergebnissen
- Erstellung einer Anleitung, sowie eines Skripts für vereinfachte Datentransfers zwischen dem TUHH HPC Cluster und meinem lokalen Gerät
- Anmerkung von Inkonsistenzen und Logik Fehlern in der textlichen Ausarbeitung
- Herausfordern meiner Standpunkte und Umsetzung ("Sparring")
- Erkennen von Fehlern im Python Code

Erklärung der Urheberschaft

Ich versichere an Eides statt, dass ich die Bachelor's Thesis im Studiengang Data Science selbstständig verfasst und keine anderen als die angegebenen Hilfsmittel – insbesondere keine im Quellenverzeichnis nicht benannten Internet-Quellen – benutzt habe. Es wurden keine Quellen genutzt, die nicht genannt wurden. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen wurden, sind als solche kenntlich gemacht. Ich versichere weiterhin, dass ich die Arbeit vorher nicht in einem anderen Prüfungsverfahren eingereicht habe und die eingereichte schriftliche Fassung der auf dem elektronischen Speichermedium entspricht.

Ort, Datum Unterschrift

Erklärung zur Veröffentlichung

Ich erkläre mein Einverständnis mit der Einstellung dieser Bachelor's Thesis in den Bestand der Bibliothek.

Ort, Datum Unterschrift